INGEGNERIA DEL SOFTWARE

Santo Lo Bianco

Introduzione

Nel XXI secolo il prodotto più importante e anche quello che genera più ricchezza(vedasi i miliardari odierni) è il software. Il software può essere inteso:

- prodotto fine a sé stesso(videogiochi, software di editing video o foto);
- veicolo di distribuzione dei prodotti(siti shopping);

In un mondo super-connesso e super-veloce il software è veicolo dell'informazione, infatti:

- elabora dati personali;
- apre la porta alle reti informative mondiali;
- permette di creare o usufruire di qualsiasi informazione in modo istantaneo ed esteso.

La storia del software è molto legata all'hardware e come visto nei sistemi operativi ci sono 4 epoche:

- **Prima era**: elaborazione batch -> programmi sviluppati da singoli per sé stessi o al massimo per l'azienda che aveva questi sistemi;
- **Seconda era**: multi-programmati e multiutente -> tecniche interattive uomo-macchina -> software sviluppato per essere distribuito sul mercato;
- **Terza era**: nascono i sistemi distribuiti -> le reti, globali o locali, e le esigenze di accesso "istantaneo" a dati remoti accrescono la pressione sugli sviluppatori;
- **Quarta era**: diffusione dei microprocessori -> prodotti intelligenti, telefoni e pc -> software non più di nicchia ma ciò che muove il mondo.

Adesso stiamo vivendo l'era dell'AI, perché dovremmo studiare metodi ingegneristici applicati alla produzione del software? Il motivo è prettamente legato al realizzare sistemi corretti, mantenibili e scalabili.

Infatti, è importante capire il codice generato e che l'AI non è in grado di risolvere problemi perché riutilizza soluzioni. Nuovi problemi non potrebbero quindi esser risolti dall'AI.

Ogni programma deve avere un apposito documento chiamato **documento dei requisiti**, ossia una serie di aspetti che l'applicazione deve avere. Se non esiste, vuol dire che non sappiamo cosa vuole fare il nostro programma(serve anche in fase di testing -> dobbiamo accertarci che il programma rispetti determinate condizioni).

Il software a differenza dell'hardware è **modificabile**, infatti, non ha parti fisiche. Se il software è di successo non viene modificato, a meno che:

- si deve riadattare alla realtà che cambia;
- richieste di estensione;
- adattamento a nuove piattaforme hardware.

Il software non si usura ma si deteriora con le modifiche apportate.

Le tecniche di ingegneria del SW provano a produrre software entro tempi previsti, costi preventivati e con una qualità accettabile. La **qualità** si può misurare attraverso i seguenti criteri:

- **correttezza** → ci assicuriamo che il nostro software funzioni correttamente. Verranno fatti dei test per verificare se il software sviluppa correttamente i punti indicati all'interno del documento dei requisiti. Quindi, si deve soddisfare il cliente in primis e il documento dei requisiti diventa un vero e proprio contratto tra il richiedente e l'azienda che produce software;
- **usabilità** → l'utilizzo del software è fortemente intuitivo(user friendly) e gli utenti riescono a sfruttare in modo efficiente il servizio offerto;
- **manutenibilità** → quanto il software popolato da errori può essere facilmente corretto. Ci possono essere varie misure quali il **tempo medio impiegato per la correzione** oppure il costo della correzione che va calcolato rispetto al tempo;
- **efficienza** → nessuno spreco di risorse(esempio: programma che carica grandi matrici in memoria centrale e calcola la covarianza degli elementi → se non è fatto con il chunking, si potrebbe venire a creare un collo di bottiglia in RAM);
- **affidabilità(dependability)** → l'affidabilità è la misura di correttezza del funzionamento di un SW anche in un lungo periodo. Quindi un SW deve mantenere un certo grado di affidabilità. La sicurezza in un software possiamo dividerla in:
    - **sicurezza** → si proteggono i dati da eventuali intrusi;
    - **safety** →un'eventuale falla del sistema non causa problemi a persone, cose o animali.

In Java a differenza di C/C++ si **programma per interfacce e non per implementazioni**, ossia si sviluppa il codice non creando una classe concreta bensì un'interfaccia di essa. Tutto ruota intorno all'astrazione del problema e non all'implementazione del problema stesso.

Si crea quindi un'interfaccia, ad esempio l'interfaccia List avente i metodi get, put, size ecc., e infine si associa un'implementazione come ArrayList. Questo permette di avere una più facile manutenibilità, sostituendo solo l'implementazione ed evitando la propagazione di modifiche in classi che utilizzano la classe oggetto di modifica.

```
class Servizio {
    Database db = new MySQLDatabase();
}
```

PROGRAMMAZIONE PER IMPLEMENTAZIONE
L'oggetto generato sarà soltanto di tipo **MySQLDatabase**. Se un giorno lo volessimo cambiare con **PostgresDatabase** dovremmo cambiare la classe Servizio.

```
class Servizio {
    Database db;

    Servizio(Database db) {
        this.db = db;
    }
```

PROGRAMMAZIONE PER INTERFACCE
In questo caso, ogni qualvolta che richiamiamo il metodo per inizializzare il database, dobbiamo passare il tipo di database(l'implementazione della classe), in modo tale che non si debba toccare la classe Servizio.

In questo modo possiamo isolare le componenti e testarle anche in ambienti simulati.

Addentriamoci all'interno dell'ingegneria del software definendo una prima nozione di codice scritto male. Quando il codice è scritto male, disordinato e difficile viene denominato in modo informale **Spaghetti Code**. Il codice sotto proposto ne è un esempio, infatti è un codice monolitico che non presenta interfacce, ma un'unica implementazione che svolge varie mansioni.

```java
public float calcola(final String n){
    importi.clear();
    try(BufferedReader f = new BufferedReader(new FileReader(new File(n))){
        totale = 0;
        massimo = 0;
        String riga;
        while((riga = f.readLine()) != null){
            if(!importi.contains(riga)){
                importi.add(riga);
                try{
                    Float valore = Float.parseFloat(riga);

                    totale += valore;
                    if(valore > massimo){
                        massimo = valore;
                    }
                }catch(NumberFormatException e){
                    System.err.println("ERRORE DI FORMATTAZIONE NELLA RIGA " +riga);
                }
            }
        }
        f.close();
    } catch(IOException e){
        System.err.println("Errore di I/O: " +e.getMessage());
    }
    return totale;
}
```

Il metodo ***calcola*** è un **anti-pattern**, ossia sembra che risolva il problema in modo veloce e intuitivo, ma produce problemi nascosti quali:

- difficoltà di comprensione;
- difficoltà nella modifica, dato che si coinvolgono molti fattori.

Questo codice monolitico quindi è inutile, dato che non è riutilizzabile e non si può verificarne la correttezza.
Possiamo quindi dividerlo in più funzioni.

In questo modo otteniamo un codice più vicino al paradigma della programmazione ad oggetti. Ogni funzione ha la propria responsabilità e posso richiamarle più volte singolarmente.
Inoltre, si è seguito un approccio o paradigma **query** e **command**, dove:

- query → restituiscono dei valori non modificando lo stato interno dell'istanza della classe(calculate, searchmax);
- command → non restituiscono nulla, cambiano lo stato dell'istanza in modo silenzioso(removedupl, converti).

```java
private void removedupl(){
    importiValidi.clear();
    for(String s: importiLetti){
        if(!importiLetti.contains(s)){
            importiValidi.add(s);
        }
    }
}

private void converti(){
    valori.clear();
    for(String s: importiValidi){
        try{
            valori.add(Float.parseFloat(s));
        }catch(NumberFormatException e){
            System.err.println("ERRORE DI FORMATTAZIONE NELLA RIGA "+s);
        }
    }
}

public float calculate(){
    float totale = 0;
    for(float v : valori) totale+=v;
    return totale;
}

public float searchmax(){
    float massimo = valori.get(index: 0);
    for(float v: valori){
```

ENHANCED FOR

Forma semplificata del costrutto for che permette di iterare su elementi di una lista.

Una metrica utile a capire se il codice non è uno Spaghetti Code potrebbe essere la **LOC**, ossia le **linee di codice** terminate dal separatore(;). Più bassa è la media di linee per metodo e più aumentano le probabilità che non siamo davanti ad un codice scritto seguendo un anti-pattern.

I miglioramenti applicabili in generale sulla maggior parte dei codici sono:

- la **separazione delle operazioni**, ciascuna con il suo metodo;
- **costruzione di astrazioni**, con metodi di basso livello richiamati da metodi di alto livello. Lo stesso approccio si applica alle classi.

```
private void readfile(final String n){
    importiLetti.clear();
    try(BufferedReader f = new BufferedReader(new FileReader((new File(n))))){
        String riga;
        while ((riga = f.readLine()) != null) {
            importiLetti.add(riga);
        }
        f.close();
    }catch( IOException e){
        System.err.println("ERRORE DI I/O: "+ e.getMessage());
    }
}
```

```
public CalcolaImporti(final String n){
    this.readfile(n);
    this.removedupl();
    this.converti();
}
```

- **seguire paradigma query-command**. Si separano quindi i metodi in metodi che restituiscono qualcosa senza modificare lo stato del sistema e viceversa.
- **implementare opportuni metodi testNomeFunzione**. I metodi test servono ad eseguire il codice in certe condizioni e valutano se il risultato fornito è corretto.

Creando quindi un ambiente adatto al test della funzione, possiamo verificare se funziona correttamente. Ad esempio:

```
private void initList(){
    values.clear();
    values.add(e: 1);
    values.add(e: 2);
    values.add(e: 3);
    values.add(e: 4);
    values.add(e: 5);
}
```

```
public void testSomma(){
    this.initList();
    int sum = 0;
    for(int v: values) sum+=v;
    if(sum == 15)
        System.out.println(x: "OK Metodo somma funzionante");
    else
        System.err.print(s: "ERRORE Metodo somma non funzionante");
}

public int somma(){
    System.out.println(x: "SOMMA");
    this.readStandardIn();
    int sum = 0;
    for(int v: values) sum += v;
    return sum;
}
```

Prima di eseguire la funzione somma sui dati presi da terminale, si verifica su una lista la cui somma è uguale a 5.

È possibile quindi verificare se la funzione soddisfa i requisiti ergo permette di ottenere la qualità del sistema software.

Nota importante: i test devono essere indipendenti tra loro e autovalutarsi, confrontando il risultato con un **valore atteso**. L'autovalutazione è importante al fine di aiutare il programmatore a capire se l'implementazione è giusta.

Il testing non è una sola fase eseguita durante lo sviluppo, bensì è anche una buona pratica di sviluppo. Infatti, si può seguire l'approccio **Test Driven Devolpment**, ossia quando si programma:

1. Prima si esegue il test;
2. Poi si scrive il codice dell'implementazione;

Questa è una tecnica di programmazione fortemente guidata dai test dove si può seguire una formalizzazione:

RED → **GREEN** → **REFACTORING**

Nel dettaglio:

- **RED** → si implementano e modificano i metodi e le classi per il testing(ancora non si è trovata la soluzione);
- **GREEN** → si scrive un codice grezzo per far passare il test;
- **REFACTORING** → migliorare la struttura del codice;

Ricordando sempre che i metodi vengono scritti secondo il **Principio di Singola Responsabilità**, ovvero se in futuro si vuole cambiare il codice si deve avere un solo motivo(svolge un solo compito).

I principi di ingegneria del software possono essere riassunti con questi concetti:

- **KISS(keep it simple stupid)** → il sistema deve essere mantenuto il più semplice e leggibile, dato che la complessità è nemica stessa del software. Classi chiare e metodi i cui nomi descrivono l'obiettivo da raggiungere. Ad esempio:
    - classe che contiene le funzioni e i parametri utilizzati per un rettangolo -> rettangolo;
    - metodo che contiene implementa una lettura da un file -> read;
- **DRY(Don't repeat yourself)** → evitare ripetizione nel codice;
- **YAGNI(You ain't gonna need it)** → non si implementano funzionalità che non sono richieste;
- **LOD(Law of Demeter)** → si mira a ridurre l'accoppiamento tra classi. Un oggetto, quindi, dovrebbe interagire soltanto con:
    - i suoi attributi;
    - oggetti creati nella classe;
    - oggetti passati come parametri;

Si possono seguire anche dei principi per i test, ed essi sono:

- **FIRST**→ ossia devono essere:
    - (FAST) veloci da eseguire;
    - (INDEPENDENT) devono essere isolati;
    - (SELF-VALIDATE) devono poter fare autovalutazioni;
    - (REPEATABLE) devono poter essere tipetuti;
    - (TIMELY) devono essere sviluppati contemporaneamente alle implementazioni o prima;
- **TDD(accennato prima)**;
- **BDD(behavior-driven development)**: è un'evoluzione del tdd, dove il focus non è **tecnico** ma sul **comportamento** del metodo.

REFACTORING

Il **refactoring** è il processo che non cambia il comportamento esterno di un sistema software, bensì che **migliora la struttura interna**. Questo processo viene adoperato non alla fine, ma durante l'implementazione dei metodi e delle classi. Si ricorda che la risoluzione di un problema, che cambia anche l'output del programma, non è il refactoring, quest'ultimo si fa solo dopo che si è arrivati ad una soluzione.

COMPORTAMENTO POSITIVO: dopo aver sviluppato una funzione o un piccolo blocco di codice, si esegue subito l'eventuale refactoring per migliorarne la struttura.

COMPORTAMENTO NEGATIVO: si implementa un'intera classe con molti metodi e solo alla fine si esegue il refactoring, accumulando codice poco strutturato e più difficile da migliorare.

Un codice ben fatto e una progettazione adeguata non si possono ottenere fin dall'inizio, per questo motivo si effettua il refactoring, affinando quello che già si è sviluppato.

I vantaggi che si ottengono da un buon refactoring sono:

DIMENSIONE DEL CODICE RIDOTTA	STRUTTURE COMPLICATE SNELLITE	SI EVITA IL DEBITO TECNICO
Spesso dopo che si applica il refactoring si riduce il codice.	Attraverso il refactoring si possono ottenere strutture più semplici, facili da capire e mantenere.	Il **debito tecnico** è l'insieme delle scelte di progettazione o **implementazione rapide ma non ottimali**. Come un debito finanziario ci permette di ottenere un **vantaggio iniziale**, che poi pagheremo quando si devono fare correzioni e refactoring. Attraverso il refactoring si riduce questo debito.

Oltre ciò, il refactoring viene fatto per **scoprire i bug** e **comprendere in modo profondo il codice**. Quindi, il benefit finale è la **programmazione più veloce** in fase di re-implementazione di metodi e classi. Ovviamente, dopo che si è re-implementata la struttura interna si deve verificare che tutti i test siano ancora superati. Se un test non è superato, il refactoring ha compromesso qualcosa che prima funzionava.

Si può fare refactoring all'infinito? Ci sono delle condizioni che ci fanno capire che è necessaria l'applicazione di questa fase? Esistono degli indizi che ci permettono di stabilire se si può o non può applicare questa fase?

Si, il refactoring principalmente viene applicato quando si presentano queste condizioni:

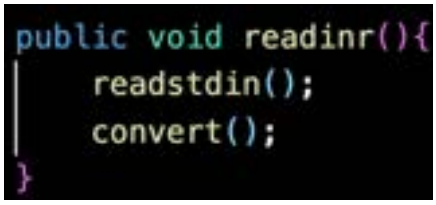
CODICE DUPLICATO	**METODI LUNGHI**	**CLASSI MOLTO GRANDI**	**LISTA PARAMETRI (DEI METODI) LUNGA**	**CAMBIAMENTI DIVERSI**
Lo stesso blocco di codice si trova in varie parti del codice stesso.	Metodi con troppe righe di codice o commenti. Molti commenti indicano che il metodo fa molte cose.	Classi con tanti parametri e funzioni non vanno bene, in nome della semplicità la classe viene divisa in classi più piccole.	Se la lista di parametri nelle chiamate di funzione è lunga, allora si sostituisce la stessa lista con un oggetto.	Se nel metodo applichiamo cambiamenti diversi allora lo stesso fa cose diverse(si può rendere più semplice).

Una tecnica utilizzata quando siamo in presenza di codice lungo o duplicato è la tecnica **Estrai Metodo**. Questa tecnica serve per **raggruppare un codice** e far diventare quel frammento un **metodo** il cui nome spiega lo scopo.

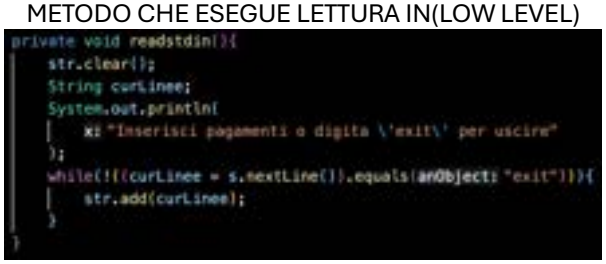
La scelta del nome dei metodi piccoli, che verranno richiamati nei metodi di alto livello, è importante e nel metodo che li contiene si genereranno delle sequenze di commenti(con i nomi dei metodi piccoli).

CODICE LUNGO

METODO DI ALTO LIVELLO



METODO CHE ESEGUE LETTURA IN(LOW LEVEL)



METODO CONVERSIONE(LOW L)

Un metodo lungo come **readinput** può essere suddiviso in tre metodi. Dopo il refactoring si ottiene:

- **readinr(r sta per refactored)** → richiama i metodi a basso livello. Attraverso una buona scelta dei nomi è possibile capire cosa fa il metodo principale.
- **readstdin** → metodo che legge dallo standard input finché la riga presa in input non è uguale a 'exit'.
- **convert** → metodo che converte le stringhe di input in interi.

Quando il codice estratto ha **variabili locali del metodo sorgente** bisogna capire la natura di quelle variabili e agire opportunamente.

- Se le variabili si utilizzano soltanto nel codice estratto allora diventeranno variabili temporanee.
- Se le variabili si utilizzano anche nel metodo sorgente allora si restituisce il valore processato e si assegna alla variabile locale del metodo sorgente.

Quando si hanno variabili temporanee, che vengono utilizzate ad esempio per la stampa, si possono sostituire con metodi query, ossia con metodi che rendono molto più modulare il codice e che calcolano direttamente il valore.

Ad esempio, in una classe rettangolo:

```
double area = base*altezza;
```

↓

```
private double area(){
        return base*altezza;
}
```

Così facendo il valore non apparterrà solo alla variabile temporanea, ma può essere esteso anche ad altri metodi, a patto che venga assegnato una sola volta.

Un altro principio da seguire quando si lavora con le variabili temporanee è quello di rispettare la condizione di responsabilità, ossia devono avere una singola responsabilità.

L'**assegnamento alla variabile deve avvenire una sola volta**, se avviene più di una volta e non è una variabile di cumulazione, vuol dire che la stessa ha più di una responsabilità.

```
double temp = 2 * (height + width);
System.out.println(temp);
temp = height * width;
System.out.println(temp);
```

↓

```
final double perim = 2 * (height + width);
System.out.println(perim);
final double area = height * width;
System.out.println(area);
```

In questo caso l'utilizzo di due variabili temporanee, una per ogni specifica responsabilità, rende la lettura del codice meno confusionaria. Per evitare che si assegni nuovamente un valore, la variabile viene dichiarata come **final**(costante).

Quando un metodo utilizza più attributi e metodi di una classe di una classe diversa da quella in cui è definito, si può **spostare il metodo**. Per lo spostamento del metodo si stabiliscono queste fasi:

- si farà una copia del metodo nella classe che sarà di destinazione;
- si fanno eventuali collegamenti ai metodi e agli attributi della nuova classe di appartenenza;
- si rimuove il vecchio metodo nella classe di partenza.

Questo fenomeno si ottiene principalmente quando due classi sono accoppiate e la soluzione è spostare il metodo per chiarificare bene le responsabilità delle due classi.

Durante il trasferimento si mettono in dubbio anche gli attributi e i metodi utilizzati dal metodo. Infatti, se sono utilizzati dal metodo spostato si spostano di conseguenza.

Ovviamente deve essere effettuato un controllo in eventuali sottoclassi o superclassi.

Tutto ciò che si è detto sui metodi, sull'esigenza di averli semplici e con una sola responsabilità può essere trasposto anche alle classi. Infatti, si utilizza la tecnica di **estrazione della classe** nel caso in cui si ha una grande classe difficile da comprendere.

Le motivazioni potrebbero anche essere legate a:

- sottoclassi implementano poche caratteristiche della classe;
- sotto-insieme di dati o metodi vengono processati insieme e separatamente da tutto il resto.

TECNICHE STUDIATE:

- ESTRAZIONE METODO;
- SOSTITUZIONE VARIABILI TEMPORANEE CON METODI QUERY;
- DIVISIONE VARIABILI TEMPORANEE;
- SPOSTA METODO;
- ESTRAI CLASSE.

JUnit

JUnit è un **framework**(struttura software riutilizzabile) **di testing per Java** che permette di scrivere ed eseguire test automatici per verificare il corretto funzionamento del codice.

Attraverso questo strumento quindi si può testare una piccola parte di un programma evitando di lasciare al programmatore stesso di effettuare la verifica. Infatti, prima si testavano i programmi con System.out.println() e attraverso l'output il programmatore verificava se la funzione elaborava i dati correttamente.

I problemi legati ai test eseguiti dagli sviluppatori sono: lentezza, poca affidabilità, automatizzazione impossibile e poca manutenibilità.

Quello che garantisce JUnit è:

- **motore di esecuzione dei test** → legge le classi e i metodi di test, genera automaticamente test e gestisce i risultati;
- **API per i test** → insieme di classi e annotazioni per scrivere i test.

Le **annotazioni** in Java sono **metadati associati** a elementi del codice, che descrivono la natura di un metodoo di una porzione di codice. Non cambia il comportamento del programma, bensì informa il compilatore con che tipo di codice sta lavorando.

Ad esempio, quando si sovraccaricano i metodi si utilizza **@Override** che indica al compilatore che la funzione in questione sovrascrive quella della superclasse.

Le annotazioni sono utilizzate anche per i metodi della classe di test, infatti si ha:

- **@Test** ⇒ è l'annotazione che identifica un metodo di test. Quando JUnit incontra un metodo annotato con la suddetta annotazione, lo esegue automaticamente e ne valuta il risultato.
  Le caratteristiche importanti dei metodi annotati con @Test sono:
  - deve essere void;
  - non deve essere private;
  - non deve avere parametri.

  Se il test fallisce allora viene lanciata un'eccezione non prevista.

```
@Test
void verificaSomma() {
    Calcolatrice c = new Calcolatrice();
    assertEquals(5, c.somma(2,3));
}
```

  Ad esempio, in questo modo si può verificare se il metodo somma della classe calcolatrice produce un risultato giusto, ergo il metodo funziona.
- **@BeforeEach** ⇒ è l'annotazione che identifica il metodo che **deve essere eseguito prima di ogni altro metodo annotato con @Test**. Serve quindi a garantire isolamento dei test e uno stato inizialmente pulito.

```
@BeforeEach
void setup() {
    c = new Calcolatrice();
}

@Test
void testSomma() {
    assertEquals(5, c.somma(2,3));
}

@Test
void testSottrazione() {
    assertEquals(1, c.sottrai(3,2));
}
```

  L'**ordine reale di esecuzione** è il seguente:

```
setup()
testSomma()

setup()
testSottrazione()
```

- **@BeforeAll** ⟹ indica un metodo che **viene eseguito una sola volta prima di tutti i test** della classe. Si utilizza quindi quando:
  - l'inizializzazione è costosa;
  - condivisa tra tutti i test;
  - immutabile durante i test;

```
@BeforeAll
static void init() {
    db = new Database();
    db.connect();
}

@Test
void testConnessione() {
    assertTrue(db.isConnected());
}
```

Per verificare il comportamento del codice si utilizzano i metodi della classe **Assertions**. Le **asserzioni** sono dei controlli che verificano se delle condizioni sono vere.

- Se la condizione è falsa → test fallito.
- Se la condizione è vera → test passato.

L'eccezione lanciata se la condizione non è soddisfatta è **AssertionFailedError**. Esistono svariati tipi di asserzioni, ad esempio si hanno:

- **assertEquals(valore atteso, valore ottenuto)** → verifica l'uguaglianza tra il valore atteso e quello ottenuto → **assertEquals(5, c.somma(3,2))**;
- **assertTrue(condition)** → verifica che una condizione sia vera → assertTrue(risultato > 0);
- **assertNull(object)** → verifica che un oggetto sia null, utile **quando i risultati dovrebbero essere assenti** o si deve avere un principio di inizializzazione dei valori → **assertNull(utente.getMail())**;

È consigliato seguire la **convenzione standard di posizionamento delle cartelle di Maven**, dato che se si rispetta questa struttura standard non si deve configurare nulla.

Maven, sviluppato da Apache, sa già dove trovare il codice e i test, inoltre, sa cosa compilare e includere nel file finale. Questo standard prevede che:

- le classi dell'applicazione si trovino dentro la cartella **src/main/java**;
- il codice di test delle relative classi si trovi dentro la cartella **src/test/java**;
- eventuali moduli siano dentro la cartella java.

Le funzionalità principali di Maven sono:

- **compilazione automatica** → trasforma il codice sorgente in bytecode;
- **scarica automaticamente le librerie esterne** da repository remoti;
- **integra** strumenti adatti ad eseguire i **test automatici**;
- usa **convenzioni di struttura dei progetti**, rendendo tutti i progetti simili e facilmente comprensibili.

Il file **pom.xml(project object model)** è il cuore del progetto Maven, dato che descrive:

- **identità del progetto** → **groupId**(chi sviluppa), **artifactId**(cosa sviluppa) e **version**;
- **dipendenze** → librerie esterne utilizzate dal progetto;
- **plugin** → permette di trovare le classi di test e generare report;
- **ciclo di vita**;

Un'annotazione fondamentale per eseguire un metodo più volte con valori in input diversi è **@ParametrizedTest**. Per fornire i valori si possono avere vari modi, uno tra questi e **@CsvSource({value, value,...})**.

```
@ParameterizedTest
@CsvSource({"-1", "1", "5"})
void testPositive(int number) {
  assertTrue(number >= 0);
}
```

Quando si effettuano i test con JUnit, i metodi richiamano effettivamente il codice dell'applicazione. Quindi se viene richiamata una funzione della classe, le **linee di codice associate a quella funzione verranno eseguite** e quindi **coperte in fase di testing**.

La **copertura del codice** o **code coverage** è il **rapporto tra le linee di codice eseguite** e **le linee totali**. Ad esempio, una copertura del 70% indica che il 70% delle linee di codice sono state eseguite dai test.

La formula quindi è la seguente:

$$\boldsymbol{Code\ coverage}(\%) = \frac{\boldsymbol{linee\ eseguite\ dai\ test}}{\boldsymbol{linee\ totali\ del\ codice}} \times \mathbf{100}$$

Bisogna però tracciare le linee che vengono eseguite durante i test e questo si fa attraverso strumenti specifici. Ad esempio, in VsCode attraverso l'estensione **Java Test Runner + Coverage Gutters**, si possono eseguire i test JUnit e visualizzare le linee coperte(verde) e quelle non coperte(rosso).

Una volta istallato Maven, si possono creare progetti già preimpostati con il comando **mvn archetype:generate**. Questo creerà le cartelle contenenti i test e le classi da testare. Inoltre, c'è un insieme di comandi che vengono utilizzati per gestire le fasi del ciclo di vita di Maven.

I comandi Maven sono i seguenti:

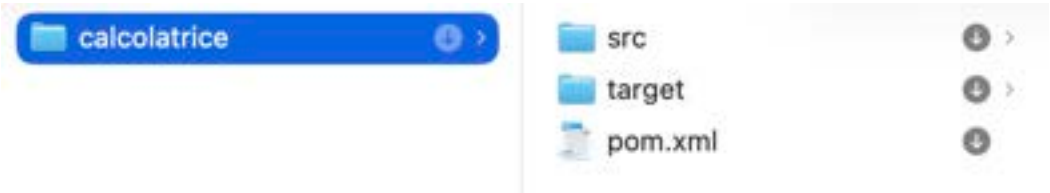
- **mvn test** → esegue i test di unità definiti in src/test/java e ritorna un report.
- **mvn compile** → compila il codice.
- **mvn package** → crea un pacchetto .jar, ossia un file compresso simile al file .zip, contenente tutto il codice compilato, eventuali librarie o risorse e informazioni di configurazione.
- **mvn verify** → effettua dei controlli di qualità prima di creare un pacchetto.

calcolatrice

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.santo		100%		100%	0	6	0	7	0	5	0	1
Total	0 of 26	100%	0 of 2	100%	0	6	0	7	0	5	0	1

Con **JaCoCo** configurato, Maven compila il codice, esegue i test unitari e raccoglie informazioni su quali parti del codice sono state eseguite.

DISPOSIZIONE STANDARD DI MAVEN



Di seguito è riportato un esempio di test effettuati sulle funzioni.

```
@Test
void testAdd() {
    assertEquals(5, calc.add(a: 2, b: 3));
    assertEquals(-1, calc.add(a: 2, -3));
    assertEquals(0, calc.add(a: 0, b: 0));
}

@Test
void testSubtract() {
    assertEquals(-1, calc.subtract(a: 2, b: 3));
    assertEquals(5, calc.subtract(a: 2, -3));
    assertEquals(0, calc.subtract(a: 0, b: 0));
}

@Test
void testMultiply() {
    assertEquals(6, calc.multiply(a: 2, b: 3));
    assertEquals(-6, calc.multiply(a: 2, -3));
    assertEquals(0, calc.multiply(a: 0, b: 5));
}

@Test
void testDivide() {
    assertEquals(2, calc.divide(a: 6, b: 3));
    assertEquals(-2, calc.divide(a: 6, -3));
    assertEquals(0, calc.divide(a: 0, b: 5));
}
```

CLASSI

La **programmazione orientata agli oggetti** è il principio fondamentale di Java. Essa permette di modellare un programma come un insieme di oggetti che interagiscono tra loro.

VANTAGGI: facile comprensione di cosa fa il programma e della natura profonda delle operazioni.

Si modellano entità del mondo reale attraverso la definizione di classi, e gli oggetti che sono istanze delle classi interagiscono tra loro per la soluzione del problema.

REALTA'

SEMPLIFICAZIONE MEDIANTE CLASSE

STUDIO DEI COMPORTAMENTI E DELLE CARATTERISTICHE DEGLI OGGETTI

Nell'esempio mostrato, vediamo come avviene lo studio della realtà e il passaggio degli oggetti/soggetti alle classi. Le due classi sono simili, infatti, hanno dei metodi in comune(mangia(), beve(), dorme()) e altri specifici per classe.

Poiché le due classi condividono comportamenti simili è possibile creare una classe più generale, ad esempio Animale, che contiene elementi comuni. Da questa classe si possono derivare le classi Cane e Gatto, senza copiare la classe originaria e modificarne i metodi(proliferazione delle classi).

Il **riuso delle classi** deve avvenire senza modificare il codice esistente, ma deve essere la via più semplice che il programmatore intraprende.

Attraverso l'ereditarietà delle classi è possibile definire una nuova classe indicando solo cosa ha in più rispetto una classe esistente.

Analizzando l'esempio precedente:

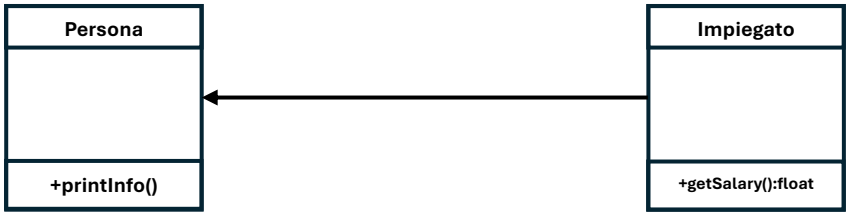
- classe Animale(superclasse)
  - ATTRIBUTI GENERICI
    - specie;
    - anni;
    - nome;
    - altezza;
    - peso;
  - METODI GENERICI
    - mangia();
    - beve();
- classe Cane(sottoclasse)
  - ATTRIBUTI
    - razza;
    - colore;
    - TipoPelo;
  - METODI
    - abbaia();
    - scodinzola();

La classe Cane erediterà gli attributi e metodi della classe Animale.

In Java per indicare che una classe è sottoclasse di un'altra si utilizza la parola chiave **extends**. Quindi, possiamo indicare che Cane è sottoclasse di Animale nel seguente modo:

```
public class Cane extends Animale{...}
```

La sottoclasse eredita metodi e attributi della superclasse e può usarli come se fossero definiti localmente. Inoltre, può aggiungere altri metodi o ridefinire(re-implementare) metodi della superclasse.



Ma quali metodi e attributi erediterà la sottoclasse?

- metodi o attributi **private** sono visibili solo alla classe superclasse e non alle sottoclassi;
- metodi o attributi **protected** sono visibili anche alle sottoclassi ma non a tutte le altre classi;
- metodi o attributi **public** sono visibili a tutti.

Il nome scelto delle sottoclassi dovrebbe avere un significato semantico simile eccetto per cosa si differenzia; invece, una superclasse deve avere nomi brevi e coincisi.

L'ereditarietà permette di creare una **classificazione tra tipi**, infatti: una sottoclasse è anche ciò che è una superclasse. Il **principio di Liskov** stabilisce che le sottoclassi devono poter sostituire le classi base senza alterare la correttezza di un programma.

C'è una tipologia di classi che non ammette l'ereditarietà tra classi, perché sono implicitamente costanti(final). La tipologia di queste classi immutabili assume il nome di **record**, e la sintassi è la seguente:

- nome della classe;
- parametri passati alla classe;

Vengono poi forniti automaticamente:

- costruttore che assegna tutti i campi;
- metodi getter per i vari camp;

Quindi, è semplicemente un **aggregatore di dati costanti**, per questo motivo i valori vengono assegnati una sola volta e non possono essere più manipolati.

```
public class ex6 {
    Run | Debug
    public static void main(String[] args) {
        Studente s1 = new Studente(nome: "Mario", media: 27.5f);
        Studente s2 = new Studente(nome: "Anna", media: 30.0f);

        Corso c1 = new Corso(titolo: "Programmazione 2", cfu: 8);
        Corso c2 = new Corso(titolo: "Ingegneria del Software", cfu: 6);

        System.out.println(s1.nome() + " media: " + s1.media());
        System.out.println(s2.nome() + " media: " + s2.media());
        System.out.println("Corso: " + c1.titolo() + " CFU: " + c1.cfu());
        System.out.println("Corso: " + c2.titolo() + " CFU: " + c2.cfu());
    }
}


record Studente(String nome, float media) { }
record Corso(String titolo, int cfu) { }
```

Una classe di tipo record non può fungere né da superclasse e né da sottoclasse. Si possono però aggiungere metodi(che non modificano i campi) e implementare **interfacce**.

Le **interfacce** sono dei **contenitori di metodi astratti**: definiscono cosa deve fare un oggetto, senza fornire l'implementazione. La classe (o il record) che implementa l'interfaccia deve quindi fornire l'implementazione dei metodi indicati.

```
interface Valutabile{
    boolean promosso();
}

record Studente(String nome, float media) implements Valutabile{
    public boolean promosso(){
        return media > 30;
    }
}
```

Data la nozione di interfacce si possono introdurre le **classi astratte**, vale a dire classi parzialmente implementate. Dove è presente la parola chiave **abstract** l'implementazione dei metodi non è richiesta, opposte se non è presente.

La classe astratta non può essere istanziata, ma le sottoclassi si dato che obbligatoriamente dovranno implementare i metodi.

Possiamo quindi interpretare le classi astratte come un compromesso tra classi e interfacce.

```
abstract class Veicolo {
  String marca;
  void muoviti() {
    System.out.println("Il veicolo si muove");
  }
  abstract void accendi();
}
```

Anche le classi possono implementare un'interfaccia(come abbiamo visto come i record). Quando una classe dichiara di implementare un'interfaccia, deve fornire tutto il codice di tutti i metodi definiti nell'interfaccia. In questo modo il client può utilizzare l'interfaccia senza conoscere i dettagli implementativi che cambiano nel tempo.

INTERFACCIA E CLASSE ASTRATTA

```
interface Movible{
    void muovi(int distanza);
}


abstract class Veicolo implements Movible{
    String modello;
    int velocitaMax;

    public Veicolo(String m, int v){
        modello = m;
        velocitaMax = v;
    }

    public void stampaInfo(){
        System.out.println("MODELLO DEL VEICOLO: " + modello);
        System.out.println("VELOCITA' MASSIMA: " + velocitaMax);
    }


    abstract public void muovi(int distanza);
}
```

Come si può vedere nel codice l'interfaccia stabilisce che la classe che la implementa deve generare il codice. In questo caso però si parla di una classe astratta; infatti, il codice viene implementato dalle sottoclassi di Veicolo.
Usando come esempio una sottoclasse Bicicletta, una possibile implementazione è quella proposta:

```
public void muovi(int distanza){
    for(int i = 1; i <= distanza; i++){
        System.out.println("N. pedalate: "+ i);
        try{
            Thread.sleep(1000/velocitaMax);
        }catch(InterruptedException e ){
        }
    }
}
```

Quando facciamo una sovrascrittura del metodo, come fa il compilatore a caricare il metodo della sottoclasse?

```
public void stampaInfo(){
    System.out.println("MODELLO DEL VEICOLO: " + modello);
    System.out.println("VELOCITA' MASSIMA: " + velocitaMax);
}
```

```
public void stampaInfo(){
    System.out.println("MODELLO DELLA MACCHINA: " + modello);
    System.out.println("MARCA DELLA MACCHINA: " +marca);
    System.out.println("VELOCITA' MASSIMA(KM/H): " + velocitaMax);
}
```

Il metodo non viene caricato in fase di compilazione(ergo non viene caricato dal compilatore), bensì a runtime avviene il **binding dinamico(late binding)** attraverso la JVM che riconosce quale metodo chiamare.

Come avviene il **dispatch**? Avviene nella seguente maniera:

- viene controllato il tipo di oggetto che fa la chiamata di funzione(a runtime);
- se il tipo di oggetto contiene il metodo allora si richiama quella implementazione del metodo;
- se il tipo di oggetto esaminato non contiene l'implementazione, si cerca nella superclasse;
- si reitera nel caso di ereditarietà a più livelli, finchè non si trova.

Il **polimorfismo** è la **capacità di un metodo di assumere comportamenti diversi a seconda dell'oggetto** su cui viene invocato, anche se il metodo ha lo stesso nome(nome di funzione) e gli stessi parametri passati.

Il late binding è tipico nei sistemi in cui esiste il polimorfismo e il polimorfismo stesso è fondamentale in sistemi ad oggetti.

Quindi riassumendo:

- quando si dichiara una variabile, si sta dicendo al compilatore quali operazioni si possono fare o no fare su quella variabile;
- la variabile può contenere oggetti di sottoclassi, per questo caso specifico si fa il casting alle sottoclassi(per il principio di Liskov si può fare);
- si effettua il dispatch come si è citato sopra;

Finora abbiamo parlato di ereditarietà, ma non di **associazione** ovvero come un oggetto usa o conosce un altro oggetto.

L'associazione risponde alla domanda: "Un oggetto di tipo A ha un oggetto di tipo B?". La relazione che risponde a questa domanda è una relazione di tipo "**has-a**".

Abbiamo tre tipi principale di associazione:

- **associazione semplice** → un oggetto conosce un altro oggetto.
- **aggregazione** → un oggetto può anche esistere anche in assenza di un altro aggetto. Ad esempio, un oggetto studente può esistere anche senza classe, ma non viceversa.
- **composizione** → l'oggetto non può esistere in assenza di altri oggetti. Ad esempio, un oggetto motore non può esistere senza l'auto(whole).

Diagrammi UML

UML(Unified Modeling Language) è un **linguaggio grafico standard** usato per descrivere, progettare e documentare un sistema software.

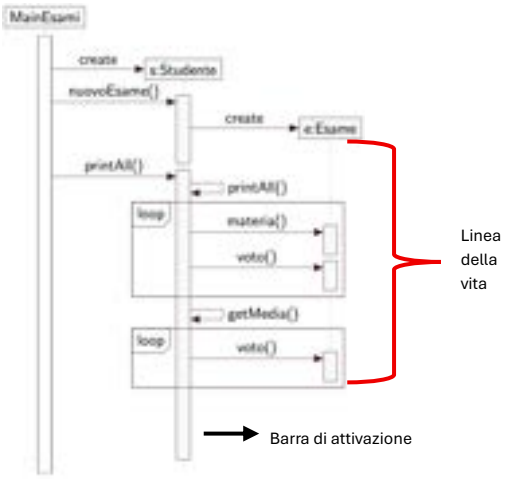
Non è un linguaggio di programmazione, bensì un insieme di notazioni grafiche che permettono di rappresentare:

- struttura di un sistema;
- comportamento del sistema;
- interazioni tra oggetti;

UML comprende diversi tipi di diagrammi che rappresentano aspetti differenti del sistema. I principali simboli adoperati(sulla base del quale si compongono le rappresentazioni grafiche) sono:

Simbolo	Significato
CLASSE + campo: type + metodo(type): type	Le classi sono rappresentate con un rettangolo diviso in tre parti: • nome classe; • attributi; • metodi; La visibilità è la seguente: • + public; • - private; • # protected;
	La freccia, a seconda del tipo, indica: • continua → associazione(utilizzo); • continua con estremità bianca → eredità(da sottoclasse - superclasse); • tratteggiata → implementazione di interfaccia
<<INTERFACCIA>> + metodo(type): type	Interfacce che verranno poi implementate dalle classi.

Il **diagramma UML di sequenza** è un diagramma UML utilizzato per **rappresentare le interazioni tra oggetti nel tempo**. Esso, infatti, mostra come gli oggetti comunicano tra loro tramite lo scambio di messaggi, evidenziando l'ordine temporale delle operazioni.



Nell'immagine a sinistra possiamo apprendere le interazioni tra oggetti come Studente ed Esame nel tempo.

L'asse temporale è inteso in verticale verso il basso e per ogni oggetto(colonna) esiste una linea tratteggiata, detta **linea della vita**, che indica se è attivo.

Infine, la chiamata di metodo è indicata da una freccia che va dalla barra di attivazione di oggetto ad un altro.

Come facciamo ad individuare una classe?

Si parte dal testo dei requisiti e si effettua una "pseudo analisi grammaticale" per individuare:

- **sostantivi** → suggeriscono classi(se sono importanti) e/o attributi(se sono dettagli);
- **verbi** → suggeriscono i metodi delle classi, cioè le operazioni che può effettuare l'oggetto;

Durante l'interpretazione del testo si devono tenere a mente i principi **KISS**(keep it simple, stupid) e **SRP**(single responsibility principle), per evitare di creare classi troppe complesse e/o con troppe responsabilità.

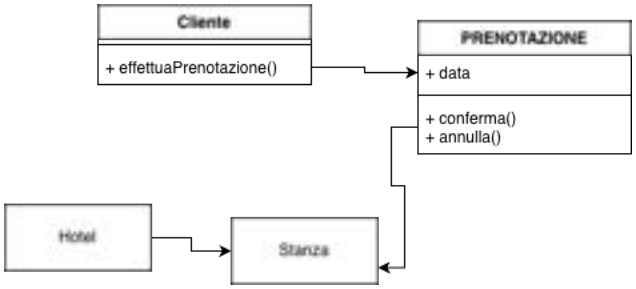
Ad esempio, se analizzassimo la seguente frase:

*"Un **cliente** può **effettuare** una **prenotazione** per una **stanza** di un **hotel**. La prenotazione ha una **data** e può essere **confermata** o **annullata** dal cliente."*

Potremmo trovare le seguenti classi:

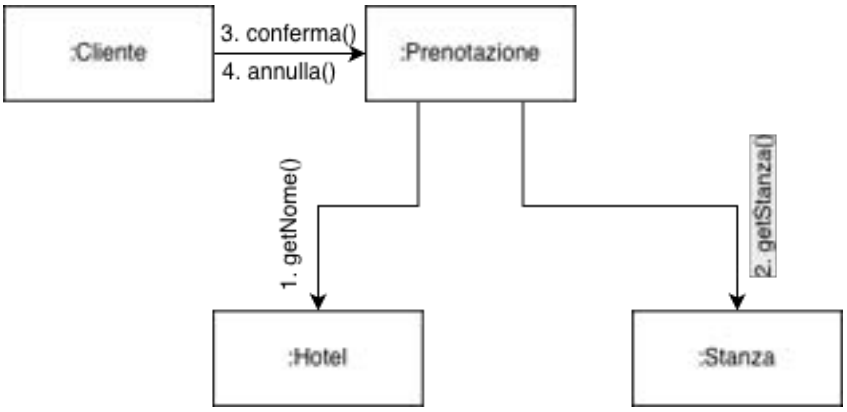
- **cliente;**
- **prenotazione;**
- **stanza;**
- **hotel;**

Il sostantivo **"data"** sarà un attributo della classe prenotazione, il metodo **"effettua"** è metodo della classe Cliente dato che è il soggetto che compie l'azione e i metodi **"conferma/annulla"** sono metodi della classe Prenotazione dato che viene cambiato il proprio stato.



I **DIAGRAMMI DI COLLABORAZIONE** sono dei diagrammi che mostrano come gli oggetti comunicano tra loro per realizzare un'operazione. In questa tipologia di diagrammi si ha:

- oggetti e non classi → si possono anche indicare con :NomeClasse;
- collegamenti(frecce) → indica il flusso di messaggi fra le istanze che partecipano all'interazione;
- messaggi → sono etichette che stanno sulle frecce e hanno:
    - numero sequenziale che indica l'ordine di esecuzione;
    - metodo chiamato;
    - valore di ritorno;

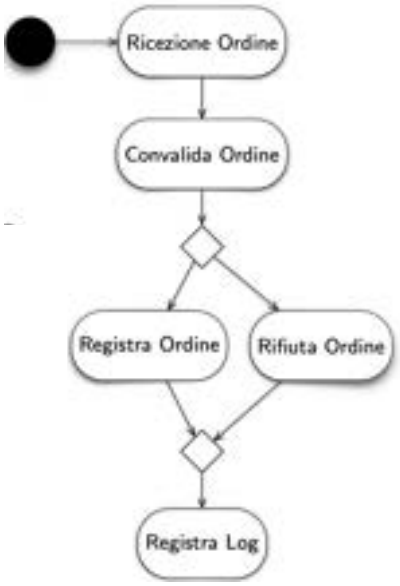


L'analisi grammaticale non risolve tutti i problemi, infatti, potremmo farci sfuggire ulteriori classi e/o metodi che non sono menzionati nel testo. Un buon progettista deve saper leggere al di fuori delle righe e **intuire ciò che è necessario**. Questo, ad esempio, potrebbe essere proposto per perseguire il principio di singola responsabilità e organizzare in modo ottimale gli oggetti.

Comprendere al meglio le richieste di un cliente, del mercato ecc., è l'azione principale dato che l'implementazione è possibile a partire da idee chiare.

Nella progettazione vengono utilizzati diversi tipi di diagrammi per chiarire e rappresentare il sistema. Tra questi, una tipologia impiegata nelle fasi iniziali dell'analisi e della progettazione è il **diagramma UML delle attività**, utile per descrivere il flusso delle operazioni e dei processi.

Il **diagramma UML delle attività**, a differenza dei diagrammi precedenti, non lavora con le classi(o istanze di esse), bensì riflette le attività che il sistema software dovrà realizzare. Questi diagrammi sono concettualmente simili ai flowchart e potrebbero essere utilizzati come punto di partenza per poi creare i diagrammi di collaborazioni tra oggetti.



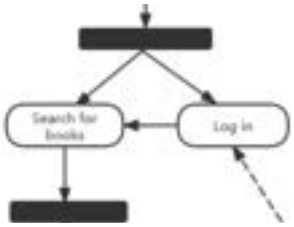
Le forme raffigurate hanno un significato, infatti:

- **cerchio** → stato iniziale;
- **rombo** → ramificazione condizionale o merge;
- **rettangolo** → operazione;
- **frecce** → flusso.

Un ulteriore funzionalità dei diagrammi delle attività sono i flussi paralleli utilizzati.

Ove è possibile, si possono eseguire le operazioni parallelamente e questa parallelizzazione del lavoro viene indicata mediante queste forme:

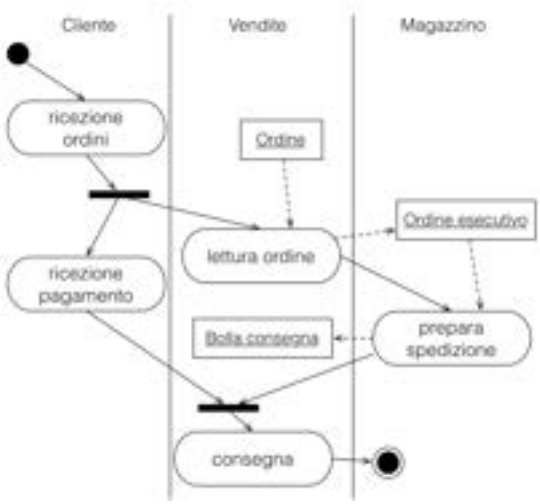
- **barre nere** → fork del lavoro ossia si divide il flusso in rami paralleli o join in caso si riuniscano;
- **cerchio pieno con circonferenza attorno** → stato finale.



Dopo aver trovato le operazioni che devono essere eseguite, si deve trovare il soggetto che le compie. Per questo motivo potremmo utilizzare il **diagramma con corsie(swimlane)**, dove ogni soggetto rappresenta il responsabile delle attività nella **corsia**.

Così viene esplicitata la distribuzione delle responsabilità tra i diversi componenti del sistema, rendendo chiara ciascuna attività e migliorando la leggibilità e comprensione del processo.

Nei diagrammi con corsie si hanno:
rettangolo → indica oggetti o dati;
frecce tratteggiate → indica flusso di dati(input o output).

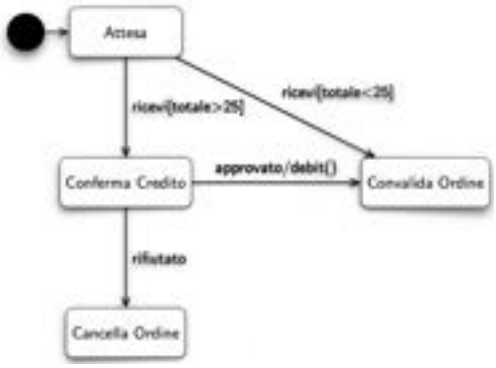


E se volessimo rappresentare i vari stati di un oggetto? Uno **stato è un insieme di caratteristiche e valori che caratterizza un oggetto in un determinato momento**.

A seconda degli stati, un oggetto può compiere diverse operazioni e per questo motivo è utile descriverle.

I **diagrammi degli stati UML** rappresentano il ciclo di vita di un oggetto, descrivendo gli **stati** in cui può trovarsi e le varie **transizioni**(cambio di stato) di stato.
Nel dettaglio una transizione è composta da:

- **evento** → attiva la transizione;
- **condizione** → fa da **guardia** allo stato precedente;
- **azioni** durante il cambio di stato;
- **stato di arrivo**.



Gli stati visti nell'immagine precedente sono **stati semplici** e solitamente vengono utilizzati quando sono pochi gli stati di un oggetto.

Nel caso in cui gli oggetti dovessero avere più stati, potrebbe essere utile organizzarli in uno **stato composto**, vale a dire una **struttura gerarchica più leggibile e gestibile**.

Ad esempio, per un oggetto di tipo Ordine si possono avere vari stati come:

- OrdineCreato;
- OrdinePagato;
- OrdineInviato;
- OrdineSpedito;
- OrdineConsegnato;
- OrdineAnnullato;
- ....

Tutti questi stati potrebbero essere raggruppati tutti all'interno di uno stato composto, per evitare migliorare la semplicità di lettura.

Lo stato esterno, quindi, rappresenta uno dei possibili stati interni e si possono avere uno o più stati interni attivi. Quando si ha un solo stato attivo?

Si fa una netta separazione tra:

- **stato composto sequenziale** → un sottostato attivo per volta;
- **stato composto concorrente** → più stati possono essere attivi contemporaneamente.

Per esprimere chiaramente la possibilità di avere più stati possiamo fare un esempio semplicissimo:

Uno smartphone potrebbe avere uno stato composto TELEFONO ATTIVO, cui al suo interno ci sono tanti sottostati come CONNESSIONE, BATTERIA. In questo caso, più stati possono essere attivi simultaneamente, poiché non si escludono.

DESIGN PATTERNS

Un design pattern è una **soluzione riutilizzabile e collaudata a un problema comune di progettazione** del software, che può essere applicata in diversi contesti senza riscrivere tutto da 0. Non è codice pronto da copiare, ma un modello concettuale e fornisce linee guida su come strutturare classi e oggetti per risolvere problemi.

VANTAGGI → riutilizo del codice, riduzione complessità e miglioramento manutenibilità.

I design pattern non dipendono da un particolare linguaggio di programmazione e come già detto si possono applicare in programmi diversi. Si possono definire come **micro-architetture**, perché non definiscono l'intero sistema, ma indicano come fare interagire alcune classi per risolvere un problema.

L'obiettivo è quindi utilizzare una soluzione già testata e conosciuta, invece di reinventare soluzioni.

I pattern specificano:

- quali classi o oggetti sono coinvolti;
- quali responsabilità hanno;
- come interagiscono tra loro;

In questo modo il pattern guida la progettazione senza un'implementazione concreta. Il punto forte è il **riutilizzo del design**(e non del codice), ad esempio con il singleton sappiamo già come risolvere problemi dove è necessaria solo un'istanza.

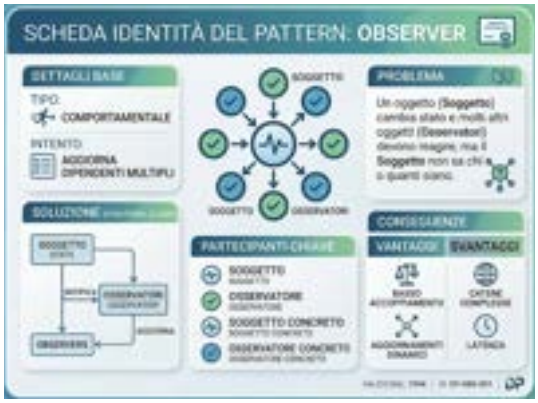
Attraverso i design patterns si ha un linguaggio comune per i progettisti e si hanno benefici sia per i principianti e sia per gli esperti. Si fornisce quindi un vocabolario universale per la soluzione dei problemi.

I design patterns non introducono nuove funzionalità, ma aiutano a implementare proprietà non-funzionali, tipo:

- Affidabilità;
- Modificabilità;
- Testabilità;
- Sicurezza;

Un design pattern nomina astrae ed identifica gli aspetti chiave di un problema di progettazione. La descrizione include 5 parti fondamentali:

NOME **INTENTO** **PROBLEMA** **SOLUZIONE** **CONSEGUENZE**



NOME: introduce un vocabolario comune ai programmatori, indicando con una sola parola ciò che fa in modo comune a tutti.
INTENTO: descrive brevemente cosa fa il design pattern e qual è il suo scopo principale.
PROBLEMA: il problema lo possiamo scindere in due parti:

- **MOTIVAZIONE**: mostra il problema che si vuole risolvere e spiega perché risolvere con una soluzione naïve(ingenua);
- **APPLICABILITA'**: indica quando utilizzare il pattern e quali sintomi suggeriscono il suo utilizzo.

Nella sezione problema si parla di ***forze***, termine rubato alla fisica, è utilizzato per indicare problemi dati dalla tensione causata da obiettivi e vincoli. Ad esempio, essere flessibile e semplice spesso non vanno bene accoppiati, dato che per essere flessibili servono più classi e più classi aumentano la complessità.
SOLUZIONE: è uno schema astratto che: descrive le classi coinvolte, specifica i ruoli e spiega le relazioni.
CONSEGUENZE: descrive vantaggi, svantaggi e i compromessi. Le conseguenze aiutano a rispondere alla domanda: "Vale davvero la pena utilizzare questo design pattern?".

Possiamo anche avere descrizioni mediante gli **esempi di utilizzo** e il **codice** che lo implementa.

I design pattern sono organizzati a seconda dello scopo, infatti abbiamo:

- **pattern creazionali** → gestiscono la creazione di oggetti in modo flessibile, sicuro e indipendente dal tipo concreto di oggetto. I vantaggi che si hanno sono:
  - assenza di duplicati per la creazione di oggetti;
  - separazione creazione e utilizzo dell'oggetto.
- **pattern strutturali** → gestire come le classi e gli oggetti sono composti per formare strutture più grandi e flessibili. I vantaggi ottenuti sono:
  - collegare oggetti diversi senza dipendere dalle loro implementazioni;
  - riutilizzo del codice senza modificare classi esistenti.
- **pattern comportamentali** → gestiscono interazioni e algoritmi tra oggetti, in altre parole stabilisce come comunicano e cooperano due oggetti.

DESIGN PATTERN: TEMPLATE METHOD

Il **template method** è un **design pattern comportamentale** che permette di definire la struttura generale di un algoritmo nella superclasse, lasciando poi le sottoclassi la possibilità di ridefinire alcuni passaggi.

DEFINIZIONE → SUPERCLASSE

IMPLEMENTAZIONE → SOTTOCLASSE

La superclasse astratta contiene lo scheletro dell'algoritmo attraverso un metodo chiamato template method. I passaggi espressi all'interno del template method possono essere:

- **metodi concreti** contenuti all'interno della superclasse;
- **metodi primitivi** definiti all'interno della superclasse ma implementati obbligatoriamente dalle sottoclassi;
- **metodi hook** di natura opzionale, con implementazioni o da superclassi o da sottoclassi(anche non necessaria implementazione);

Facendo una forzatura per l'esempio, si pone List come superclasse e non come interfaccia. Noto questo nuovo vincolo che abbiamo imposto: possiamo dire che List è la superclasse che definisce metodi come add(), size() ecc., mentre ArrayList è una delle sottoclassi che implementa questa classe.

I metodi template portano ad avere una **struttura di controllo invertita**, che permette ad una superclasse di chiamare le operazioni di una sottoclasse. ⟹ **PRINCIPIO HOLLYWOOD**

CLASSE ASTRATTA

```
abstract class Document {
    public void open() {//template method
        read();
        print();
        hook();
    }
    abstract void read();//metodo primitivo
    void print() {
        System.out.println(x: "Stampo documento");
    }
    void hook() { } //metodo hook
    // ->            definisce un comportamento opzionale
}
```

SOTTOCLASSE IMPLEMENTATIVA

```
class MyDocument extends Document {
    void read() {
        System.out.println(x: "Leggo MyDocument");
    }
    void hook() {
        System.out.println(x: "Operazione aggiuntiva");
    }
}
```

Oggetti e classi

Un **oggetto è un'istanza** della relativa **classe**, infatti, una classe sta ad un oggetto come un programma sta al processo. Una classe può avere 0 o più istanze, ma in sé è una **descrizione statica**, mentre un oggetto è un'entità concreta creata a runtime.

Le classi formano il vocabolario con cui si modella(si passa dal problema reale ad uno astratto) il dominio del problema e gli oggetti attraverso le loro interazioni cercano di soddisfare i requisiti del problema.

```
Studente s1 = new Studente(nome: "Mario", media: 27.5f);
Studente s2 = new Studente(nome: "Anna", media: 30.0f);

Corso c1 = new Corso(titolo: "Programmazione 2", cfu: 8);
Corso c2 = new Corso(titolo: "Ingegneria del Software", cfu: 6);
```

In questo caso s1, s2, c1, c2 sono tutte istanze delle classi Studente e Corso.

DESIGN PATTERN: SINGLETON

Come possiamo stabilire che di una classe viene generata un'unica istanza? Si potrebbe utilizzare un design pattern, nel dettaglio si potrebbe utilizzare il design pattern **Singleton**.

Il **Singleton** è un **design pattern creazionale** che permette di avere una **classe con una sola istanza**, il cui **punto di accesso è globale**. Perché è desiderato avere sola un'istanza? Le motivazioni potrebbero essere:

- spreco di risorse;
- incoerenze di stato.

Ad esempio, non si può avere più di un windows manager, altrimenti non è garantita la coerenza.

Il design pattern preso in esame agisce nel seguente modo:

- rende il costruttore privato, quindi nessuno può creare oggetti liberamente;
- mantiene un riferimento statico alla propria unica istanza;
- fornisce un metodo pubblico per accedere a quella istanza.

```
class Logger{
    private static Logger lgr = new Logger();

    private Logger(){
        System.out.println(x: "Logger creato");
    }

    public static Logger getInstance(){
        return lgr;
    }

    public void log(String Message){
        System.out.println("LOG: " + Message);
    }
}
```

Perché il costruttore è privato? Per impedire che altre classi(esempio classe main) possano creare istanze di questa classe. In questo modo solo all'interno di questa classe si possono creare istanze della stessa.

Perché la variabile è statica? Essendo statica vive per tutta la durata del programma e non è legata ad una singola istanza.

Perché si fornisce il metodo getIstance()? Perché è l'unico punto globale di accesso.
Tutti i client ottengono sempre l'unica istanza, senza che si creino duplicati.

Così si rispetta il **principio delle conseguenze locali**, molto più semplicemente le modifiche avvengono in modo centralizzato e un cambiamento in un punto non avrebbe effetti su altri punti.

Possiamo anche stabilire che ci sia un numero finito di istanze(non per forza 1 sola) e questo si fa attraverso un design pattern simile al Singleton, utilizzando il **Multiton**.

Ogni **istanza del Multiton deve essere identificata da una chiave**, per questo motivo possiamo utilizzare **Map**(interfaccia) e **HashMap**(implementazione) o più semplicemente un'associazione <chiave, valore>.

In questo modo creiamo una struttura dinamica: se la chiave esiste già, non viene creata una nuova istanza della classe, ma viene restituita quella esistente.

```java
class DB{
    private static Map <String, DB> instances = new HashMap<>();
    private String dbName;

    private DB(String dbName){
        this.dbName = dbName;
        System.out.println("CONNESSIONE CREATA PER DATABASE: "+dbName);
    }

    public static DB getInstance(String dbName){
        if(!instances.containsKey(dbName)){
            instances.put(dbName, new DB(dbName));
        }
        return instances.get(dbName);
    }
```

Viene creata la struttura incaricata a tener traccia delle istanze create. In questo caso si ha una struttura che è composta da:
<chiave, valore>

Se la chiave non viene trovata, viene creata una nuova istanza.

Sebbene il Singleton sembri aggiustare qualche problema, ha dei contro. Tali sono:

- singleton funziona come variabile globale(causa static);
- accoppiamento forte, dato che il codice dipende fortemente da una sola istanza;
- non è facile effettuare dei test, quando la classe dipende da un singleton.

Design Pattern Creazionali

Nei design pattern creazionali si astrae la creazione degli oggetti, in altre parole si separa il codice che usa gli oggetti dal codice che li istanzia. Inoltre, sono importanti nel caso in cui i sistemi evolvono per dipendere su composizioni di oggetti(oggetti contenuti all'interno di altri oggetti) che su ereditarietà tra classi.

Si nascondono le classi complete che vengono utilizzate in un sistema. Solo una parte del sistema conosce la reale implementazione delle classi realmente create, il resto lavora solo con interfacce o classi astratte.

DESIGN PATTERN: FACTORY METHOD

(INTENTO)L'obiettivo del **Factory Method** è definire un'interfaccia per creare un oggetto e poi lasciare che le sottoclassi decidano quale classe istanziare.

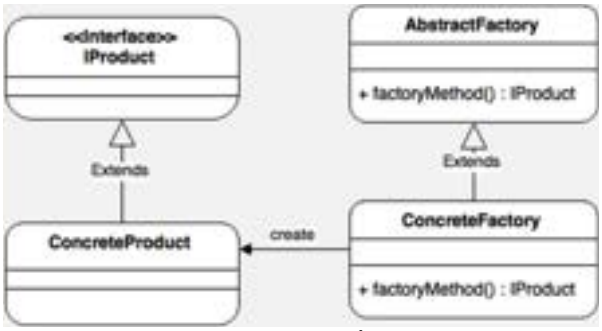
Quindi, una classe base dichiara un metodo per creare un oggetto, ma non decide quale oggetto concreto creare. La **scelta è delegata alle sottoclassi**.

(Problema) Il problema nasce quando un framework lavora con classi astratte ma deve creare oggetti concreti. Infatti, non si può creare un'interfaccia da una classe astratta o da un'interfaccia, bensì da sottoclassi.

Il **Factory Method** introduce un **metodo factory** che incapsula la **creazione degli oggetti** e delega le sottoclassi alla scelta della classe concreta da istanziare.(QUINDI IN BASE ALLA SOTTOCLASSE SI ISTANZIA UN OGGETTO PIUTTOSTO CHE UN ALTRO)

In questo design pattern si hanno 4 protagonisti(SOLUZIONE), vale a dire: **product**, **concrete product**, **creator** e **concrete creator**

- **PRODUCT** → interfaccia o classe astratta che rappresenta l'oggetto da generare. Essa stabilisce solo cosa l'oggetto deve fare, ma non come lo deve fare.
- **PRODUCT CONCRETO** → classe che implementa l'interfaccia e che viene effettivamente istanziata.



- **CREATOR** → classe che implementa l'interfaccia per la creazione di un oggetto. È quella che contiene il **factory method** e altre operazioni che ci permettono di utilizzare l'oggetto.
- **CREATOR CONCRETO** → è l'implementazione della classe CREATOR, che permette di scegliere l'oggetto da creare e lo ritorna.

PRODUCT E PRODUCT CONCRETE

```
interface notifica{
    public void ricevi(String message, String mittente);
}


class Email implements notifica{
    public void ricevi(String message, String mittente){
        System.out.println("Nuova email da: " + mittente);
        System.out.println(message);
    }
}
```

CREATOR E CREATOR CONCRETO

```
abstract class ServizioNotifiche{
    abstract public notifica creaNotifica();
}

class ServizioMail extends ServizioNotifiche{
    public ServizioMail(){}
    public notifica creaNotifica(){
        return new Email();
    }
}
```

(CONSEGUENZE) Il client non sa mai quale classe viene concretamente creata, in questo modo si puà cambiare il prodotto senza modificare il client. La modifica, quindi, avviene soltanto nella sottoclasse del Creator.

Inoltre, questo rende i Concrete Product intercambiabili, ad esempio:

```
notifica n = new ServizioMail().creaNotifica();
n = new ServizioSMS().creaNotifica();
```

Il **contro** più evidente del factory method è la **proliferazione di classi**, infatti, generare un ConcreteCreator per ogni ConcreteProduct può aumentare la complessità e complicare la manutenzione.

Object Pool

Le istanze già create possono essere messe in strutture adeguate e riutilizzate all'occorrenza. L'**object pool** gestisce un insieme di istanze riutilizzabili.

Si potrebbe utilizzare l'object pool perché la creazione di un oggetto è costosa e si vuole controllare il numero massimo di istanze.

Ci possono essere due modi possibili per implementare il pool:

- **Pool dinamico** → se non ci sono oggetti disponibili se ne creano di nuovi.
- **Pool a dimensione fissa** → se non ci sono oggetti disponibili il client o aspetta o fallisce.

Dopo che il client utilizza l'oggetto, l'**istanza tonerà nel pool previo reset** dato che si devono pulire i dati del vecchio utilizzo.

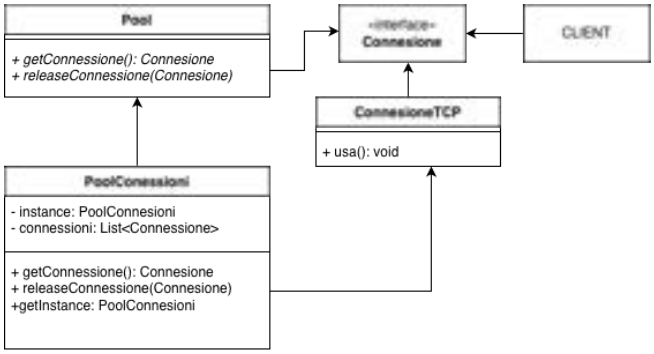
Il **pool** può essere **gestito** tramite **factory method**, infatti: il pool stesso può fungere da creator, il metodo getInstance() può essere il factory method e il product è l'oggetto contenuto all'interno del pool.

L'**incapsulamento del pool** può essere fatto mediante **singleton**, in modo tale che ci accertiamo che esiste una e una sola istanza.

Il codice riportato accanto è un esempio di object pool e come si è detto precedentemente si sono utilizzati i due design pattern che permettono:

- **factory method** → separare la classe che utilizza l'oggetto da quella che lo istanzia;
- **singleton** → creare un'unica istanza della classe.

Di seguito è riportato il diagramma UML che descrive la struttura dell'object pool che si è implementato.



In questo caso il ConcreteCreator è a conoscenza di che tipo di product istanziare, ma se si volesse lasciare decidere a chi utilizza l'istanza dell'object pool?
Si possono inserire le dipendenze necessarie(tipo di product da istanziare) nel costruttore del ConcreteCreator.
Questa tecnica è chiamata **Dependency Injection** e permette di **separare la costruzione delle istanze dal loro uso**.

```java
interface Connessione{ // PRODUCT
    public void usa();
}

class ConnessioneTCP implements Connessione{ // CONCRETE PRODUCT
    public ConnessioneTCP(){}
    public void usa(){
        System.out.println("Connesione TCP stabilita");
    }
}
abstract class Pool{ // CREATOR
    abstract public Connessione getConnessione();
    abstract public void releaseConnessione(Connessione c);
}
class PoolConnesioni extends Pool{ // CONCRETE CREATOR implementato m
    private static PoolConnesioni instance;
    private List<Connessione> connessioni;
    private PoolConnesioni(){
        System.out.println("Pool Connessioni creato");
        connessioni = new ArrayList<>();
    }
    public static PoolConnesioni getInstance(){
        if(instance == null){
            instance = new PoolConnesioni();
        }
        return instance;
    }
    public Connessione getConnessione(){ // FACTORY METHOD
        if(connessioni.size() == 0){
            connessioni.add(new ConnessioneTCP());
        }
        return connessioni.remove(connessioni.size()-1);
    }
    public void releaseConnessione(Connessione c){ // metodo di reins
        connessioni.add(c);
    }

}
```

```java
private static PoolConnessioni instance;
private List<Connessione> connessioni;
private Connessione prototype;

private PoolConnessioni(Connessione proto){
    System.out.println("Pool Connessioni creato");
    connessioni = new ArrayList<>();
    this.prototype = proto;
}

public static PoolConnessioni getInstance(Connessione proto){
    if(instance == null){
        instance = new PoolConnessioni(proto);
    }
    return instance;
}
```

Qui avviene dependency injection.

```java
public Connessione getConnessione(){ // FACTORY METHOD
    if(connessioni.size() == 0){
        try{
            connessioni.add(
                this.prototype.getClass()
                .getDeclaredConstructor()
                .newInstance()
            );
        }catch(Exception e){
            e.printStackTrace();
        }
    }
    return connessioni.remove(connessioni.size()-1);
}
```

Si analizza la dipendenza e si istanzia un nuovo oggetto del tipo della dipendenza

DESIGN PATTERN ABSTRACT FACTORY

L'**abstract factory** è un **design pattern creazionale** che permette di fornire un'**interfaccia per creare famiglie di oggetti correlati tra loro**(INTENTO). Il client non saprà quali classi concrete verranno istanziante, infatti, esso vedrà un blocco unico composto da tanti oggetti simili.

Ad esempio, prendendo in esame la GUI(graphic user interface) di Windows, tutto viene mostrato in modo compatto e gli oggetti appartengono tutti alla stessa famiglia. Non possiamo quindi avere: BottoneWindows e TextFieldMac.

Il problema che si vuole risolvere è la dipendenza diretta che hanno i programmi rispetto le classi concrete(PROBLEMA).

Nel particolare si promuove una programmazione ad interfacce dove si utilizzano famiglie di oggetti compatibili e un sistema indipendente dalle implementazioni specifiche.

Come nel factory method, si hanno della classi concrete e delle classi o interfacce astratte, che vengono implementate da sottoclassi. I protagonisti principali dell'abstract factory sono:

- **ABSTRACT FACTORY** → interfaccia astratta che dichiara i metodi per creare i prodotti;
- **CONCRETE FACTORY** → classi che implementano la factory e creano una famiglia specifica di oggetti → ogni factory crea una famiglia coerente;
- **ABSTRACT PRODUCT** → è l'interfaccia dei prodotti che specifica quali metodi devono rispettare;
- **CONCRETE PRODUCT** → sono le implementazioni concrete dei prodotti che poi vengono raggruppate dalle concrete factory.

Il client utilizza soltanto le interfacce dichiarate da **AbstractFactory** e **AbstractProduct**.

In breve, si utilizzano le interfacce per fare i prodotti e le "famiglie di prodotti", implementati poi da sottoclassi che le implementano.

```java
interface Button{
    public void render();
}

interface CheckBox{
    public void render();
}

interface GUIFactory{
    public Button createButton();
    public CheckBox createCheckBox();
}
```

```java
class WindowsButton implements Button{
    public void render(){
        System.out.println(x: "RENDERING WINDOWS BUTTON");
    }
}
class WindowsFactory implements GUIFactory{
    public Button createButton(){
        return new WindowsButton();
    }

    public CheckBox createCheckBox(){
        return new WindowsCheckbox();
    }
}
```

In questo modo verranno quindi create e utilizzati dei prodotti consistenti, con famiglie che sono intercambiabili tra loro. (CONSEGUENZE) Il problema principale è che l'aggiunta di un nuovo prodotto alle famiglie di classi potrebbe comportare una modifica del codice un po' più complessa, si dovrà considerare:

- aggiunta del metodo in abstract factory;
- aggiunta del metodo nelle concrete factory;
- creazione interfaccia prodotto;
- implementazione delle interfacce.

DESIGN PATTERN: PROTOTYPE

Il **prototype pattern** è un **design pattern creazionale**, che permette di creare nuovi oggetti a partire da un'istanza prototipo già esistente, clonandola.

Consente quindi di ottenere oggetti con uno **stato già configurato** senza istanziarli direttamente con new(INTENTO).

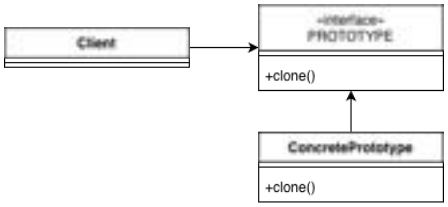
Si utilizza un'istanza prototipo e si delega la creazione all'oggetto stesso tramite un **metodo clonazione**, invece che al client.

(MOTIVAZIONE) In molte applicazioni, gli oggetti non vengono creati da "zero", ma devono partire da **uno stato già configurato**. Si devono utilizzare un pattern che eviti la creazione di dipendenze, vale a dire che eviti che il client conosca direttamente la classe concreta o che per generare un nuovo oggetto dello stesso tipo del prototipo si crei un metodo factory apposito.

Si riduce quindi il **coupling tra classi**, permettendo la creazione all'oggetto stesso ed evitando errori per raggiungere lo stato configurato proposto dal prototipo stesso.

Inoltre, si supera il problema del factory method che per l'istanziazione utilizzava la classe concreta e la classe astratta.

(SOLUZIONE) Come nei design pattern precedenti, definiamo i protagonisti del prototype pattern. Essi sono:



- PROTOTYPE: interfaccia che definisce per le operazioni comuni alle classi concrete e un metodo per la clonazione;
- CONCRETE PROTOTYPE: è una delle implementazioni che permette la clonazione;
- CLIENT: una delle classi che crea nuovi oggetti tramite la clonazione dell'istanza prototipo, senza indicare la classe in modo esplicito.

I vantaggi(CONSEGUENZE) legati all'utilizzo del prototype sono:

- l'utilizzo da parte del client di classi specifiche pur non conoscendone il nome;
- riduce il coupling tra il client e le classi concrete;
- è possibile aggiungere nuovi prodotti e rimuovere prototipi esistenti anche a runtime

(IMPLEMENTAZIONE) Spesso si utilizza un **manager dei prototipi**, ossia una **struttura dati** dove vengono **registrate istanze prototipo** e dalle quali all'occorrenze si richiama il **metodo clone()**. Questo permette che i prototipi vengano registrati a run-time e che vengano sostituiti dinamicamente.

Ci sono due tipi di implementazione di clonazione:

- **shallow copy(copia superficiale)**: copia i campi primitivi(variabili), i riferimenti agli oggetti e alle strutture dati. Proseguendo così potremmo avere istanze dipendenti tra di loro, vale a dire che quando si modifica un elemento della struttura condivisa di un'istanza si modifica anche per le altre.
- **deep copy(copia profonda)**: copia i campi primitivi ma le strutture non sono condivise bensì indipendenti. Richiede un'implementazione manuale.

Inoltre, il clone potrebbe richiedere ulteriori metodi aggiuntivi per modificare alcuni campi o per inizializzazioni specifiche.

DEEP COPY

```
public Personaggio clone(){
    Map<String, String> abilitiesCopy = new HashMap<>(this.abilities);
    return new PersonaggioConcreteD(PS,PF, abilitiesCopy);
}
```

SHALLOW COPY

```
public Personaggio clone(){
    return new PersonaggioConcreteS(PS,PF, abilities);
}
```

DESIGN PATTERN: BUILDER

Consideriamo una classe in cui si vogliono avere **attributi immutabili**(final) e i cui valori di alcuni potrebbero essere non necessari(non si assegna alcun valore).

Per istanziare la classe si può utilizzare un **costruttore telescopico**, cioè una **tecnica che permette di avere più costruttori sovraccaricati**, ognuno con un numero o con un diverso tipo di parametri presi in input.

Questo costruttore potrebbe prevedere diverse istanze, ma si hanno svantaggi legati a:

- complessità nell'implementazione del pattern stesso;
- necessità da parte del client di sapere tutte le firme;
- passare i parametri in modo appropriato.

Si potrebbero assegnare i valori mediante i tipici metodi **setNomeVariabile()**, ma gli svantaggi sarebbero:

- non utilizzo delle variabili immutabili(final);
- stati incompleti o incoerenti, dato che il client potrebbe anche decidere spontaneamente di non settarli.

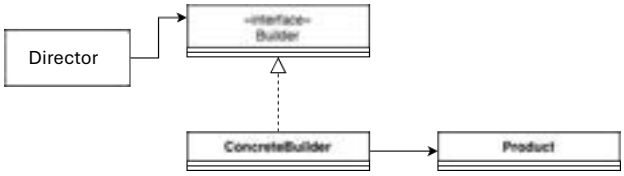
Un modo per aggirare il problema è utilizzare il **design pattern Builder**. Il Builder è un pattern riconducibile alla famiglia dei design patterns creazionali e permette di separare la costruzione complesso dalla sua costruzione.

Questo permette in pratica anche di costruire degli oggetti con diverse rappresentazioni(INTENTO).

L'implementazione avviene nel seguente modo(SOLUZIONE):

- Il client istanzia la classe Builder, con i parametri richiesti dal costruttore;
- Si applicano i metodi set alle variabili;
- Il client richiama un metodo della classe Builder chiamato build per ottenere l'oggetto immutabile.

Gli attori che entrano in gioco in questo design pattern sono:



- **Builder** → definisce un'interfaccia per creare parti di un product;
- **ConcreteBuilder** → costruisce parti del product, ed implementa il **builder**. Essa, contiene il metodo **build()**;
- **Director** → si crea un ulteriore strato di astrazione, che permette di incapsulare il progetto di costruzione di un oggetto e definendo le operazioni da seguire. Infine, le varie funzioni del director ritornano le istanze costruite.
- **Product** → rappresenta l'oggetto complesso da costruire. Esso definisce le parti che poi vengono assemblate nel risultato finale.

Il builder nasconde la rappresentazione e la struttura interna del product, isolando così il codice di costruzione da quello di rappresentazione. Ogni **ConcreteBuilder contiene il codice per creare e assemblare un particolare tipo di product**. Solo quando il product è completo viene preso da dentro il builder(CONSEGUENZE).

Istanza specifica ottenuta mediante il Director

```
public House getFamiliarHouse(){
    HouseBuilder large = new ConcreteHouseBuilder(r: 4, rr: 2, ro: 3);
    large.setGarage(g: true);
    large.setGarden(g: true);
    return large.getHouse();
}
```

Interfaccia che deve essere implementata dai vari ConcreteBuilder

```
interface HouseBuilder{
    public void setGarage(boolean g);
    public void setGarden(boolean g);
    public void setSolarPanels(boolean s);
    public House getHouse();
}
```

Il client chiamerà la funzione **getFamiliarHouse**, per avere già una classe costante con i parametri descritti all'interno del metodo. Si separa il processo di costruzione dalla rappresentazione finale.

DESIGN PATTERN: ADAPTER

Un problema che è alla base della produzione del software è legato all'aggiornamento del codice, infatti, quando il codice cambia si potrebbero avere problemi di **incompatibilità con i client**.

Principalmente questa incompatibilità viene causata quando:

- si cambia firma del metodo;
- si cambia ritorno della funzione;
- il comportamento all'interno della funzione;

Per evitare **breaking change**, vale a dire che una modifica potrebbe rompere i client esistenti, il **design pattern Adapter(strutturale)** propone di trovare una soluzione mediante astrazioni.

La proposta dell'Adapter è di **convertire l'interfaccia di una classe con quella che i client stessi si aspettano**(INTENTO), in modo tale che venga abbattuto il problema dell'incompatibilità.

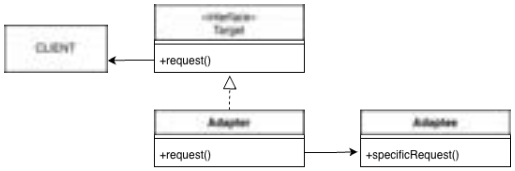
Il problema(PROBLEMA) che nasce dal non utilizzo dell'Adapter sono legati principalmente all'impossibilità di poter cambiare le classi concrete e le interfacce legate, ergo non si può cambiare questo stato d'incompatibilità.

Ci sono due implementazioni(SOLUZIONI) che permettono di utilizzare l'Adapter, queste sono:

1) OBJECT ADAPTER(COMPOSIZIONE)

I protagonisti coinvolti sono:

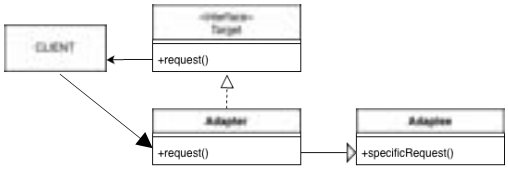
- **TARGET**: l'interfaccia che il client si aspetta;
- **CLIENT**: usa oggetti e metodi conformi a quelli proposti dall'interfaccia target;
- **ADAPTEE**: oggetto che necessita l'adattamento;
- **ADAPTER**: la classe che implementa l'interfaccia target, e che viene utilizzata dal client. Tiene il riferimento all'oggetto da adattare, invocandone i metodi.



2) CLASS ADAPTER(EREDITARIETA')

I protagonisti coinvolti sono:

- **TARGET**: l'interfaccia che il client si aspetta;
- **CLIENT**: usa oggetti e metodi conformi a quelli proposti dall'interfaccia target;
- **ADAPTEE**: oggetto che necessita l'adattamento;
- **ADAPTER**: la classe che **implementa l'interfaccia target e che è sottoclasse di Adaptee**.



La seconda implementazione realizza l'Adapter a due vie, vale a dire un Adapter che soddisfa queste due condizioni:

- può essere visto come target dal client(dato che implementa l'interfaccia);
- come classe originale(dato che estende la super-classe);

I vantaggi(CONSEGUENZE) che si hanno nell'utilizzare l'Adapter sono:

- client e libreria originale rimangono indipendenti. Quindi, non dovranno essere modificati, bensì si dovrà creare una classe che faccia da ponte tra i due.
- si possono implementare i **test di pre-condizione**(verificare ciò che bisogna soddisfare prima che venga eseguito) e di **post-condizione**(verificare i risultati che si sono avuti).
- si può creare l'oggetto Adaptee quando è utile → **Lazy initializzation;**

Lo svantaggio principale e dato dal **livello di indirettezza** in più che si aggiunge, ergo si complica un po' la gestione.

CLIENT

```java
public class ex15{
    Run | Debug
    public static void main(String args[]){

        ILogger l = new Logger();
        l.logInfo(message: "APERTO CORRETTAMENTE IL FILE");
        l.logError(message: "IL FILE NON E' DI TIPO ISBLK");

        ILogger l2 = new Logger2();
        l2.logInfo(message: "APERTA CARTELLA");
        l2.logError(message: "ERRORE DURANTE L'ESECUZIONE DEL PROCESSO");
    }
}
```

INTERFACCIA

```java
// INTERFACCIA -> FUNZIONI UTILIZZATE DAL CLIENT
interface ILogger{
    public void logInfo(String message);
    public void logError(String message);
}
```

ADAPTEE

```java
class LegacyLogger{
    public void write(String l, String m){
        System.out.println("[ "+ l +" ] " + m);
    }
}
```

OBJECT ADAPTER

```java
class Logger implements ILogger{
    private LegacyLogger ll;


    public Logger(){
        ll = new LegacyLogger();
    }

    public void logInfo(String message){
        ll.write(l: "INFO", message);
    }

    public void logError(String message){
        ll.write(l: "ERROR", message);
    }
}
```

CLASS ADAPTER

```java
class Logger2  extends LegacyLogger implements ILogger {
    public Logger2(){
    }

    public void logInfo(String message){
        this.write(l: "INFO",message);
    }

    public void logError(String message){
        this.write(l: "ERROR",message);
    }
}
```

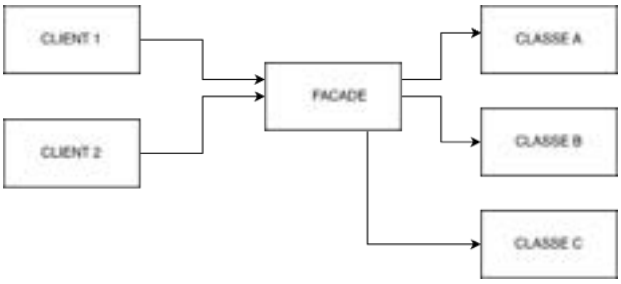
Come già detto, l'**Adapter** adatta metodi presenti nella classe **Adaptee** implementando ciò che l'interfaccia detta.

DESIGN PATTERN: FACADE PATTERN

Il **Facade** è un **design pattern strutturale**, che permette di creare un'interfaccia unificata(attraverso una classe) che semplifica un sottosistema composto da più oggetti(INTENTO).

Questo design pattern nasconde la complessità interna, offrendo un unico punto di accesso semplice.

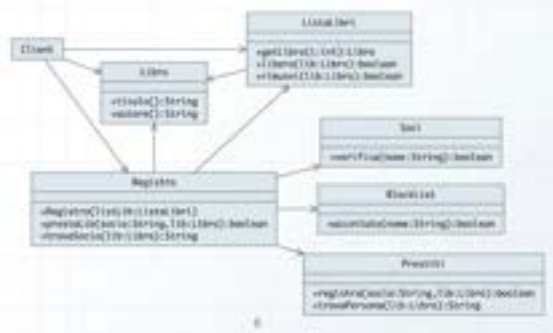
Il pattern in questione risolve problemi legati alla **correlazione tra classi**, che cooperano tra loro, il cui utilizzo potrebbe risultare **complesso e difficile da comprendere al client che le utilizza**. Inoltre, in assenza di questa soluzione si avrebbe una **dipendenza diretta tra client e sottosistema**(PROBLEMA).



(SOLUZIONE) Mediante l'utilizzo di Facade il client interagisce solo con l'istanza della classe stessa, non conoscendo i metodi e gli oggetti che sono nascosti.
L'interfaccia semplificata potrebbe **ridurre la complessità delle chiamate**.
I vantaggi, quindi, sono(CONSEGUENZE):

- interfaccia semplice che nasconde l'implementazione del sottosistema;
- accoppiamento debole tra sottoinsieme e client;
- in caso di cambio di una classe del sottosistema, non si ricompila il client.

```
class Registro{
    private Libri libri;
    private Soci soci;
    private BlockList blockList;
    private Prestiti prestiti;
```



```
public boolean prestaLib(){
    Socio socio = soci.fetchSocio();
    if(!soci.verify(socio)){
        System.out.println("Socio non trovato");
        return false;
    }

    if(!blockList.accepted(socio)){
        System.out.println("Socio è presente nella BlockList");
        return false;
    }

    Libro libro = libri.fetchLibro();
    if(!libri.isFree(libro)){
        System.out.println("Il libro non è libero o non è presente nel catalogo");
        return false;
    }

    if(!prestiti.register(socio, libro)){
        return false;
    }

    libri.removeLibro(libro);

    return true;
}
```

In questo caso la classe registro è la classe che funge da Facade(Facciata) e che ingloba al suo interno tutte le classi che poi verranno usate indirettamente dal client.
Attraverso la funzione **prestaLib()** che richiamerà poi il client si gestisce un sistema molto complesso.

JAVA STREAM

Java è nato come **linguaggio imperativo**(si specifica come fare una cosa) e **OOP**(object-oriented programming), ma nel tempo ha ridotto il codice "cerimoniale", spostando la complessità:

- nelle librerie standard;
- nel linguaggio stesso;
- dentro il compilatore;

L'obiettivo di questa riduzione della complessità nel codice è per:

- migliorare la leggibilità;
- ridurre gli errori;
- facilitare la parallelizzazione;

In genere, la programmazione in java negli ultimi anni tende più ad essere **funzionale**, o meglio una programmazione dove il programma è costruito attraverso le funzioni.

In questo modo, si descrive cosa calcolare e non come modificare lo stato del programma passo per passo. Le caratteristiche della programmazione funzionale sono: **concisione** → meno codice ripetuto, **espressività** → si esprime cosa si vuole fare e **facilità nella parallelizzazione** → usa funzioni pure(non modificano lo stato esterno) e non modificano lo stato condiviso.

Ad esempio, una più semplice implementazione introdotta che permette di ridurre il boilerplate(diverse sezioni nel programma ripetute con poche modifiche) è l'utilizzo delle **classi anonime**.

Una **classe anonima è una classe senza nome che è dichiarata e istanziata nello stesso momento**. Essa viene utilizzata per implementare un'interfaccia o estendere una classe astratta in modo rapido, quando serve solo in un punto del codice.

```
public static void main(String args[]){

    Thread t = new Thread(new Runnable(){
        public void run(){
            for(int i = 1; i <= 5; i++){
                System.out.println("Messaggio " + i + "thread");
                try{
                    Thread.sleep(millis: 5);
                }catch(InterruptedException e){
                    System.err.println(e);
                }
            }
        }
    });
    t.start();
    System.out.println(x: "Esecuzione thread principale finita");
}
```

IN QUESTO MODO SI ISTANZIA E IMPLEMENTA LA CLASSE ANONIMA. NEL CASO SPECIFICO SI IMPLEMENTA IL METODO RUN CHE POI VERRA' RICHIAMATO DALLA FUNZIONE START.

COSI' SI AVRA' QUESTA SPECIFICA IMPLEMENTAZIONE CHE POTRA' ESSERE UTILIZZATA UNA SINGOLA VOLTA.

Per implementare interfacce che hanno solo un metodo, al posto di utilizzare le classi anonime si possono utilizzare le **funzioni lambda**.

Un'**espressione lamba** è una **funzione anonima**, i cui parametri passati si trovano a sinistra della freccia e le linee di codice a destra. Essa può anche ritornare qualcosa, infatti può essere passata come argomento di un metodo.

È coincisa, dato che non servono preamboli per introdurla, ad esempio classi o definizione delle funzioni o simili.

Nei linguaggi di programmazione funzionale, le funzioni lambda sono pure e non hanno coscienza dello stato esterno, invece in Java la funzione lambda può accedere allo stato esterno.

Un’espressione lambda può assumere le seguenti forme:

- **METODO SENZA PARAMETRI IN INGRESSO E VALORI IN USCITA**

```
Runnable r = () -> System.out.println(x: "Esempio di funzione lambda");
r.run();
```

- **METODO CON PARAMETRI IN INGRESSO E VALORI IN USCITA**

```
Somma s = (a,b) -> a+b;
s.calcola(a: 5, b: 6);
```

- **METODO SVILUPPATO SU PIU’ RIGHE**

```
Thread t2 = new Thread(
    () ->{
        for(int i = 1; i <= 5; i++){
            System.out.println("Messaggio espressione lambda "+ i + " thread");
            try{
                Thread.sleep(millis: 500);
            }catch(InterruptedException e){
                System.err.println(e);
            }
        }
    }
);
```

Sulla base di queste tre forme si possono scrivere espressione lambda di qualsiasi tipo. La cosa importante è che implementano un’interfaccia e l’oggetto istanziato chiami il metodo attribuito all’interfaccia.

Infatti, se non richiamata l’espressione lamba non verrà eseguita perché comunque è una funzione.

Quando una funzione è su più linee di codice, si necessita la chiusa nelle parentesi graffe.

Per una più facile lettura del codice e uno sviluppo istantaneo di esso, molti programmatori preferiscono appoggiarsi a librerie standard che permettono una **programmazione dichiarativa** piuttosto che imperativa. In questo **modo si specifica cosa fare e i dettagli dell’implementazione sono affidati alle librerie sottostanti**.

Si ha un minore controllo sul metodo specifico, ma i vantaggi legati all’utilizzo di librerie standard sono molti.

La programmazione funzionale in Java è dichiarativa, che aggiunge allo stile dichiarativo le funzioni di ordine più alto, ossia funzioni che:

- hanno altre funzioni come parametri;
- creano funzioni;
- ritornano funzioni;

Si ricorda la differenza tra funzione e metodo, la funzione non è associata ad alcuna classe invece il metodo è una parte di essa.

Nelle librerie di Java esistono numerosissime interfacce e classi collections. Ad esempio, si utilizza List senza sapere che List è un’interfaccia che a sua volta estende **Collection**.

Infine, oltre a fornire le interfacce vengono fornite anche varie implementazioni, come ArrayList, List e Vector. Le varie implementazioni divergono nella gestione delle liste.

L’ArrayList ha un array interno che viene istanziato per un certo numero di elementi e quando viene superata la capienza, si copia l’istanza in un nuovo array grande il 50% in più. Quindi, se si ha un **ArrayList** con 4 elementi e si prova ad inserire il 5 allora l’array verrà nuovamente istanziato per 6 elementi.

L’implementazione **Vector** si utilizzava in passato perché Thread-safe; quindi, permetteva un inserimento sincronizzato all’interno della struttura dati. L’eccessiva sincronizzazione rallentava l’utilizzo della struttura e per questo motivo è stato deprecato.

Un esempio di collection diversa è **Set** che non ammette duplicati. L’implementazione utilizzata solitamente è **TreeSet**, che fa degli inserimenti ordinati all’interno dell’insieme.

Java 8, per evitare che la modifica di un'interfaccia porti all'impossibilità di non poter compilare tutte le classi che la implementano, ha introdotto i **metodi di default** per le interfacce.

I metodi di default sono dei metodi implementati all'interno delle interfacce che non possono accedere allo stato perché le **interfacce non hanno campi di istanza**.

Ad esempio, un metodo di default dell'interfaccia **Collection** è il metodo **stream()**, che prendendo una lista restituisce uno Stream<T> (dove T è il tipo di elemento della collection su cui è evocato il metodo stream).

Il metodo precedentemente citato è molto utilizzato perché:

- prende gli elementi di una lista;
- applica le operazioni al singolo elemento;
- combina più operazioni tra loro;
- non viene scritto esplicitamente il ciclo.

Quindi viene manipolata l'intera lista senza scrivere esplicitamente i cicli, anche se internamente esistono.

Questo riduce ulteriormente il boilerplate e facilita la lettura del codice. Un esempio è proposto di seguito:

```
List<Integer> numbers = Arrays.asList(...a: 1,2,3,4,5,6,7,8,9,10);
numbers.stream()
    .filter(n -> n%2 == 0)
    .forEach(n -> System.out.println(n));
```

Con il ciclo si controlla l'iterazione e la gestione delle variabili è migliore, ma il codice non è dichiarativo con l'iterazione visibile.

Da codice proposto infatti possiamo facilmente leggere: "Prendi i numeri della lista, filtra solo quelli pari e infine stampali".

Una versione imperativa potrebbe essere la seguente:

```
List<Integer> number_s = Arrays.asList(...a: 1,2,3,4,5,6,7,8,9,10);

for (Integer n : numbers) {
    if (n % 2 == 0) {
        System.out.println(n);
    }
}
```

Da questo codice proposto possiamo invece leggere: "Per ogni numero appartenente alla lista numbers, verifica se è pari e nel caso in cui sia pari stampalo".

La lettura diventa molto di più verbosa e per questo motivo si preferisce almeno in questo caso l'applicazione dello stream.

Il **tipo Stream definisce metodi che accettano in ingresso interfacce funzionali**(contengono un solo metodo), permettendo quindi di passare delle **espressioni lambda**.

Le operazioni con gli stream possono essere di due tipi:

- **operazioni lazy** → operazioni **intermedie**, che vengono eseguite quando è necessario e non sempre dove sono indicate → il **tipo di ritorno** è di tipo **Stream** → filter(), map(), sorted().
- **operazioni eager** → operazioni terminali, che forza l'esecuzione delle precedenti operazioni(di tipo lazy) e quindi attivano la pipeline → **restituiscono un risultato o effetto** →count(), forEach(), reduce().

Come già detto, le operazioni sugli Stream possono accettare come parametri **interfacce funzionali**, cioè interfacce che definiscono un solo metodo astratto.

Le interfacce funzionali vengono utilizzate principalmente per passare **espressioni lambda**: il compilatore, riconoscendo che l'interfaccia ha un solo metodo astratto, sostituisce automaticamente quel metodo con la funzione anonima fornita.

Metodo reduce

Il metodo **reduce()** è un metodo molto utilizzato quando si opera con gli stream. Questo metodo è di tipo terminale e serve per ridurre tutti gli stream ad un singolo valore.

Quindi, permette di eseguire un'operazione che aggrega tutti i dati in un singolo valore. Un esempio è la somma degli Stream, infatti:

```
List<Integer> toSum = Arrays.asList(...a: 1,2,3,4,55,6,7,10,15,77);
int sum = toSum.stream()
    .reduce(identity: 0,(accum,v) -> accum+v);
System.out.println("Somma uguale a " + sum);
```

La firma della funzione reduce sarà quindi:

reduce(valore iniziale, (accumulatore, elemento) -> risultato)

in questo modo quindi descriviamo in modo molto leggibile cosa si vuole ottenere.

L'operatore binario((accumulatore, elemento) -> risultato) può anche non essere un'espressione lambda, bensì può essere anche la chiamata ad un metodo. Ad esempio, si può anche utilizzare **Integer::sum** che indica il riferimento a un metodo della classe Integer.

Il metodo a cui ci riferiamo in realtà corrisponde alla funzione:

static int sum(int a, int b);

Nel caso in cui non ci sia un valore iniziale viene ritornato un **Optional<T>** ossia un valore che può o non può esistere. I metodi associati a Optional<T> sono:

isPresent(); → ritorna vero se è presente un valore

get(); → permette di prendere il valore se

orElse(x); → se non esiste il valore ritorna ciò che è passato come parametro.

Metodo Map

Un ulteriore metodo utilizzatissimo con gli stream di Java è il metodo **map()**. Questo metodo consiste di prendere in ingresso una funzione mapper e restituisce uno stream contenente i risultati dell'esecuzione.

Map è un'operazione intermedia che chiama una funzione su ciascun elemento dello stream iniziale e il risultato è inserito in un nuovo stream.

```
List<Double> toSquare = Arrays.asList(...a: 22d,55d,453d,669d,34d,81d);
List<Double> roots = toSquare.stream().map(Math::sqrt).toList();
roots.forEach(x-> System.out.println(x));
```

In questo caso possiamo vedere l'applicazione di **Math::sqrt** a tutti gli elementi dello stream. L'output sarà sempre uno Stream, ma con il risultato dell'applicazione della radice quadrata alla lista di partenza.

Come per i casi precedenti, possiamo vedere una leggibilità del codice facilità, grazie all'utilizzo degli stream.

Map e Reduce sono due metodi che concettualmente stanno molto bene insieme, perché:

- **map** → trasforma ogni elemento dello stream;
- **reduce** → applica una funzione di aggregazione e raggruppa in un unico risultato gli elementi.

Questo viene fatto anche e soprattutto quando si deve lavorare con un grosso carico di dati. Infatti:

- si divide il lavoro tra i diversi core adibiti all'elaborazione;
- si elabora in parallelo;
- il risultato viene combinato.

Solitamente l'utilizzo di **reduce** viene **sostituito da operatori di aggregazione più specifici**, come sum(), count(), average(), ecc., che eseguono direttamente il calcolo desiderato senza dover scrivere manualmente la funzione di riduzione.

Altri metodi

Oltre ai due metodi menzionati se ne hanno altri che fanno cose specifiche e che utilizzano ulteriori interfacce funzionali. Ad esempio, il metodo **max()** è di tipo eagers e serve a trovare l'elemento massimo dello stream, secondo un criterio di confronto.

L'interfaccia funzionale richiesta come parametro è il **Comparator** che la cui funzione **comparing** indica l'ordine tra due oggetti.

```
Comparator<Persona> comp = (p1,p2) -> p1.getEta() – p2.getEta;
```

In questo modo si ha che:

- se p1 > p2 ⇒ valore maggiore di 0;
- se p1 < p2 ⇒ valore minore di 0;
- se p1 = p2 ⇒ valore uguale a 0.

Un metodo utile a convertire gli stream in strutture dati è il metodo **.collect()** che permette di raggruppare gli elementi e restituisce la struttura dati.

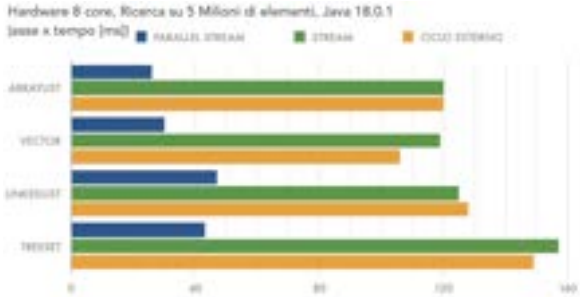
Possiamo applicare operazioni intermedie come map() e poi utilizzare collect() con i metodi della classe **Collectors** per ottenere una struttura dati concreta come una List, un Set, una Map, ecc.

Il metodo sopraccitato è un metodo terminale perché restituisce una struttura dati e non uno stream.

Come già accennato prima, gli stream possono essere utilizzati in parallelo dato che operazioni come **map** e **reduce** sono **stateless**. Questo facilità la paralizzazione infatti si potrebbe evitare l'utilizzo della classe Thread e l'implementazione diretta della sincronizzazione.

Inoltre, Collection ha i metodi di default parallelStream() che implementa internamente la sincronizzazione e le prestazioni variano da:

- numero di elementi dello stream;
- operazione da svolgere;
- hardware;
- tipo di Collection su cui si invoca l'operazione.



Quali operatori degli stream sono **statefull**? Gli operatori **stateful** degli stream sono quelli che hanno bisogno di vedere lo stream per intero per eseguire le operazioni indicate.

Questi operatori potrebbero essere:

- **sorted(Comparator)** → utile a riordinare gli elementi di uno stream;
- **distinct()** → utile a prendere gli elementi dello stream una volta, evitando le ripetizioni.

Operatori che apparentemente potrebbero sembrare stateful non lo sono. Ad esempio, il metodo max() seleziona tra gli elementi disponibili e lo conserva. Per il primo elemento si conserva l'elemento stesso e per ogni nuovo elemento si fa il confronto con il massimo.

```
ordini.stream()
.filter(o -> o.categoria().equals(anObject: "Informatica"))
.sorted(Comparator.comparing(Ordine::cliente))
.distinct()
.forEach(o -> System.out.println("Cliente acquisti \'informatica\'': "+ o.cliente()));
Optional<Ordine> max = ordini.stream()
.max(Comparator.comparing(o-> o.prezzo() * o.quantita()));
if(max.isPresent())System.out.println("Ordine più costoso: " + max.get());
```

STATEFUL

STATELESS

CREAZIONE DEGLI STREAM

In Java possiamo anche creare stream a partire da funzioni. Le due funzioni incaricate per la creazione sono:

- **Iterate()**→ genera una sequenza infinita di elementi a partire da un seme. Questa sequenza è ordinata infatti la funzione si applica a:
  - **seme;**
  - **f(seme);**
  - **f(f(seme));**
  - **...**

  L'ordine è assicurato dall'applicazione della funzione al risultato che si è ottenuto precedentemente. Solo attraverso **limit(int)** si un vincolo di terminazione, costringendo la generazione a terminare al numero di iterazioni indicato.

```
List<Integer> pari = Stream.iterate(seed: 0, n -> n+2)
.limit(maxSize: 501)
.collect(Collectors.toList());
pari.stream().forEach(o -> System.out.println(o));
```

  L'operazione iterate è **sequenziale**, dato che ogni elemento dello stream dipende dal valore precedente, ergo non può essere parallelizzata(comunque verrebbe eseguita in modo sequenziale).
- **Generate()**→ permette di produrre uno stream infinito di elementi non ordinato. La generazione degli elementi avviene mediante una funzione di tipo **Supplier**, ovvero una funzione che fornisce un valore. Tra gli elementi non c'è alcuna dipendenza come iterate, questo significa che il calcolo può essere distribuito tra vari Thread.

```
List<Integer> dispari = Stream
.generate(() -> 2 * (int)(Math.random() * 50) + 1)
.limit(maxSize: 500)
.collect(Collectors.toList());
dispari.stream().forEach(d -> System.out.println(d));
```

CARATTERISTICA	**ITERATE**	**GENERATE**
Scopo	Itera una funzione su valori che sono stati ottenuti precedentemente attraverso la funzione stessa.	Genera valori mediante una funzione Supplier, che genera elementi.
Input	Valore iniziale(seme) e funzione anonima che ritorna un valore.	Funzione Supplier che restituisce un elemento.
Stile	Elemento calcolato dal precedente.	Elemento creato da zero.
Parallelizzabile	No	Si

Supplier<T> non prende nulla in input, mentre **Function<T,R>** prende un parametro in input.

Per il debug degli stream, si potrebbe utilizzare una funzione **peek()** incaricata a mostrare i valori delle lazy operation, senza che i valori dello stream vengano consumati.

Essa a sua volta è una lazy operation, per questo motivo ha bisogno di un'operazione terminale per essere eseguita.

Perché non utilizzare forEach()? Perché forEach essendo un'operazione terminale consuma gli elementi.

```
System.out.println(x: "DEBUG TRAMITE PEEK");
parallelRandom.stream()
.limit(maxSize: 10)
.peek( x -> System.out.println("PRIMA DI APPLICARE *10: " + x))
.map( x -> x*10)
.peek( x -> System.out.println("DOPO AVER APPLICATO *10: " + x))
.toList();
```

```
DEBUG TRAMITE PEEK
PRIMA DI APPLICARE *10: 0.5134508782761686
DOPO AVER APPLICATO *10: 5.134508782761687
PRIMA DI APPLICARE *10: 0.98541766786438
DOPO AVER APPLICATO *10: 9.8541766786438
```

Dato che molte volte gli stream vengono utilizzati per interi, esistono degli Stream di interi chiamati **IntStream**. I vantaggi legati all'utilizzo degli IntStream sono:

- maggiore efficienza;
- metodi matematici già pronti;
- evitare problemi di unboxing.

Le operazioni principali che si possono applicare sono:

- **rangeClosed(min,max)** → creazione dello Stream con gli interi che vanno da min a max;
- **sum()**;
- **average()**;
- **min()**;
- **max()**.

DESIGN PATTERN: COMPOSITE

(INTENTO) Il **Composite** è un **design pattern strutturale**, che permette di comporre oggetti in strutture ad albero, così facilitando la **rappresentazione di gerarchie**.

Questo design pattern permette al client di trattare composizioni e oggetti singoli allo stesso modo.

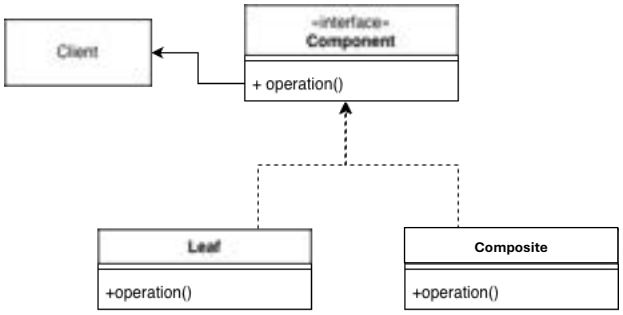
```
Cartella
 ├─ File
 ├─ File
 └─ Cartella
     ├─ File
     └─ File
```

In questo caso vorremmo poter utilizzare sia cartella che file allo stesso modo. Potremmo ad esempio stabilire che sia cartella(che contiene al suo interno una lista di file) e sia file implementino un metodo di tipo size.
Nel file ritorna il numero di byte occupati o comunque la dimensione del file, invece, nella cartella si calcoli la dimensione di ogni file contenuto.
Come accennato prima, tutti e due hanno lo stesso metodo in comune ma implementato in modo diverso.

(MOTIVAZIONE) Se ci sono due diverse implementazioni tra classi per semplici elementi e classi contenitori di questi semplici elementi, il codice è complicato. Infatti, si hanno delle ridondanze è una gestione che potrebbe risultare incoerente nel caso in cui stiamo utilizzando o l'uno o l'altro tipo di elemento.

Composite permette di creare una composizione ricorsiva, senza che il client debba fare distinzione tra tipi.

La soluzione prevede l'impiego di tre elementi fondamentali(SOLUZIONE):



- **COMPONENT** → interfaccia che rappresenta elementi semplici e non, dichiarando le operazioni che devono essere compiute dagli oggetti. Eventualmente possono essere contenute operazioni che permettono di gestire i figli come: add(), remove(), getChild().
- **LEAF(FOGLIA)** → classe che implementa gli elementi semplici, che non ha figli e implementa direttamente le operazioni da effettuare.
- **COMPOSITE** → classe che rappresenta gli elementi contenitori. Quest'implementazione è sempre sottoclasse di Component e può contenere strutture dati di figli, implementando operazioni che gestiscono gli element child.

Riprendiamo quindi la nozione di associazione e le diverse variazioni:

- **Associazione → una classe usa l'altra, tutte e due esistono anche senza la relazione**;
- **Aggregazione** → una classe detta whole aggrega le altre parti. Il **composite** in questo caso è l'aggregatore e **leafs** sono le parti, e ambedue **possono esistere separatamente**.
- **Composizione → la parte non esiste senza l'aggregazione(whole)**. La parte esiste se l'intero esiste ed è distrutta quando l'intero è distrutto.

Le conseguenze che si hanno all'applicazione del sopraccitato design pattern sono(CONSEGUENZE):

- **Composizione ricorsiva** → costruzione ricorsiva di strutture gerarchiche in modo totalmente semplice;
- **Uniformità nel trattamento** → il client potrà utilizzare la composizione come se stesse utilizzando una singola foglia;
- **Estendibilità del codice** → si possono aggiungere svariati tipi di leaf e composite a patto che non si rompa il codice;
- **Difficoltà nel vincolare i tipi** → non ci sono dei controlli automatici a run-time che verificano se l'istanza di Component che verrà immessa nella composizione sia effettivamente valida(CONTRO).

Per un utilizzo più consapevole potrebbe essere una buona pratica passare il riferimento all'istanza di leaf con l'oggetto che la contiene. La gestione del riferimento si può fare con opportuni metodi, previsti o non dal Component.

Se sono contenuti all'interno del Component si ha maggiore **trasparenza** ma potrebbe non essere sicuro, dato che un metodo potrebbe avere senso utilizzarlo nell'istanza di Composite e non in Leaf(potrebbe causare errori se il metodo non è stato pensato per Leaf). Di solito si **evita la trasparenza** e si **preferisce** una maggiore **sicurezza** all'interno del codice.

Si potrebbe inserire dei metodi specificati nel Component, quali:

- **getComposite()** → se il client vuole capire se sta lavorando con una Leaf o un Composite come fa? Se volesse aggiungere nuovi elementi e non sa con che tipo di istanza sta lavorando? Si ritorna null nel caso in cui si stia richiamando da una Leaf e il Composite stesso se si sta richiamando la struttura gerarchica.
- **metodi di ordinamento** → l'ordinamento potrebbe essere un requisito necessario per il Composite.
- **Cache** → potrebbe essere utile conservare delle informazioni frutto di calcoli precedenti già effettuati. Infatti, se dobbiamo eseguire operazioni costose può essere conveniente:
    - calcolare il risultato;
    - conservarlo all'interno della memoria;
    - all'occorrenza riutilizzarlo;
    - se la memoria non è valida viene ricalcolato.

COMPONENT

```java
interface File{
    public int size();
    public String getName();
    public File getComposite();
}
```

LEAF

```java
class NormalFile implements File{
    String name;
    int size;
    public NormalFile(String s){
        name = s;
        size = (int)(Math.random()*100);
    }
    public int size(){return size;}
    public String getName(){return name;}
    public File getComposite(){return null;}
}
```

IMPLEMENTAZIONE DEI METODI DELL'INTERFACCIA COMPONENT

COMPOSITE

```java
class Directory implements File{
    String name;
    int size;
    boolean dirty;
    List<File> dir;
    Directory parent;

    public Directory(String s, Directory p){
        name = s;
        size = 0;
        dirty = true;
        parent = p;
        dir = new ArrayList<>();
    }
```

```java
public boolean add(File fileToAdd){
    if(alreadyExists(fileToAdd.getName())) return false;
    if(fileToAdd.getComposite() != null){
        ((Directory)fileToAdd).parent = this;
    }
    boolean added = dir.add(fileToAdd);
    if(added) invalidateCache();
    return added;
}

public boolean remove(String fileToRemove){
    boolean removed = dir.removeIf(x -> x.getName().equals(fileToRemove));
    if(removed) invalidateCache();
    return removed;
}
```

METODI PER LA GESTIONE DELLA COMPOSIZIONE

```java
public int size(){
    if(dirty){
        dirty = false;
        size =  dir.stream()
        .map(x -> x.size())
        .reduce(identity: 0,(accum, x) -> accum +x);
    }
    return size;
}

private void invalidateCache(){
    dirty = true;
    if(parent != null){
        parent.invalidateCache();
    }
}
```

METODO SIZE → implementa metodo dell'interfaccia

METODO INVALIDATECACHE → utile per segnalare modifiche a tutto il resto dell'albero

DESIGN PATTERN: DECORATOR

Il **decorator** è un **design pattern strutturale**, che **permette di aggiungere dinamicamente funzionalità ad un oggetto senza modificare funzionalità della sua classe**.

(INTENTO)Il decorator è un'alternativa flessibile all'implementazione di sottoclassi per estendere le funzionalità.

(MOTIVAZIONE) Molte volte si vuole aggiungere una o più funzionalità a singoli oggetti che all'intera classe, e mediante l'utilizzo di ereditarietà questo si fa staticamente. Se volessimo farlo in modo dinamico questa è una limitazione perché il client non può decidere come e quando utilizzare la funzionalità.

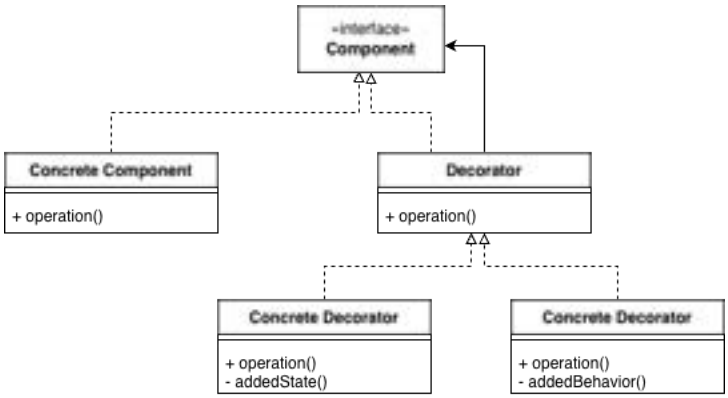
L'approccio più flessibile consisterebbe nel **chiudere**(**wrappare**) l'**oggetto all'interno di un altro oggetto**, detto **decorator**, che permette di utilizzare le ulteriori funzionalità.

Le funzionalità possono essere aggiunte o rimosse in modo dinamico e a volte si possono annidare più decorator per combinare comportamenti diversi.

Questo approccio evita l'esplosione di sottoclassi che verrebbero create per prevedere tutte le funzionalità.

L'oggetto che racchiude chiamerà le funzioni dell'oggetto racchiuso e aggiunge altre attività.

(SOLUZIONE) I protagonisti di questo design pattern sono:



- **COMPONENT→** interfaccia comune a tutti a cui possono essere aggiunte dinamicamente responsabilità;
- **CONCRETE COMPONENT →** oggetto base a cui possono essere aggiunte responsabilità;
- **DECORATOR →** mantiene il riferimento ad un oggetto di tipo component e implementa la stessa interfaccia, delegando così le implementazioni delle funzionalità al concrete decorator;
- **CONCRETE DECORATOR →** estende il decorator e implementa le responsabilità aggiuntive, modificando o arricchendo il comportamento del componente.

Solitamente il decorator è una classe astratta che viene poi implementata dai Concrete Decorator, ma questa è solo una convenzione di implementazione e non un vincolo imposto dal pattern.

(CONSEGUENZE) C'è molta più flessibilità rispetto all'utilizzo dell'ereditarietà, infatti, si può avere un Component che inizialmente è un Component ma che poi può implementare altre funzionalità a **runtime**. Questo è possibile attraverso la fase di wrapping. Un esempio può essere:

- **EREDITARIETA' →** O si utilizza Caffè o CaffèConLatte si decide a compile time;
- **DECORATOR →** Inizialmente si utilizza Caffè e poi in un secondo momento estendere le funzionalità di CaffèConLatte nel seguente modo: m = new CaffeConLatte(currentCoffee);

Una responsabilità può essere aggiunta più volte, wrappando più volte l'istanza con lo stesso decorator. Ad esempio:

```
Text t = new Text("ciao");
t = new BoldDecorator(t);          // 1 volta → <b>ciao</b>
t = new BoldDecorator(t);          // 2 volte → <b><b>ciao</b></b>
```

Importante è ricordare che l'istanza dell'oggetto è diversa dall'istanza con il wrapper applicato, quindi:

Component c1 = new ConcreteComponent();

Component c2 = new ConcreteDecorator1(c1);

→ c1 != c2

I vantaggi sono legati alla flessibilità e ad avere un codice molto più modulare e senza troppe sottoclassi.

Gli svantaggi sono legati alla complessità introdotta e alla difficoltà a capire l'identità dell'oggetto.

COMPONENT

```
interface Messaggio{
    public String getDestin();
    public String getTesto();
}
```

DECORATOR

```
class Modificatore implements Messaggio{
    private Messaggio p;

    public Modificatore(Messaggio c){
        p = c;
    }

    public String getDestin(){
        System.out.println(x: "[Modificatore.getDestin] ");
        return p.getDestin();
    }

    public String getTesto(){
        System.out.println(x: "[Modificatore.getDestin] ");
        return p.getTesto();
    }

}
```

CONCRETE COMPONENT

```
class Testo implements Messaggio{
    private String testo;
    private String dest;

    public Testo(String t, String d){
        testo = t;
        dest = d;
    }

    public String getDestin(){
        System.out.println(x: "[Testo.getDestin]");
        return dest;
    }

    public String getTesto(){
        System.out.println(x: "[Testo.getTesto]");
        return testo;
    }

}
```

CONCRETE DECORATOR 1

```
class Indirizzi extends Modificatore{
    public Indirizzi(Messaggio c){
        super(c);
    }

    public String getDestin(){
        System.out.println(x: "[Indirizzi.getDestin]");
        String s = super.getDestin();
        s = s.replaceFirst(regex: "Sara", replacement: "3336543211");
        s = s.replaceFirst(regex: "Tara", replacement: "3939685732");
        return s;
    }

}
```

CONCRETE DECORATOR 2

```
class Riduttore extends Modificatore{
    public Riduttore(Messaggio c){
        super(c);
    }

    public String getTesto(){
        System.out.println(x: "[Riduttore.getTesto] ");
        String s = super.getTesto();
        s = s.replaceAll(regex: ":", replacement: "");
        s = s.replaceAll(regex: "four", replacement: "4");
        s = s.replaceAll(regex: "going to", replacement: "gonna");
        s = s.replaceAll(regex: "would you", replacement: "wanna");
        return s;
    }
}
```

DESIGN PATTERN: BRIDGE

Il **Bridge** è un **design pattern strutturale** che ha l'obiettivo di **separare un'astrazione dalla sua implementazione**, permettendo alle due di evolvere in modo indipendente(INTENTO).

Solitamente, quando si utilizza l'ereditarietà, si utilizza una classe astratta che definisce l'astrazione generale, mentre le sottoclassi forniscono implementazioni specifiche.

Questo approccio non è flessibile perché collega direttamente e in modo permanente l'astrazione all'implementazione, rendendo difficile modificare o estendere le due parti(MOTIVAZIONE).

Un esempio utile a comprendere il pattern potrebbe essere:

"Si hanno varie forme che si possono disegnare e vari modi per disegnarle" → Se dovessimo farlo con l'ereditarietà avremmo la classe "Forme" e poi tutte le combinazioni tra forme e modalità di disegno:

1) CerchioFile;
2) CerchioDisegno;
3) CerchioProiezione;
4) RettangoloFile;
5) RettangoloDisegno;
6) RettangoloProiezione;
7) RomboFile;
8) RomboDisegno;
9) RomboProiezione;

Questo approccio non è scalabile, perché all'aumentare delle forme o modalità di disegno, il numero delle classi cresce in modo combinatorio.

Se separassimo l'astrazione dall'implementazione, si otterrebbe una struttura più flessibile composta da:

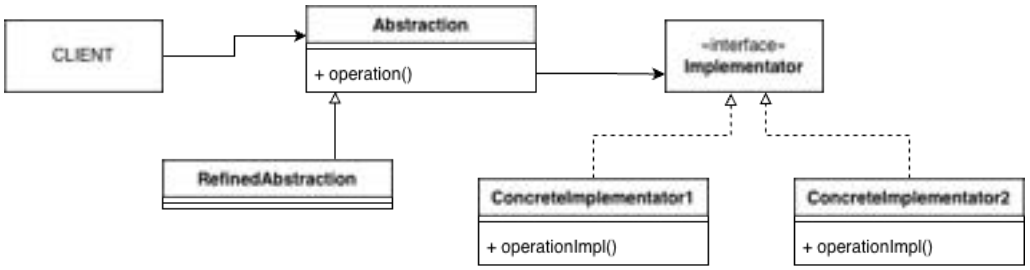
- Forme: Cerchio, Rettangolo, Rombo;
- Modalità di disegno: File, Disegno, Proiezione.

Le forme non implementano direttamente il modo in cui vengono disegnate, ma delegano ad un oggetto separato tale responsabilità.

Come abbiamo visto, **si vuole evitare la proliferazione di classi**, avendo una struttura dove all'aumento di variazioni: l'**incremento è lineare**.

I protagonisti principali di questo design pattern sono(SOLUZIONE):

- **Abstraction** → definisce l'interfaccia generale, **mantenendo un riferimento a un Implementor** e delegando il lavore all'implementazione → spiega cosa fare ma non in che modo.
- **RefinedAbstraction** → versione concreta dell'astrazione → specializza il comportamento.
- **Implementor** → definisce le operazioni primitive e non conosce l'astrazione → nell'esempio sa come disegnare ma non sa cosa sta disegnando.
- **ConcreteImplementor** → implementa l'interfaccia di Implementor e fornisce le operazioni concrete.



In questo modo si ottengono ulteriori separazioni e livelli di astrazione maggiore, permettendo all'implementazione di non essere totalmente connessa ad un'interfaccia.

Inoltre, si hanno degli ulteriori benefici, dato che cambiando l'implementazione non è necessario ricompilare l'Abstraction e i Client(viene poi associata l'implementazione a runtime).

Con questo livello di separazione, le gerarchie di Abstraction e Implementor possono evolvere in modo indipendente(CONSEGUENZE).

ABSTRACTION

```java
abstract class Telecomando{ // ABSTRACTION
    protected Dispositivo dispositivo;
    public Telecomando(Dispositivo d){
        dispositivo = d;
    }

    public void accendi(){
        dispositivo.accendi();
    }

    public void spegni(){
        dispositivo.spegni();
    }
}
```

IMPLEMENTOR

```java
interface Dispositivo{
    public void accendi();
    public void spegni();
    public void setVolume(int v);
    public int getVolume();
}
```

REFINED ABSTRACTION

```java
class TelecomandoSemplice extends Telecomando{ // REFINED ABSTRACTION
    public TelecomandoSemplice(Dispositivo d){
        super(d);
    }

    public void aumentaVolume(){
        int volume = dispositivo.getVolume();
        System.out
        .println("Volume prima dell'incremento: " + volume);
        dispositivo.setVolume(volume+1);
        System.out
        .println("Volume dopo l'incremento:" + dispositivo.getVolume());
    }

    public void decrementaVolume(){
        int volume = dispositivo.getVolume();
        System.out
        .println("Volume prima del decremento: " + volume);
        dispositivo.setVolume(volume-1);
        System.out
        .println("Volume dopo il decremento:" + dispositivo.getVolume());
    }
}
```

CONCRETE IMPLEMENTOR

```java
class TV implements Dispositivo{ // CONCRETE IMPLEMENTOR
    public int volume;

    public TV(){
        volume = 50;
    }

    public void accendi(){
        System.out.println(x: "TV accesa");
    }
    public void spegni(){
        System.out.println(x: "TV spenta");
    }

    public void setVolume(int v){
        if(v > 100){
            volume = 100;
        }else if(v < 0){
            volume = 0;
        } else{
            volume = v;
        }
    }


    public int getVolume(){ return volume;}
}
```

Come mostrato nel codice, il Bridge prevede il collegamento tra Abstraction e Implementor, possibile mediante un **riferimento all'Implementor all'interno dell'Abstraction**. Il passaggio di questo elemento può essere fatto tramite costruttore o altri meccanismi di inizializzazione.

PROCESSI DI SVILUPPO DEL SOFTWARE

Un processo software descrive l'insieme delle attività e dei task necessari per sviluppare un prodotto software. Sostanzialmente, stabilisce i requisiti a un sistema funzionante e mantenibile.

Possiamo suddividere il processo di produzione del software nelle seguenti fasi:

- **Analisi dei requisiti** → si raccolgono e definiscono le esigenze del cliente e degli utenti → l'output è la **specifica dei requisiti**(cosa deve fare il sistema) → sono presenti requisiti funzionali e non;
- **Progettazione (Design)** → si definirà come il sistema realizzerà i requisiti, ovvero:
  - architettura del sistema;
  - struttura dei componenti;
  - scelta dei pattern/tecnologie;
- **Implementazione(Codifica)** → scrittura del software in un linguaggio di programmazione;
- **Convalida/Testing** → test unitari, di integrazione, di sistema → individuazione dei bug e difetti;
- **Manutenzione** → correzione di errori, aggiornamenti ed evoluzione del sistema(si aggiungono nuove funzionalità e se ne rimuovono alcune non più utili);

L'analisi dei requisiti è la fase che porta a definire le specifiche, stabilisce i servizi richiesti e i vincoli del software. Si deve fare una netta separazione tra:

- **requisiti** → le caratteristiche che il software deve avere → tendono ad essere granulari (molti e piccoli);
- **specifica** → descrizione rigorosa delle caratteristiche del software;
- **feature** → un insieme di requisiti correlati tra loro, permettendo di soddisfare un obiettivo comune.

Per l'ingegneria dei requisiti si definiscono le seguenti fasi:

1) **Studio di fattibilità** → serve a capire se il progetto si può fare: tecnicamente, economicamente e temporalmente;
2) **Analisi dei requisiti** → si raccolgono i requisiti dalle parti interessate(stakeholder) → interviste, questionari, ecc.;
3) **Specifica dei requisiti** → si documentano i requisiti in modo chiaro e non ambiguo;
4) **Convalida dei requisiti** → i requisiti devono essere: corretti, completi, coerenti e non ambigui.

I requisiti possono essere:

- **funzionali** → cosa il sistema deve fare;
- **non-funzionali** → come il sistema lo fa (affidabilità, efficienza, prestazioni, ...).

In fase di progettazione, si stabilisce la struttura software che implementa le specifiche.

Inizialmente si suddividono i requisiti e vengono organizzati, in modo tale da definire una struttura generale del sistema → moduli, componenti e sottosistemi → **progettazione architetturale**.

Ad ogni sistema viene assegnata la responsabilità e quindi quali compiti deve gestire → **separazione delle responsabilità**. Infine, viene fatta una progettazione dettagliata definendo: interfacce, componenti, strutture dati e algoritmi.

Ogni attività produce in output un modello, spesso UML → diagrammi di sequenza, di stato, modello strutturale e data flow(flusso di dati).

Progettazione e implementazione spesso vengono fatte contemporaneamente, infatti: mentre si implementa si può migliorare il design e correggere eventuali errori progettuali.

L'implementazione consiste nella codifica dei modelli del progetto e della rimozione degli errori di un programma. Durante la fase di sviluppo si effettuano test sul codice per individuare:

- bug logici;
- errori di implementazione;
- comportamenti non previsti.

Per la rimozione degli errori, vale a dire il processo di debugging, si seguono queste linee guida:

1. localizza gli errori nel codice;
2. correggere il modello se necessario;
3. applicare la correzione del codice;
4. rieseguire i test.

Dopo aver finito la fase di implementazione, si segue con la fase di convalida o Verifica e Validazione(VV) che prevede:

- **Verifica** → si controlla se il **software** sia **costruito correttamente rispetto alle specifiche** → sto costruendo il sistema nel modo giusto? Questo può contemplare diverse azioni come: **revisione del codice**, **controllo dei documenti** e **test tecnici**;
- **Validazione** → si controlla se il software sia adatto alle esigenze del cliente e dell'utente finale → sto costruendo il sistema giusto? Le azioni che si seguono solitamente per la Validazione sono: **test con utenti** e **test su scenari reali**.

La fase di test serve a verificare il comportamento del software mediante esecuzioni, individuando errori e controllando la conformità ai requisiti. I test sono organizzati per livelli, dal più semplice al più complesso.

I test, quindi, possono essere suddivisi in questo modo:

- **Test di componenti o unità** → singoli componenti testati indipendentemente → funzioni, classi e moduli;
- **Test di sistema** → si testa l'intero sistema → il focus è rivolto ai comportamenti che derivano dall'interazione tra componenti;
- **Test di accettazione(alpha test)** → test utili a capire se il sistema soddisfi i requisiti del cliente → svolti in ambienti controllati, dagli sviluppatori stessi o con il cliente;
- **Beta test** → effettuati da utenti reali con sistema quasi completo;

Durante queste fasi vengono prodotte varie versioni del software come: **alpha**, **beta** e **gold**.

La **versione aplha** è la versione che prevede il **sistema integrato ma incompleto** e con molti bug.

Segue la **versione beta** o meglio una **versione con tutte le feature implementat**e con ancora **tanti difetti** presenti.

Infine, si ha la **versione gold**, cioè una **versione finale stabile** e **pronta per il rilascio**.

Il percorso che si segue per lo sviluppo di un sistema software è il seguente:

Specifica dei requisiti → Design del sistema → Design dettagliato → Codice e test → Test di sistema → Test di accettazione

Il processo software è generalmente sequenziale, ma può essere iterativo, con ritorni a fasi precedenti fino al completamento corretto delle attività.

Come già abbiamo detto, il software dato che non ha fisicità può essere cambiato nel tempo a seconda delle esigenze. I sistemi software, infatti, non si usurano fisicamente ma posso diventare **obsoleti**, e l'obsolescenza può essere contenuta mediante aggiornamenti e modifiche.

Il software deve cambiare perché cambiano i requisiti, che a loro volta dipendono dall'ambiente in cui il sistema opera. I cambiamenti tipici sono dati da:

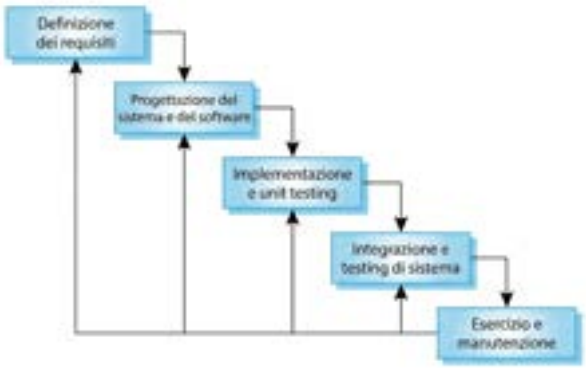
- regole aziendali;
- nuovi servizi richiesti;
- nuovo hardware;
- nuove esigenze(ad esempio lo scrolling di tiktok che poi è stato adottato in tutti i social)
- leggi(privacy).

Se il software non dovesse evolvere potrebbe: perdere di valore, diventare inutilizzabile e non soddisfare più i requisiti.

Come vengono gestiti questi cambiamenti nel processo di sviluppo? Scegliere un processo di sviluppo adeguato al nostro problema. Ci sono diversi tipi di processi di sviluppo e quelli analizzati sono: **processo a cascata**, **processo evolutivo**, **processo di sviluppo incrementale**, **processo CBSE** e **a Spirale**.

PROCESSO A CASCATA

Uno dei primi processi creati è stato il **processo a cascata**, cioè il processo che prevede uno **sviluppo sequenziale e rigido**, in cui le fasi sono seguite una dopo l'altra. I **requisiti** di questo modello **devono essere definiti completamente all'inizio**.



Focalizzandosi sul prodotto completo si passa alla fase successiva solo quando la precedente è completata.

Lo schema tipico che si segue è: si specifica tutto → si progetta tutto → si implementa tutto → si testa tutto.

Questo è un progetto statico con tanta documentazione, dove si hanno poche interazioni con i clienti e si hanno difficoltà ad introdurre i cambiamenti richiesti.

Gli svantaggi sono quindi dovuti alla rigidità e ai tempi lunghi per ottenere il prodotto finale.

I vantaggi sono:

- **consistenza tra gli artefatti** → ogni fase produce documenti ben definiti e questo garantisce coerenza tra requisiti, design e implementazione;
- **ampia documentazione** → facilità la manutenzione futura permettendo di capire il sistema agli sviluppatori futuri;
- **adatto se i requisiti sono stabili** → si evitano modifiche costose → ad esempio, un'applicazione del meteo una volta fatta non necessita di modifiche eccessive;
- **adatto a sistemi grandi, complessi e critici** → controllo rigoroso, documentazione completa → sistemi bancari.

Come si può vedere nell'immagine proposta precedentemente, le varie fasi vengono eseguite a cascata e solo dopo che si arriva all'ultima fase, per eventuali aggiornamenti si torna nelle fasi precedenti.

PROCESSO EVOLUTIVO

È un modello di sviluppo in cui il software viene costruito progressivamente, attraverso versione successive. Non viene definito tutto all'inizio bensì il sistema evolve nel tempo.

Questo processo di sviluppo e adatto in contesti in cui o i requisiti non sono completamente noti oppure cambiano frequentemente.

Le varianti del processo evolutivo sono:

- **sviluppo per esplorazione** → gli sviluppatori lavorano a stretto contatto con il cliente, partendo da una versione iniziale e migliorando il prodotto tramite trasformazioni successive. Si parte solitamente da pochi requisiti chiari che aumentano nel tempo e si raffinano.
- **sviluppo build and fix** → si costruisce una prima versione che viene modificata, corretta ed estesa. Questo passaggio viene ripetuto molte volte fino a quando il cliente è soddisfatto.

La **flessibilità** e lo **stretto contatto con il cliente** sono i **vantaggi principali** di questo processo di sviluppo, ma la quasi **assenza di documentazione potrebbe diventare un problema**, soprattutto in sistemi molto grandi.

Infatti, i sistemi sviluppati secondo questo approccio sono difficilmente comprensibili e modificabili, probabilmente non corretti. Un ulteriore problema è la mancanza di visione d'insieme del progetto. Ci si potrebbe approcciare a questo modello di sviluppo quando si hanno sistemi di piccole dimensioni, sistemi con vita breve(ad esempio bozze di applicazioni) o singole parti di grandi sistemi.

PROCESSO DI SVILUPPO INCREMENTALE

Il **processo di sviluppo incrementale** è un modello in cui il **sistema viene costruito a pezzi**, appunto **incrementi**. L'idea alla base e che non si sviluppa tutto insieme, ma parti singole quasi in modo indipendente e che poi vengono collegate. Nelle versioni successive si aggiungono nuove funzionalità.

Il funzionamento di questo modello prevede le seguenti fasi:

- si implementano le funzionalità di base;
- si rilascia una prima versione;
- si aggiungono nuove funzionalità → ogni incremento è funzionante, testato e utilizzabile;
- si ripete il processo fino al completamento.

I vantaggi sono i seguenti:

- rilascio rapido di versioni funzionanti;
- maggiore flessibilità;
- gestione migliore dei requisiti.

Gli svantaggi sono legati all'integrazione progressiva che potrebbe essere complessa e poco coerente se mal gestita.

PROCESSO CBSE(COMPONENT-BASED SOFTWARE ENGENEERING)

Il modello CBSE è un modello basato sul riuso di componenti esistenti. Infatti, è un modello basato su COTS(commercial off-the-shelf) vale a dire sull'utilizzo di software già pronti come: librerie, moduli e framework.

Questo processo prevede un'analisi dei componenti esistenti che soddisfano i requisiti, e nel caso in cui non collidono perfettamente i requisiti possono essere adattati ai componenti disponibili. Quindi: non sempre si sviluppa ciò che si vuole ma si può adattare il sistema a ciò che già esiste.

I vantaggi sono legati alla riduzione dei costi e tempo di sviluppo, oltre all'affidabilità maggiore, ma lo svantaggio è la dipendenza da componenti esterni e l'adattamento dei requisiti.

MODELLO A SPIRALE

Il **modello a spirale** è uno dei più importanti perché **introduce il concetto di gestione del rischio**. Questo **modello** è **iterativo ed evolutivo** che combina:

- **sviluppo incrementale;**
- **attenzione ai rischi.**

L'idea alla base è che: non si deve sviluppare tutto insieme bensì si producono **prodotti parziali**(sottosistemi funzionanti) ed **a ogni ciclo si aggiunge qualcosa al sistema**.

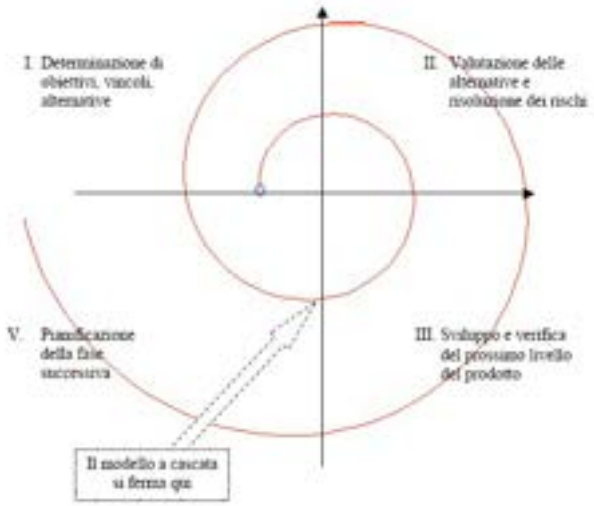
Ogni ciclo è composto da 4 attività fondamentali:

- **definizione degli obiettivi** → si stabiliscono gli obiettivi della fase e le funzionalità da sviluppare;
- **valutazione dei rischi** → valutare se c'è qualcosa di incerto che può compromettere il successo del progetto → si cercano soluzioni ai rischi;
- **sviluppo e convalida** → si realizza una parte del sistema → si testa e si valida;
- **revisione e pianificazione** → si valuta quanto fatto e si pianifica il ciclo successivo → è compreso anche il feedback del cliente.

I vantaggi sono i seguenti:

- gestione esplicita dei rischi;
- possibilità di interagire con il cliente;
- rilascio rapido di versioni iniziali;
- ogni fase produce codice testato e integrato.

Gli svantaggi sono direttamente legati alla complessità introdotta da questo modello, infatti sono: economici ed esperienza richiesta per la valutazione dei rischi.



Manifesto for Agile software development

Il manifesto Agile definisce un approccio diverso allo sviluppo del software, basato su:

- flessibilità;
- collaborazione;
- adattamento continuo.

Il manifesto stabilisce quattro principi chiave:

- gli **individui** e le **interazioni** sono più **importanti** dei processi e degli strumenti;
- il **software funzionante** è più importante della documentazione;
- la **collaborazione con il cliente** è più importante della negoziazione dei contratti;
- **rispondere** prontamente **ad un cambiamento** è più importante di seguire un piano;

L'approccio **Agile**(rapido, flessibile) si traduce in alcune pratiche concrete come: accettare i cambiamenti anche in fase di sviluppo, rilasciare frequentemente software funzionante e favorire team auto-organizzati, collaborativi e motivati.

L'Agile non è un processo rigido ma una mentalità che **elimina la pianificazione bloccata** ed **introduce l'adattamento continuo**.

L'**XP** o **Extreme Programming** è una **metodologia Agile** che punta sullo sviluppo rapido, sulla qualità del codice e la forte collaborazione tra team e cliente. Le caratteristiche principali dell'XP sono:

- **sviluppo incrementale** → software costruito e rilasciato in piccoli incrementi di funzionalità;
- iterazioni brevi e gruppi ridotti;
- poca documentazione a fronte di un software funzionante;
- testabilità dei prodotti sin dall'inizio.

L'XP è utilizzato o particolarmente adatto quando: i requisiti cambiano spesso, il progetto è innovativo o incerto, ci sono tempi di consegna stretti oppure i rischi sono elevati(requisiti poco chiari).

Come deve essere un software prodotto seguendo l'XP? Deve essere: semplice, testato di continuo(anche con l'interazione con l'utente), facilmente modificabile e di qualità eccezionale.

La documentazione è quasi totalmente abbandonata, se non per l'utilizzo di:

- **Story Card** → schede leggere usate per descrivere i requisiti dal punto di vista del cliente → scritte dal cliente o con il cliente → descrivono un'esigenza in 2-3 fasi e devono permettere di stimare il tempo di sviluppo → possono avere una priorità.

Solitamente c'è un modello da seguire per le story card, composto da:

  - data, numero, priorità, tempo stimato;
  - descrizione del requisito(2-3 fasi);
  - tasks per ciascun requisito;
  - note;

  → il foglio rappresentato nell'immagine è 15x7cm
- **CRC cards** → servono per progettare il sistema a livello di classi → aiutano a capire chi fa cosa nel sistema. Le CRC cards contengono: il nome della classe(class), la responsabilità(responsibility cosa fa) e con quali altri classi collabora(collaboration)

La differenza importante che si deve fare è che: le story cards servono a capire cosa vuole il cliente, invece, le CRC come si realizza nel codice.

Nell'**Extreme Programming** ci sono **12 pratiche**, che rappresentano un insieme di linee guida operative che traducono i principi dell'Agile in attività concrete di sviluppo. Esse migliorano la qualità del software, favorendo una collaborazione tra team e cliente e permettendo una rapida adattabilità ai cambiamenti.

L'obiettivo è creare un processo leggero ma efficace, basato su iterazioni brevi, feedback continuo e rilascio frequente di software funzionante, riducendo i rischi e permettendo ai team di adattarsi ai cambiamenti continui.

Di seguito sono elencate le 12 pratiche, che verranno approfondite paragrafo per paragrafo:

- **gioco di pianificazione;**
- **pair programming;**
- **piccole release;**
- **metafora → utilizzare una singola metafora per rendere facile la comprensione di ciò che si sta facendo e potrebbe mettere a suo agio il cliente;**
- **testing;**
- **refactoring;**
- **cliente in sede → non è una figura non esterna, ma collabora attivamente ogni giorno;**
- **design semplice;**
- **possesso del codice collettivo → chiunque trova un errore lo sistema perché tutti sono responsabili dell'intero sistema;**
- **integrazione continua → tutti i test devono essere superati, se un test fallisce la coppia che ha prodotto il codice deve riparlo(se non si risolve si butta il codice);**
- **settimana di 40 ore → ore extra di lavoro è sintomo di un problema serio → troppo lavoro → pianificazioni frequenti evitano a ciascuno di avere troppo lavoro;**
- **usare gli standard per il codice;**

GIOCO DI PIANIFICAZIONE

Il **planning game** è la pratica con cui, in XP, si decide cosa fare e quando svilupparlo, attraverso la **collaborazione continua tra clienti e sviluppatori**.

Questa interazione avviene mediante le story card, ossia delle card dove l'utente brevemente specifica cosa vuole. Se la storia è troppo grande essa viene suddivisa in storie più piccole.

Gli addetti al business stimano la data di una release, cosa deve prevedere e le priorità di ogni task. Gli sviluppatori dividono le story card in **task**, che poi verranno implementati da singoli sviluppatori o interi team.

Infine, la release non deve essere pianificata troppo in avanti, ma fissata in un arco temporale breve, in modo che il prodotto sia già utilizzabile e il cliente non debba attendere a lungo. Il rilascio frequente consente di ottenere feedback immediato, permettendo di apportare rapidamente eventuali modifiche ed evitando lo sviluppo di sistemi troppo complessi e difficili da adattare nel caso in cui il cliente non sia soddisfatto.

La priorità nell'implementazione dei task viene data ai task più rischiosi.

PAIR PROGRAMMING

Il **pair programming** è una pratica in cui **due sviluppatori lavorano insieme sullo stesso codice**, condividendo la stessa postazione. I ruoli della coppia sono i seguenti:

- **driver** → utilizza tastiera e mouse per implementare concretamente al meglio ciò che è richiesto nei tasks.
- **navigator** → osserva e guida il lavoro, pensando alla correttezza dell'approccio e alla creazione di eventuali test → suggerisce soluzioni semplici e migliori.

I **ruoli** non sono però rispettati rigidamente, infatti solitamente **vengono invertiti per mantenere alta la concentrazione e coinvolgere** entrambi. Gli ulteriori benefici ottenuti dall'inversione sono: conoscenza approfondita del sistema, la disciplina nello sviluppo e la riduzione degli errori.

PICCOLE RELEASE

Avere piccole release sviluppate in tempi brevi (1-2 settimane) che implementano solo le funzionalità essenziali in quel momento, può portare ad avere molti vantaggi.

Questi vantaggi, come già citato prima, sono:

- feedback valido dal cliente;
- nascita di fiducia da parte del cliente;
- dare agli sviluppatori un senso concreto di progresso;
- ridurre i rischi;
- adattarsi facilmente ai cambiamenti richiesti.

L'idea chiave è quindi: **rilasciare spesso e in piccolo è meglio che rilasciare raramente e in grande**.

DESIGN SEMPLICE

In XP, il design del SW deve essere il più semplice possibile, ma comunque sufficiente a soddisfare i requisiti attuali. L'idea principale è che: **non si progetta per il futuro, ma per il presente**.

Un design semplice è tale quando:

- codice non duplicato;
- esprime chiaramente le intenzioni dello sviluppatore;
- usa il minore numero di classi e metodi.

Non si costruisce qualcosa in vista del futuro, ma del presente stesso evitando complessità inutili.

Usare le **CRC(card responsibility collaboration) cards** aiutano a progettare in modo semplice, permettendo di: ragionare meglio in termini di oggetti e avere una visione globale del sistema.

TESTING

Nel XP, il testing non è una fase finale bensì un'attività centrale e continua durante tutto lo sviluppo. Si testa quindi tutto ciò che può fallire sin dalle prime fasi, il più spesso possibile.

Un requisito è soddisfatto solo se il test passa e quindi il test ha un ruolo importante dato che poi diventerà utile per: documentazione, verifica e guida allo sviluppo.

I test vengono eseguiti più volte al giorno e soprattutto quando viene aggiunto nuovo codice, per questo motivo c'è un'integrazione continua.

Ci sono due tipi di test:

- **test funzionali** → servono per verificare se l'implementazione rispecchia i requisiti proposti dal cliente. Questi test sono automatizzati ed eseguiti frequentemente, oltre ad essere parte della specifica dei requisiti.
- **unit test** → sono dei test su singole unita, su metodi e classi, e sono scritti direttamente dagli sviluppatori → garantiscono qualità tecnica → possono essere scritti anche prima dell'implementazione (principio TDD(test driven development));

L'utilizzo dei test permette di avere benefici come: riduzione degli errori, codice più mantenibile e pulito, allineamento continuo con i requisiti e feedback immediato.

STANDARD DI CODIFICA

Gli standard di codifica sono un insieme di regole condivise che guidano la scrittura del codice da parte di tutto il team. L'obiettivo è rendere il codice uniforme, leggibile e facilmente mantenibile, indipendentemente da chi lo ha scritto.

I principi di questa pratica sono:

- costruzioni non troppo complesse;
- codice scritto in modo chiaro e coerente;
- tutti i membri devono seguire le stesse convenzioni.

Si possono seguire delle convenzioni come le Java Code Convention che parlano di:

- **nomenclatura → PascalCase** per le classi e **camelCase** per metodi e variabili. Per le costanti si utilizza l'UPPER_CASE;
- **identazione** e struttura del codice → **tab** necessari per capire all'interno di quale blocco di codice ci troviamo e parentesi graffe coerenti;
- **organizzazioni all'interno delle classi** → si segue un ordine del tipo → **variabili, metodo costruttore, metodi pubblici e metodi privati**;
- utilizzo dei commenti per spiegare il perché;
- import ordinati e codice organizzato in modo leggibile e prevedibile.

CONSIDERAZIONI

XP è una metodologia Agile che si concentra principalmente su codice e test, privilegiando lo sviluppo di software funzionante rispetto ad attività accessorie come documentazione e processi complessi. È fortemente centrata sulle persone, perché la conoscenza del sistema viene condivisa tramite comunicazione diretta tra i membri del team.

È inoltre una metodologia leggera, che riduce al minimo le attività non strettamente necessarie e punta a ottenere qualità attraverso test rigorosi, feedback continuo e refactoring.

I principi di XP non sono completamente nuovi, ma rappresentano una sintesi di pratiche già esistenti nel software engineering, rese coerenti in un unico approccio.

Nel tempo, XP si è evoluta grazie all'evoluzione tecnologica e dei metodi di lavoro: dalla crescita delle community online, al mob programming, fino al lavoro remoto e all'uso di strumenti di collaborazione e intelligenza artificiale.

SCRUM

Scrum è un **framework agile** nato da Schwaber e Sutherland, progettato per aiutare i team a gestire e risolvere i problemi in modo iterativo e incrementale. Scrum abbraccia il cambiamento e promuove l'adattamento continuo attraverso i **cicli brevi** e **feedback regolari**.

Scrum è quindi una struttura di base utilizzata per generare soluzioni a problemi complessi e si regge su tre pilastri fondamentali:

- **trasparenza** → tutti gli aspetti del processo devono essere visibili e chiaro a chi: produce, decide e valuta. Tutte le parti devono capire le cose allo stesso modo. → sono presenti degli artefatti che devono essere comprensibili e sono:
    - **product backlog** → lista prioritaria di tutto ciò che serve al prodotto;
    - **sprint backlog** → cosa facciamo nello sprint e come lo facciamo;
    - **incremento** → ciò che viene prodotto nello Sprint.
- **ispezione** → il team e gli stakeholder valutano regolarmente il progresso e i risultati. L'ispezione avviene durante 5 eventi:
    - **sprint** → ciclo di lavoro a tempo fisso, solitamente 4 settimane;
    - **sprint planning** → all'inizio dello sprint, dove si decide cosa fare e come farlo;
    - **daily scrum** → si effettua ogni giorno(15 min) per verificare se lo sprint sta convergendo verso l'obiettivo;
    - **sprint review** → si effettua alla fine di ogni sprint e serve a mostrare il lavoro agli stakeholder per raccogliere feedback → momento di collaborazione con il cliente;
    - **sprint retrospective** → analisi fatta dopo la review per verificare ciò che è andato bene in: individui, interazioni, processi e strumenti. Si compila la Definition of Done per verificare quando un lavoro può essere considerato davvero completato, secondo la lista di criteri.
- **adattamento** → sulla base dell'ispezione, il team apporta aggiustamenti per migliorare.

La struttura di base di Scrum è costituita da queste figure:

- **product owner** → rappresenta gli interessi degli stakeholder(parti interessate) e dei clienti → crea e mantiene il **product backlog**(lista ordinata di tutto il lavoro da fare). Il compito del product owner è definire le priorità in base al valore di bussiness.
  È la figura centrale perché funge da intermediario tra il team di sviluppo e le parti interessate, e ha il potere di accettare/rifiutare il lavoro consegnato;
- (**scrum team) development team** → trasforma il lavoro prioritario in un prodotto funzionante → trasforma il lavoro in un incremento durante uno Sprint(iterazione)→ è responsabile della qualità del prodotto e i membri al suo interno possono essere anche multidisciplinare.
  È un componente che lavora in autonomia, dopo che vengono stabilite le tasks.
- **(scrum team) scrum master** → figura che valuta i risultati e che fa aggiustamenti per il prossimo Sprint. aiuta l'organizzazione a comprendere e adottare Scrum e rimuove gli ostacoli che impediscono il progresso.

Scrum è semplice e incompleto dato che è un framework, infatti, non sostituisce l'ingegneria del SW ma la affianca. Esso definisce: i **ruoli** citati prima(product owner, scrum master, developers), **eventi**(sprint) e **artefatti**(product backlog, sprint backlog, incremento).

Scrum è basato su:

- **empirismo** → non si pianifica tutto dall'inizio ma si impara facendo, infatti: non si assume che tutto sia prevedibile e si lavora per tentativi controllati;
- **lean(pensiero snello)** → si rimuove tutto ciò che non crea valore e ci si concentra solo sull'essenziale → esempio di sprechi: feature inutili, documentazione eccessiva, attese, bug e multitasking inefficiente.

DESIGN PATTERN: OBSERVER

Il **design pattern Observer** è un **pattern comportamentale** che ha come obiettivo di **mantenere la coerenza tra più oggetti** quando uno tra loro cambia stato. Infatti, quando uno tra essi cambia di stato tutti gli altri oggetti dipendenti vengono notificati e aggiornati automaticamente(INTENTO).

Questo design pattern è conosciuto anche come **Publish-Subscribe**, perché gli **observer**(dipendenti) si **iscrivono(subscribe)** al **subject** per ricevere aggiornamenti, mentre il subject pubblica(publish) le notifiche.

Il motivo per cui è nato il design pattern observer è(MOTIVAZIONE):

in un sistema composto da classi che cooperano, è necessario mantenere la consistenza tra oggetti correlati. Se applicassimo solo le separazioni per avere componenti pulite e riutilizzabili, in caso di cooperazione non ci sarebbe coerenza. Per questo motivo è stato sviluppato questo design pattern che permette di notificare i cambiamenti e mantenere uno stato coerente.

Ad esempio, se si ha una classe che contiene i dati (Subject) e una classe che li visualizza (Observer), la seconda dipende dalla prima. In caso di cambiamento dello stato della classe che contiene i dati, se la classe di visualizzazione non viene notificata, si genera una situazione di incoerenza, poiché essa continua a mostrare informazioni non aggiornate.

Il design pattern Observer stabilisce come deve essere fatta una relazione uno-a-molti, in modo che quando uno cambia tutti gli altri vengano notificati.

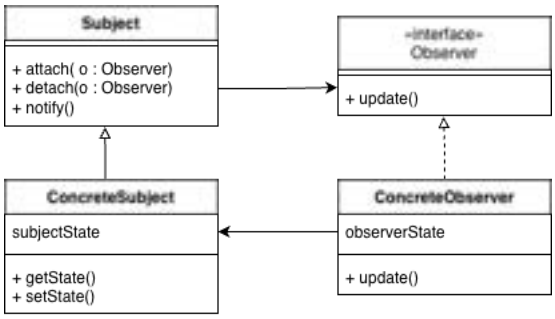
Gli oggetti principali coinvolti, come accennato prima, sono:

- il **subject**, che rappresenta l'oggetto osservato;
- gli **observer**, che dipendono dal Subject.

Un Subject può avere più Observer associati, e ogni volta che il suo stato cambia, provvede a notificare automaticamente tutti gli observer associati.

L'applicabilità è quella suddetta, cioè quando si hanno più oggetti indipendenti e un oggetto **deve notificare** gli altri **senza conoscere l'identità** → si evita il forte accoppiamento tra classi.

Come per i design pattern precedenti, analizziamo i partecipanti(SOLUZIONE):



- **subject** → classe che permette di contenere i suoi osservatori e fornisce le operazioni per la gestione: inserimento, rimozione o notifica degli Observer. Non conosce i dettagli degli observer, bensì solo l'interfaccia.
- **observer** → definisce l'interfaccia comune a tutti gli oggetti che necessitano la notifica. Al suo interno è dichiarato il metodo update() che permette di leggere lo stato dal subject e si aggiorna.
- **concreteSubject** → è la classe concreta che contiene lo stato reale → contiene la lista di observer → quando il suo stato cambia richiama il metodo notify() e avvisa tutti gli observer;
- **concreteObserver** → implementa l'interfaccia observer, mantenendo un riferimento al concreteSubject e avendo uno stato locale che deve essere coerente col subject → interroga il subject() e aggiorna il suo stato;

I vantaggi legati all'utilizzo di questo pattern(CONSEGUENZE) sono:

- **concreteSubject** e **concreteObserver non sono accoppiati**, infatti: il codice è riutilizzabile e modificabile→ si aggiungono observer senza modificare il subject e non sapere chi sono;
- **numero dinamico di osservatori** → il subject manda la notifica a tutti senza sapere chi sono e quanti sono → gli observer ricevono la notifica e decidono se ignorarla o adeguarsi;
- **il subject non dice cosa è cambiato al suo interno ma che è stata completata un'operazione** → gli observer devono consultare lo stato e verificare se quel cambiamento è indice che anche loro devono aggiornarsi.

Un'implementazione di questo design pattern potrebbe essere la seguente:

CLASSE ASTRATTA SUBJECT

```java
abstract class Subject{
    protected List<Observer> observers = new ArrayList<>();

    public void attach(Observer o){
        observers.add(o);
    }

    public void detach(Observer o){
        observers.remove(o);
    }

    public void notifyObservers(){
        for(Observer o:observers)
            o.update();
    }
}
```

INTERFACCIA OBSERVER

```java
interface Observer{
    public void update();
}
```

CONCRETE SUBJECT

```java
class WeatherStation extends Subject{
    private int temperature;
    public void setTemperature(int t){
        temperature = t;
        notifyObservers();
    }


    public int getTemperature(){return temperature;}
}
```

CONCRETE OBSERVER

```java
class PhoneDisplay implements Observer{
    private WeatherStation station;

    public PhoneDisplay(WeatherStation w){
        station = w;
        station.attach(this);
    }

    public void update(){
        int temperature = station.getTemperature();

        System.out.println("Phone display: temperatura = "+ temperature);
    }
}
```

Lo svantaggio principale è che l'**effetto a cascata** generato dall'update potrebbe portare ad avere un comportamento indefinito da prevedere.

Ci sono delle considerazioni da fare, infatti a seconda delle implementazioni reali si possono avere tante implementazioni.

Infatti, ci possono essere diversi casi che permettono delle eccezioni tipo:

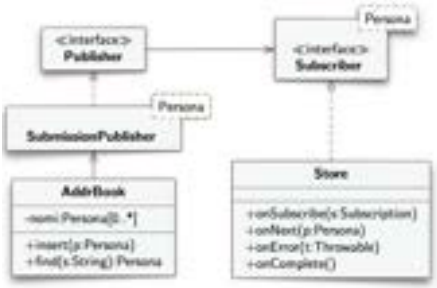
- Se ci sono tanti subject e pochi observer potremmo creare una mappa dove si ha subject → lista di observer. Questo permette di avere dei notevoli risparmi in memoria.
- Se un observer osserva più subject, si potrebbe passare il riferimento all'interno del metodo update per stabilire in quale subject è avvenuto il cambio di stato.
- Potrebbe esserci qualche caso particolare dove anche il cambiamento dell'observer potrebbe essere segnalato al subject → si potrebbe incorrere in loop infiniti.
- Si deve pensare anche il caso in cui venga distrutto il subject → si devono preavvisare gli observer per evitare dangling reference.
- Si possono anche avvisare solo i cambiamenti di cui gli observer sono interessati.

La soluzione proposta da questo pattern risolve problemi comuni, per questo motivo è fornita dalle librerie standard di Java. Si utilizzano: la **classe Observable(subject)** e l'**interfaccia Observer()**. Il problema di queste implementazioni standard è che utilizzare la classe Observable, dato che è una classe, potrebbe limitare l'ereditarietà e avere un design poco flessibile.

Il problema principale era anche legato all'utilizzo dello stesso thread sia per observer e sia per observable, per questo motivo nella versione **Reactive Streams** si introduce asincronia(mediante l'utilizzo di threads separati) e controllo del flusso. In questo modo si migliora la scalabilità del sistema e si evita il blocco del publisher.

La sincronizzazione viene gestita tramite la classe **Subscription**, che si interpone tra Publisher(interfaccia) e Subscriber.

Essa rappresenta il collegamento tra i due e permette di controllare il flusso dei dati. Infatti, viene instaurata una connessione tramite la quale il Subscriber può richiedere quanti elementi ricevere attraverso il metodo **request(n)**.



Il Publisher invia quindi le notifiche **(onNext)** in base al numero di elementi richiesti, evitando di sovraccaricare il Subscriber e realizzando così il meccanismo di **backpressure**.

SubmissionPublisher è un'implementazione concreta dell'interfaccia Publisher introdotta in Java 9, che consente l'invio asincrono di elementi ai Subscriber e supporta il meccanismo di backpressure tramite gestione del buffer.

MVC(MODEL VIEW CONTROLLER)

Un pattern architetturale che utilizza il design pattern Observer è il Model View Controller, che permette di ampliare il concetto di applicazione interattiva.

Questo pattern consente di individuare tre componenti e va oltre l'Observer:

- **Model** → gestisce i dati e lo stato dell'applicazione → indipendente da interfacce grafiche e input utente → avviene la separazione da come vengono mostrati i dati e chi li sta visualizzando(indipendente) → molto simile all'observable/subject. I compiti del Model sono quindi:
    - **registrazione di view e controller;**
    - **avvisare view e controller di eventuali cambiamenti dei dati;**
- **View** → si occupa della presentazione dei dati → possono esistere più view per gli stessi dati → molto simile all'observer.
- **Controller** → gestisce l'input dell'utente → traduce gli input in operazioni sul Model;

Le motivazioni dell'utilizzo di questo pattern sono legate alla separazione delle responsabilità, dividendo:

- logica;
- interfaccia;
- input;

A sostegno di queste motivazioni c'è il cambiamento continuo delle interfacce e della rappresentazione dei dati, quindi: non si vogliono cambiare i dati, ma come essi vengono mostrati → cambiamento della GUI non deve riguardare la logica principale.

DESIGN PATTERN: STATE

Il **design pattern state** è un **design pattern comportamentale**, che **permette di alterare il suo comportamento quando il suo stato cambia**(INTENTO). In questo modo fa sembrare che l'oggetto ha cambiato classe, dato che all'esterno sembra che sia diventato un oggetto completamente diverso.

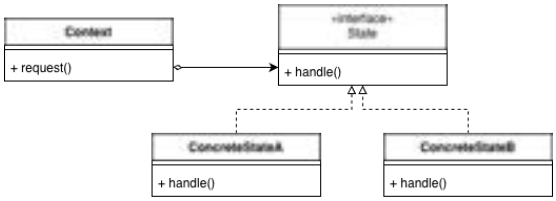
I problemi che affronta sono i seguenti(PROBLEMA):

- **comportamento dipende dallo stato dell'oggetto** → deve cambiare a run-time il proprio comportamento → ad esempio se una stampante ha i seguenti stati: pronta, occupata o in errore;
- **controllo degli stati(spesso lo stesso) sparso in vari metodi** → avere lo stesso if ripetuto ovunque → codice duplicato, sparso e difficile da mantenere → si rischia di incorrere in errori;

La soluzione proposta(SOLUZIONE) è di inserire in **ogni ramo condizionale in una classe separata**, in modo tale da evitare if ripetuti e avere direttamente il comportamento incapsulato di uno stato specifico. Per evitare la proliferazione inutile di classi, si utilizza quando ci sono grandi rami condizionali e sono molteplici.

I protagonisti principali di questo pattern sono:

- **context** → interagisce ai client e mantiene un'istanza di ConcreteState che definisce lo stato corrente → è utilizzato dal client → delega il comportamento allo stato corrente;
- **state** → è un'interfaccia che non rappresenta uno stato specifico bensì rappresenta tutto ciò che gli stati devono saper fare → serve per trattare tutti gli stati in modo uniforme;



- **concreteState** → sono tutte le classi che implementano ciascuno stato → definiscono il comportamento e possono determinare lo stato successivo.

CONTEXT

```
class Document{
    private State state;
    private String name;


    public Document(String s){
        state = new DraftState();
        name = s;
    }

    public void setState(State state){
        this.state = state;
    }

    public void publish(){
        state.publish(this);
    }

    public String getName(){return name;}
}
```

STATE

```
interface State{
    public void publish(Document doc);
}
```

CONCRETE STATE

```
class DraftState implements State{
    public void publish(Document doc){
        System.out.println("Il documento: "+ doc.getName()+" passa in revisione.");
        doc.setState(new ModerationState());
    }
}
```

CONCRETE STATE

```
class ModerationState implements State{
    public void publish(Document doc){
        System.out.println("Il documento: "+ doc.getName()+" passa in pubblicazione");
        doc.setState(new PublishedState());
    }
}
```

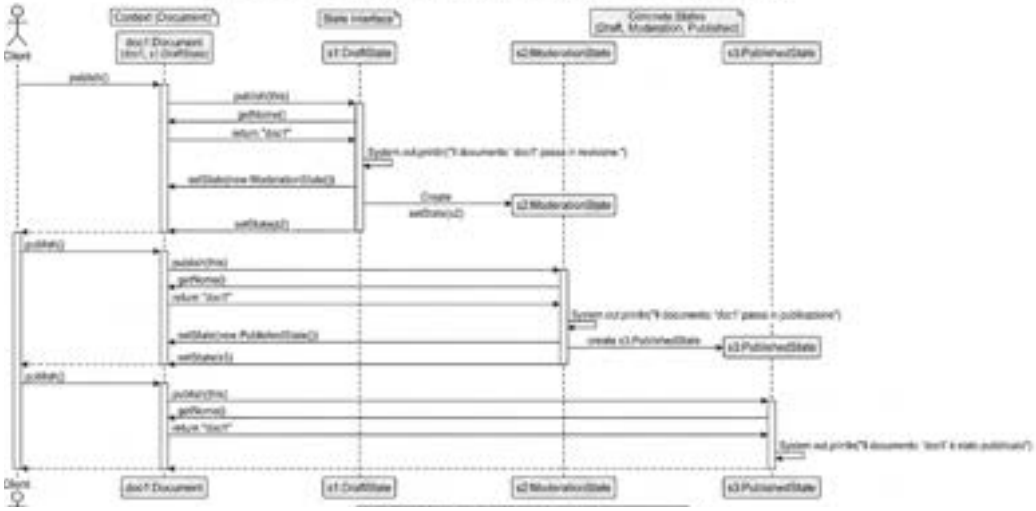
CONCRETE STATE

```
class PublishedState implements State{
    public void publish(Document doc){
        System.out.println("Il documento: "+ doc.getName()+" è stato pubblicato");
    }
}
```

Ogni stato mantiene un riferimento al contesto da cui può attingere informazioni e in questo specifico esempio lo stato viene cambiato nei concrete state, ma il prof suggerisce di affidare il cambiamento di stato al context. Nel context si ha un metodo per settare lo stato interno e un riferimento allo stato attuale → **setState()**.

Come si può osservare dal codice le richieste vengono passate agli stati, infatti le operazioni vengono implementate lì. In questo caso il metodo publish in context assume il valore di **request()** e invece nei concrete state funge da **handle()**.

Di seguito viene proposto il diagramma UML di sequenza dell'esercizio precedente:



I vantaggi all'utilizzo di questo design pattern sono(CONSEGUENZE):

- comportamento associato a uno stato è localizzato in una sola classe → riprendendo l'esempio di prima ogni stato gestisce il proprio publish();
- si evitano blocchi di if o switch infiniti e confusionari;
- si possono aggiungere nuovi stati facilmente → non si deve modificare una classe enorme e piena di condizioni;
- il cambiamento passa attraverso il context, vedasi il metodo setState → poi si può affidare il cambiamento di stato ai concrete state o ai client che utilizzano context;

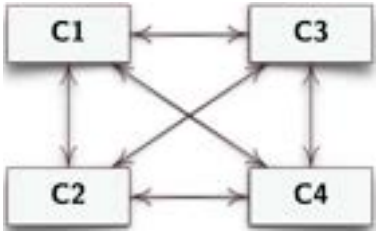
Lo svantaggio principale che si ha è il numero di classi che si viene a creare, vale a dire il costo di questa separazione.

DESIGN PATTERN: MEDIATOR

Il **mediator** è un **design pattern comportamentale**, che promuove il lasco(debole/allentato) accoppiamento fra oggetti perché evita che interagiscano direttamente.

Questo pattern promuove un mediatore tra oggetti, in modo tale da disincentivare la conoscenza totale tra classi, ergo **riduce la dipendenza many-to-many tra oggetti**(INTENTO).
Nell'esempio riportato a destra si ha:

C1 → C2	C2 → C1	C3 → C1	C4 → C1
C1 → C3	C2 →C3	C3 → C2	C4 → C2
C1 → C4	C2 → C3	C3 → C4	C4 → C3



Come abbiamo visto nell'esempio questo caso è il caso peggiore: un oggetto conosce tutti gli altri. Le molteplici connessioni rendono l'oggetto dipendente da altri e invece di promuovere una programmazione ad oggetti(riutilizzabili), si rischia di avere un **codice monolitico(MOTIVAZIONE)**.

Infine, la **logica del programma sta ovunque** e non è possibile vedere il quadro completo, dato che ad esempio:

- Button dice a TextBox di disabilitarsi;
- TextBox dice a CheckBox di aggiornarsi;
- CheckBox chiama Button...

Si possono evitare questi problemi **incapsulando il comportamento collettivo in una sola classe Mediator**, **dove si definiscono lì le regole** della comunicazione. In questo modo:

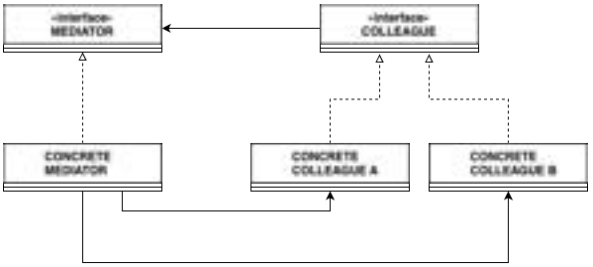
- Button → Mediator → TextBox;
- TextBox → Mediator → CheckBox;
- Checkbox → Mediator → Button...

Il pattern si utilizza quando:

- un insieme di oggetti comunicano in un modo ben definito ma complesso → gli oggetti collaborano davvero ma lo fanno in modo intricato;
- riutilizzo di un oggetto in contesti diversi non si può utilizzare dato che è legato a tanti oggetti → utilizzare il mediator permette di staccare l'oggetto dal sistema e riutilizzarlo in altri contesti;
- se si vuole cambiare l'interazione tra classi si devono cambiare le stesse → con il mediator si modifica una sola classe.

La soluzione che si utilizza è il Mediator che consiste nell'**isolare le comunicazioni tra oggetti in una classe separata**. Le comunicazioni diventano responsabilità del mediatore che decide cosa fare. All'interno del Mediator si implementa il comportamento cooperativo: come gli oggetti collaborano tra loro.

I protagonisti di questo pattern sono(SOLUZIONE):



- **Mediator** → definisce un'interfaccia, ed è il contratto comune attraverso cui tutti i colleghi comunicano → definisce le operazioni che i colleghi possono utilizzare per segnalare eventi;
- **ConcreteMediator** → implementa il comportamento cooperativo e coordina gli oggetti Colleague → decide chi deve reagire e in che ordine;
- **Colleague** → parte che interagisce con il Mediator e che delega la comunicazione allo stesso;
- **Concrete Colleague** → invia eventi al mediator e riceve azioni tramite lui.

I pro ottenuti dal Mediator sono utili a preferire quest'implementazione piuttosto che un'implementazione complessa e frammentata(CONSEGUENZE). I vantaggi sono:

- complessità spostata nel mediator e oggetti più semplici;
- colleague più riutilizzabili → non sono dipendenti agli altri oggetti;
- interazione tra oggetti semplice da modificare → basta modificare il concrete mediator;

Gli svantaggi principali sono:

- poca riusabilità del mediator → diventa specifico per l'applicazione;
- rischio di concentrare troppa logica in una sola classe → potrebbe venire meno i principi di singole responsabilità ecc.

MEDIATOR

```java
interface Mediator{ // MEDIATOR
    void requestLanding(Airplane plane);
}
```

CONCRETE MEDIATOR

```java
class ControlTower implements Mediator{ // CONCRETE MEDIATOR
    private boolean runwayBusy = false;

    public void requestLanding(Airplane plane){
        if(!runwayBusy){
            runwayBusy = true;

            System.out.println(plane.getName() + " autorizzato ad atterrare");
        } else{
            System.out.println(plane.getName() + " deve attendere");
        }
    }

    public void freeRunway(){
        runwayBusy = false;
        System.out.println("Pista libera");
    }
}
```

COLLEAGUE

```java
abstract class Airplane{ // COLLEAGUE
    protected Mediator mediator;
    protected String name;

    public Airplane(Mediator m, String n){
        mediator = m;
        name = n;
    }

    public String getName(){
        return name;
    }

    public void requestLanding(){
        mediator.requestLanding(this);
    }
}
```

CONCRETE COLLEAGUE

```java
class PassengerPlane extends Airplane { // CONCRETE COLLEAGUE
    public PassengerPlane(Mediator m, String n){
        super(m, n);
    }
}
```

In questo specifico esempio, il mediator è il centro ma i colleague non comunicano tra loro. Semplicemente il mediator coordina le operazioni.

DESIGN PATTERN: CHAIN OF RESPONSIBILITY

Il **chain of responsibility** è un **design pattern comportamentale** che **serve a passare una richiesta lungo una catena di oggetti**, finché uno di essi la gestisce.

Gli obiettivi del Chain of Responsibility sono(INTENTO):

- **evitare che chi manda la richiesta conosca direttamente chi la elabora**(si evita quindi l'alto accoppiamento);
- dare a più oggetti la possibilità di gestire la richiesta(INTENTO);
- far scorrere la **richiesta lungo una catena finché qualcuno se ne occupa**.

L'utilizzo è motivato da vari problemi che si hanno senza il suo utilizzo, ossia il forte accoppiamento tra mandante e ricevente, perché l'oggetto che invia la richiesta dovrebbe conoscere esplicitamente quale oggetto deve gestirla(MOTIVAZIONE).

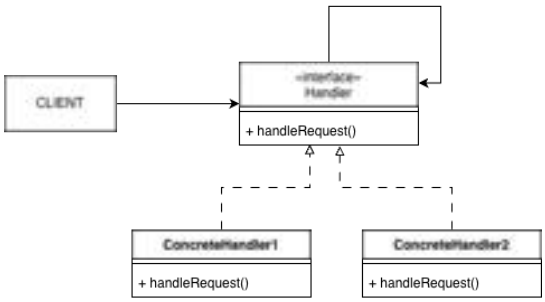
I motivi principali sono quindi: rigidità del codice, scarsa estensibilità e assenza di delega dinamica, ovvero che non sono gli oggetti stessi a propagare la richiesta.

Per risolvere questi problemi, il pattern organizza i possibili riceventi in una **catena di handler**, dove ogni oggetto:

1) verifica se può gestire la richiesta;
2) se può la elabora;
3) altrimenti la inoltra al nodo successivo.

I ruoli principali che si hanno all'interno di questo pattern sono(SOLUZIONE):

- **Handler** → definisce l'interfaccia comune per gestire le richieste, tipicamente tramite un'operazione **handleRequest()** → mantiene un riferimento al successore, ovvero al prossimo oggetto nella catena → definisce il meccanismo di propagazione della richiesta;
- **ConcreteHandler** → gestore concreto della catena → ogni ConcreteHandler conosce il proprio successore e coopera con gli altri della catena;



- **Client** → l'oggetto che genera la richiesta → non conosce quale oggetto la gestirà effettivamente, ma invia la richiesta al primo della catena → da quel momento saranno gli handler a propagare la richiesta fino a trovare chi la gestisce.

Quando si effettua la richiesta, si possono avere dei casi dove non esiste l'handler che la sappia gestire? Si, infatti, potremmo avere il caso in cui la richiesta attraversi la seguente catena H1→ H2→ H3 → NULL. In questo caso si possono intraprendere diverse strade:

- la richiesta fallisce;
- viene gestita da un default handler.

Nell'implementazione il successore viene specificato all'interno dell'Handler astratto(se è una classe astratta), in modo tale che tutti i ConcreteHandler ereditano la medesima logica.

Si può utilizzare ho il riferimento all'handler successivo oppure l'utilizzo di Chain + Composite(design pattern per utilizzare allo stesso modo oggetti e composizioni di oggetti) e quindi di una gerarchia esistente.

Una possibile implementazione del Chain of Responsibility è la seguente:

CLIENT

```java
public class ex30{ // CLIENT
    Run | Debug
    public static void main(String args[]){
        Approver tl = new TeamLeader();
        Approver m = new Manager();
        Approver d = new Director();


        tl.setSuccessor(m);
        m.setSuccessor(d);

        tl.handleRequest(amount: 70);
        tl.handleRequest(amount: 700);
        tl.handleRequest(amount: 3000);
        tl.handleRequest(amount: 7000);
    }
}
```

CONCRETE HANDLER 2

```java
class Manager extends Approver{ // CONCRETE HANDLER
    public void handleRequest(int amount){
        if(amount <= 1000){
            System.out.println("Manager approva " + amount);
        }else if(successor != null){
            successor.handleRequest(amount);
        }
    }
}
```

HANDLER

```java
abstract class Approver{ // HANDLER
    protected Approver successor;


    public void setSuccessor(Approver successor){
        this.successor = successor;
    }

    public abstract void handleRequest(int amount);
}
```

CONCRETE HANDLER 1

```java
class TeamLeader extends Approver{ // CONCRETE HANDLER
    public void handleRequest(int amount){
        if(amount <= 100){
            System.out.println("TeamLeader approva " + amount);
        }else if(successor != null){
            successor.handleRequest(amount);
        }
    }
}
```

CONCRETE HANDLER 3

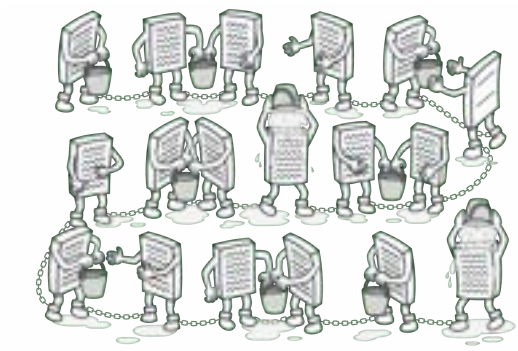
```java
class Director extends Approver{ // CONCRETE HANDLER
    public void handleRequest(int amount){
        if(amount <= 5000){
            System.out.println("Director approva " + amount);
        }else{
            System.out.println(x: "Richiesta non gestita");
        }
    }
}
```

Uno dei vantaggi principali del pattern è la **riduzione dell'accoppiamento tra l'oggetto che genera la richiesta e chi la gestisce effettivamente(CONSEGUENZE)**.

Un altro aspetto importante è che ogni oggetto della catena non ha una visione globale della struttura: bensì conosce solo il proprio successore.

Il pattern inoltre introduce una **notevole flessibilità nella gestione delle responsabilità**. È possibile modificare l'ordine della catena, aggiungere nuovi handler o rimuoverne alcuni a run-time.

Tuttavia questa flessibilità introduce uno svantaggio principale legato alla **non garanzia che la richiesta venga gestita**. Dato che non c'è un destinatario esplicito, la richiesta può attraversare tutta la catena senza trovare un oggetto in grado di elaborarla.



DESIGN PATTERN: COMMAND

Il **command** è un **design pattern comportamentale**, che si occupa di **incapsulare la richiesta di un oggetto** e decidere come: **gestirla, trasmetterla ed eseguirla**.

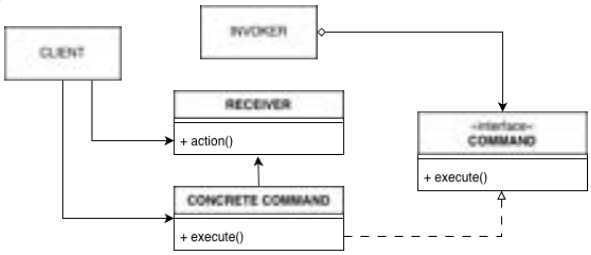
Questo pattern ha come obiettivo l'**incapsulamento della richiesta in un oggetto** → si costruisce un oggetto che rispecchia la richiesta(INTENTO). L'azione non è quindi più legata ad una chiamata diretta, bensì diventa qualcosa che può essere trattata come un oggetto.

Le motivazioni a seguito di questo pattern sono le seguenti(MOTIVAZIONE):

- a volte si deve inviare una richiesta senza sapere: quale operazione verrà fatta, chi la farà e/o come verrà realizzata;
- disaccoppiare invocatore e ricevente, in questo modo si ha: invocatore → command → ricevente;
- l'idea può essere trattata come oggetto;
- **command** definisce un metodo comune per l'esecuzione → le sottoclassi specificano ricevente e azione(richiesta)

I ruoli che collaborano nel Command sono i seguenti(SOLUZIONE):

- **Command** → astrazione della richiesta → definisce l'**interfaccia comune che tutti i comandi devono offrire** → contiene il metodo **execute**();
- **Concrete Comand** → è il comando concreto e **implementa** una richiesta specifica → contiene **Receveir + azione** la coppia che identifica il comando → **ponte tra Invoker** e Receiver → chiama le operazioni sul Receiver;
- **Client** → **configura i pezzi** → associa al concrete command un receiver;



- **Invoker** → **chiede al command di effettuare una richiesta** → è ignorante su: cosa fa il comando, chi lo esegue e/o come funziona;
- **Receiver** → chi **esegue direttamente la richiesta** → non conosce il command (dipendenza monodirezionale).

Nel caso in cui l'azione sia reversibile, Concrete Command conserva lo stato necessario ad implementare undo(), in modo tale da annullare gli effetti di execute().

CLIENT & INVOKER all'interno della stessa classe

```java
public class ex31{
    Run | Debug
    public static void main(String args[]){
        Editor editor = new Editor();

        Command copy = new CopyCommand(editor);
        Command paste = new PasteCommand(editor);
        Command cut = new CutCommand(editor);

        copy.execute();
        paste.execute();
        cut.execute();
    }
}
```

Client

Invoker

RECEIVER

```java
class Editor{
    public void copy(){
        System.out.println(x: "Testo Copiato");
    }

    public void paste(){
        System.out.println(x: "Testo Incollato");
    }

    public void cut(){
        System.out.println(x: "Testo Tagliato");
    }
}
```

COMMAND

```java
interface Command{
    public void execute();
}
```

CONCRETE COMMAND1

```java
class CopyCommand implements Command{
    private Editor editor;
    public CopyCommand(Editor e){
        editor = e;
    }

    public void execute(){
        editor.copy();
    }
}
```

CONCRETE COMMAND 2

```java
class PasteCommand implements Command{
    private Editor editor;
    public PasteCommand(Editor e){
        editor = e;
    }

    public void execute(){
        editor.paste();
    }
}
```

I vantaggi legati all'utilizzo del Command sono i seguenti(CONSEGUENZE):

- **Disaccoppiamento dell'oggetto che invoca la richiesta da chi la esegue realmente** e sa come eseguirla. Così non c'è un accoppiamento forte → obiettivo principale dell'ingegneria del software. Come si trasforma in vantaggio? → **Si può cambiare il Receiver senza che l'Invoker cambi a sua volta**.
- La **richiesta diventa un oggetto** e quindi **può essere trattata come un dato** → si può utilizzare in una lista di log e/o metterla in una coda di gestione delle richieste.
- Si **possono comporre i comandi e creare un macro-comando** → possiamo utilizzare il design pattern composite per la composizione.
- Si possono aggiungere nuovi comandi senza modificare Invoker/Client/Receiver → **Open/Closed Principle**.

Gli svantaggi principali sono: l'esplosione delle classi piccole e la complessità architetturale introdotta a seguito delle chiamate interne.

Per questo motivo è suggerito applicare questo pattern a problemi di medio-grandi dimensioni.

DESIGN PATTERNS

Quando si decide di **progettare per preservare** in ingegneria del software, l'idea alla base è di costruire **sistemi che rimangano stabili** nel tempo anche quando cambiano: i requisiti, le tecnologie o le decisioni progettuali.

Non si tratta soltanto di far funzionare il programma, ma di poter aver la possibilità di modificarlo, estenderlo e riutilizzarlo con il minimo impatto sul codice esistente.

Il **problema principale** nei sistemi software è che se c'è **alto coupling tra le classi**, ogni cambiamento tende a propagarsi involontariamente a cascata. Questo porta ad avere sistemi rigidi, difficili da mantenere e costosi da evolvere.

Una **buona progettazione mira a ridurre le dipendenze**, introducendo livelli di astrazione e separando le responsabilità.

Uno dei **problemi** più comuni è la **creazione esplicita degli oggetti tramite le classi concrete**. Quando il codice istanzia direttamente una classe, si lega ad una specifica implementazione. Per evitare questo legame forte tra l'oggetto e chi lo usa, si delega la creazione di un oggetto ad una classe preposta.

Un esempio di patterns utilizzati per questo specifico problema sono:

- **Factory method** → separa chi crea l'oggetto da chi lo utilizza;
- **Prototype** → instazia oggetti con specifici stati attraverso il metodo clone.

L'**obiettivo di questi pattern è di lavorare su astrazioni e non su implementazioni** dirette.

Un altro **problema** si verifica quando la **logica per gestire richieste è scritta direttamente nel codice**, tramite mega blocchi condizionali e/o operazioni fisse. Questo **impedisce di avere un sistema non scalabile e difficile da modificare**, dato che chi fa la richiesta e chi la soddisfa non sono separati totalmente.

La **soluzione è trasformare le richieste in oggetti** oppure farle scorrere in liste dinamiche di classi di handler che le gestiscono.

I pattern utilizzati sono:

- **Chain of responsibility** → scorre la richiesta in una lista di handler → uno tra essi la potrà gestire(non è garantito);
- **Command** → trasformare la richiesta in oggetto → sa chi deve soddisfare la richiesta e cosa deve fare.

Molti sistemi dipendono da algoritmi che evolvono nel tempo e se questi stessi algoritmi sono mescolati con la logica delle classi, i cambiamenti potrebbero essere costosi.

La soluzione è isolare gli algoritmi dalla logica delle classi. Patterns che seguono questo principio sono:

- **State** → separa il comportamento dallo stato. Invece di concentrare in una classe grandi blocchi condizionali che cambiano comportamento in base allo stato interno, si incapsula ogni stato in un oggetto separato. Quando cambia lo stato, cambia anche il comportamento dell'oggetto, senza modificare il client;
- **Template Method** → isola le parti variabili di un algoritmo mantenendone fisso lo scheletro generale. La struttura dell'algoritmo rimane stabile, mentre alcuni passi vengono ridefiniti nelle sottoclassi.
- **Builder** → separa l'algoritmo di costruzione di un oggetto complesso dalla sua rappresentazione finale, permettendo di cambiare il processo costruttivo senza modificare il prodotto.

Il beneficio principale che si ottiene è che il cambiamento dell'algoritmo non implica il cambiamento degli oggetti che lo utilizzano.

Si **deve promuovere un lasco accoppiamento**, infatti, lo stretto accoppiamento porta a sistemi monolitici → modificare una classe inevitabilmente ne modifica altre.

I loose coupling permette di avere dei vantaggi in termini di: riusabilità, estendibilità e riduzione della propagazione dei cambiamenti.

Molti patterns promuovono il lasco accoppiamento:

- **Factory** → separa creazione e utilizzo degli oggetti.
- **Bridge** → separa astrazione e implementazione.
- **Chain of Responsibility** → separa chi fa una richiesta da chi la gestisce.
- **Command** → separa chi invoca un'azione da chi la esegue.
- **Facade** → riduce dipendenze nascondendo complessità.
- **Mediator** → centralizza interazioni tra oggetti.
- **Observer** → separa soggetti e osservatori tramite notifiche.

Estendere funzionalità tramite sottoclassi è **white-box reuse**: un comportamento che richiede conoscenze della sopraclasse e dipende fortemente dall'implementazione.

I problemi, quindi, sono i seguenti: forte dipendenza dalla superclasse, override complessi e esplosioni di sottoclassi.

Per questo è preferibile favorire la composizione rispetto all'ereditarietà(black-box reuse). I pattern che preferiscono la composizione sono:

- **Decorator** → aggiunge responsabilità dinamicamente;
- **Composite** → gestisce strutture ad albero;
- **Chain of responsibility** → estende il comportamento aggiungendo handler.

Le nuove funzionalità si aggiugnono componendo oggetti invece di creare nuove gerarchie.

Potrebbe essere difficile modificare il codice esistente quando si ha:

- Codice legacy;
- Librerie esterne;
- Framework di terze parti;

In questi casi esistono dei pattern che permettono di estendere senza modificare. I protagonisti principali sono:

- **Adapter** → adatta un'interfaccia esistente ad una nuova richiesta;
- **Decorator** → aggiunge comportamento senza modificare il codice originale.

Si estende il sistema rispettando il principio **Open/Closed Principle**: aperto all'estensione e chiuso alla modifica.

In pratica "progettare per preservare" significa progettare oggi pensando ai cambiamenti di domani.

EVOLUZIONE DEL SOFTWARE E METRICHE

Il ciclo di vita di un software non termina quando il software stesso viene consegnato al cliente, ma appunto dopo il rilascio inizia la fase più costosa e lunga della sua vita: la **manutenzione**.

I requisiti cambiano, emergono errori, l'ambiente tecnologico evolve e/o gli utenti chiedono nuove funzionalità. Per questo motivo il **software non è un prodotto statico**, ma un prodotto soggetto a continue trasformazioni.

Il processo di introduzione di modifiche a un prodotto software dopo la consegna al cliente è detto **manutenzione**. La maggior parte dei costi di un sistema software è incentrato nella manutenzione, infatti si stima che i costi si aggirino intorno al 67%-70%.

La manutenzione può essere di vari tipi:

- **Correttiva** → correzione degli errori → costi di manutenzione si aggirano al 17%;
- **Adattiva** → serve ad adattare il software a cambiamenti esterni → cambiamenti del SO, hardware, ecc. → costi di manutenzione circa 18%;
- **Perfettiva** → migliorare o estendere il sistema → costi di manutenzione circa 60%;
- **Preventiva** → serve ad evitare problemi futuri → costi di manutenzione circa 5%.

La maggior parte dei costi sono impiegati non per riparare il software, ma per farlo evolvere. Per questo motivo è buona pratica anticipare i cambiamenti a **design time**. Applicare le regole imparate dai design patterns(disaccoppiamento, incapsulamento e parametrizzazione) potrebbe essere utile a rendere il nostro codice flessibile ai cambiamenti.

Il termine evoluzione può essere utilizzato con il significato di: manutenzione o modifiche che prevedono l'inserimento di nuove feature.

Le **dinamiche di evoluzione** sono i processi di cambiamenti di un sistema software durante la sua vita.

Come si comportano i sistemi quando evolvono nel tempo? Lehman e Belady hanno studiato empiricamente l'evoluzione di software complesso sviluppato in grandi organizzazioni.

Da queste osservazioni sono nate le **leggi di Lehman**, cioè regolarità osservate nell'evoluzione del software. Queste leggi sono:

1) **Cambiamento continuo** → un sistema software deve essere continuamente adattato → non potrebbe più soddisfare ai clienti, diventa meno utile o perde aderenza ai requisiti.
2) **Aumento della complessità** → quando un sistema evolve aumenta la complessità → si deve preservare la semplicità attraverso: refactoring, redesign, ristrutturazione e design pattern.
3) **Auto-regolazione** → alcuni attributi del sistema tendono a rimanere stabili → numero di errori trovati per release, intervallo tra release e dimensione degli incrementi → l'evoluzione non cresce in modo arbitrario, ma mostra comportamenti regolari.
4) **Stabilità organizzativa** → anche aumentando persone e risorse il tasso di sviluppo tende ad essere costante.
5) **Conservazione di familiarità** → l'incremento di crescita tende ad essere costante o diminuire → gli sviluppatori devono rimanere familiari con il sistema, perché se cresce rapidamente: diventa incomprensibile, si aumenta il rischio di errori e cala la controllabilità.
6) **Continua crescita** → per risultare appetibile, il contenuto funzionale (comportamento) del software deve crescere continuamente → si potrebbe perdere utenza se viene a mancare questa crescita.
7) **Diminuzione della qualità** → se il software non viene gestito e adattato la qualità diminuisce → decadimento del software.

Le leggi quindi ci segnalano che il software tende naturalmente a: dover cambiare, diventare più complesso, crescere; per questo il software deve essere progettato per preservare ed essere continuamente manutenuto.

Le **leggi di Lehman** sopraccitate sono in generale applicabili in sistemi di grandi dimensioni; infatti, **non è chiaro quando si adattino ad altri contesti**. Ad esempio, nei sistemi piccoli: il codice è molto più contenuto, le dipendenze sono minori e l'evoluzione può essere meno complessa.

Quindi fenomeni osservati in grandi sistemi software, probabilmente non possono essere generalizzati ad altri casi che hanno caratteristiche diverse.

Ad esempio, in sistemi che integrano un certo numero di COTS l'evoluzione dipende anche da soggetti esterni. Le dinamiche previste da Lehman sono alterate rispetto ai casi su cui sono state studiate.

Il costo della manutenzione non dipende solo dal numero di modifiche, dipende anche da altri fattori tecnici e organizzativi. Questi fattori sono i seguenti:

- **Costo ridotto se il team di sviluppo è impiegato nella manutenzione** → se agiscono esterni il costo potrebbe lievitare → inoltre il tempo impiegato è minore dato che conoscono il sistema;
- **Mancanza di incentivi a progettare per il cambiamento** → se i programmatori non hanno responsabilità contrattuali sulla manutenzione, possono avere pochi incentivi a progettare pensando al futuro → debito tecnico.
- **Man mano che si introducono nuovi cambiamenti la struttura software si degrada** → per questo motivo ogni cambiamento diventa sempre più costoso → si perde la semplicità che si all'inizio nelle prime release.

Un cambiamento software non viene gestito sempre allo stesso modo; infatti, esistono diversi tipi di **modelli di manutenzione**, che rappresentano diversi approcci per modificare ed evolvere un sistema.

Questi modelli variano per profondità di modifica, controllo della qualità e visione a lungo termine. I principali modelli di manutenzione sono:

- **Quick-fix** → approccio immediato e reattivo → modifiche fatte direttamente a livello di codice spesso per risolvere un problema specifico. Intervento rapido ed economico che mira a risolvere il problema e non la struttura complessiva. Il problema principale è legato alla qualità del codice prodotto che potrebbe aumentare la complessità.
- **Miglioramento iterativo** → le modifiche vengono fatte a partire da un'analisi del sistema esistente → ogni cambiamento viene valutato nel contesto dell'intera struttura. → limita la degradazione architetturale e riduce costi futuri ma richiede tempo e risorse.
- **Riuso** → invece di modificare interamente il sistema si analizza cosa si può riutilizzare → aumenta la produttività e riduce lo sviluppo di nuovo software però si hanno vincoli posti dai componenti riutilizzati e un adattamento complesso.

Quando si apportano modifiche, non solo vengono aggiunte nuove funzionalità ma si interviene nella struttura stessa del sistema per comprenderlo, migliorarlo o trasformarlo. I principali tipi di modifica sono:

- **Refactoring/ Restructuring** → si migliora la struttura interna del codice e non gli obiettivi;
- **Reverse Engineering** → studio del sistema per estrarre informazioni sulla struttura o sul suo comportamento → non si modifica ma si studia;
- **Re-engineering** → alterare il sistema per ricostruirlo in un'altra forma → si trasforma il sistema → obiettivo rigenerare il sistema in una forma migliore → migrare da COBOL a Java.

Le metriche vengono impiegate in ingegneria del software per quantificare proprietà del software che altrimenti sarebbe valutate in modo soggettivo. Una **metrica** definisce un **insieme di misure per descrivere le caratteristiche di un sistema software**.

Gli obiettivi principali dell'adozione di metriche sono:

- monitoraggio mentre il sistema si costruisce → servono ad osservare lo stato del software durante lo sviluppo: crescita del codice, complessità e copertura test;
- identificare dei livelli/soglie per ciascuna metrica → livelli funzionano come indicatori di qualità;
- rimediare se i livelli non sono sufficienti → intervento → metriche a supporto di decisioni;

Gli **attributi** osservabili sono solo quelli **interni**, ovvero **proprietà osservabili all'interno del software**. Esempi di attributi interni sono:

- dimensione del codice;
- numero classi;
- accoppiamento;
- complessità.

Gli **attributi esterni** sono **proprietà percepite sul comportamento del sistema**, che non possono essere misurate direttamente. Sono proprietà più astratte e potrebbero essere:

- affidabilità;
- qualità;
- manutenibilità.

Potrebbe essere utile analizzare la **complessità ciclomatica**(cc) che indica il numero di percorsi indipendenti che si possono seguire. La cc si può calcolare in modo pratico in questo modo:

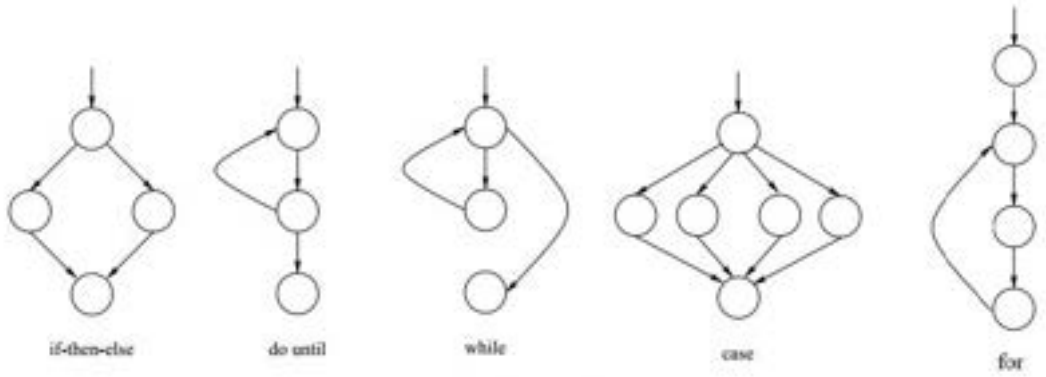
$$cc = numero\ di\ decisioni + 1$$

dove ogni decisione corrisponde a: if, while, for, case, cath.

Nella forma più tradizionale il calcolo di cc avviene attraverso la seguente formula:

$$cc = numero\ archi - numero\ nodi + 2$$

Nell'immagine sottostante sono presenti degli esempi legati ai vari costrutti.



Altre metriche misurate direttamente dal software sono:

- **LOC(linee of code)** → l'insieme di tutte le righe contenenti: istruzioni, commenti e dichiarazioni. Questo è un buon indice su quanto il nostro codice sia manutenibile e soggetto ai bug;
- **NCNB(Not Code Not Blanck)** → variante precisa di LOC che non prende in considerazione commenti e righe vuote;
- **Comment Percentage** → indica quanto il codice è commentato internamente → è la percentuale di commenti e la soglia massima da raggiungere è il 30%.

Oltre alle metriche tradizionali, esistono un insieme di **metriche per valutare la qualità di sistemi object-oriented** ossia le **metriche di Chidamber e Kemerer**.

Esse servono a misurare:

- complessità delle classi;
- riusabilità;
- accoppiamento;
- coesione;
- profondità della gerarchia.

In pratica misurano quanto una classe è buona dal punto di vista del design.

Tra le metriche proposte c'è la WMC o Weighted Method per Class, **che misura la complessità dei metodi per una classe**. La formula è la seguente:

$$WMC = \sum_{i=1}^{n} c_i$$

dove n è il numero dei metodi e c è la complessità del metodo i-esimo.

Provando ad interpretare i valori che assume questa metrica si ha che:

- se WMC è alto → classe complessa → più difficile da mantenere e testare;
- se WMC è basso allora → classe semplice → riutilizzabile.

Un'altra metrica riguarda l'ereditarietà, infatti: la metrica **DIT** o **Depth of Inherance Tree** misura la profondità nell'albero dell'ereditarietà.

Una classe può ereditare da classi che a loro volta ereditano; questo indica un **buon riuso del codice**(dato che si sfrutta l'ereditarietà) ma anche una complessità maggiore e una non facile interpretazione.

- DIT più alto → maggiore ereditarietà;
- DIT più basso → minore ereditarietà(radice = 0);

Sempre soffermandoci sull'ereditarietà possiamo anche parlare della metrica **NOC**(**Number of Child)**, che misura il numero di sottoclassi dirette di una classe. Un **valore alto di NOC** indica una **maggiore influenza** della classe e un buon riutilizzo, ma anche maggiore responsabilità nel mantenimento.

Il problema che si vuole affrontare mediante l'utilizzo di pattern e principi di ingegneria del software è l'accoppiamento alto tra classi. Una metrica CK utilizzata per misurare l'accoppiamento tra classi è la metrica **CBO**(**Coupling Between Object Classes)**.

Questa metrica ci permette di calcolare il **numero di classi accoppiate con la classe presa in considerazione**. Un valore maggiore di CBO potrebbe indicare: difficoltà nelle modifiche, nel riuso e una forte dipendenza.

Una metrica altrettanto importante è la metrica **RFC** (**Responses For a Class)**. Questa metrica permette di sapere **quanti metodi vengono richiamati quando viene fatta una richiesta alla classe**.

Infatti, l'idea è di calcolare il numero di metodi chiamati se viene effettuata una chiamata anche ad un singolo metodo. Ipotizzando di avere una classe DB e i seguenti metodi: insert(), save() updateView() e insertQuery(). Se insertQuery al proprio interno inglobasse tutti gli altri metodi, la classe DB avrebbe come RFC il valore 4.

Con un RFC alto, la classe per ogni singola richiesta: scatena molte operazioni e ha comportamenti imprevedibili.

L'ultima metrica è una delle più sottili, dato che calcola **quanto una classe è coesa**. Questa metrica è importante soprattutto per capire se la classe presa in questione ha una singola responsabilità o molteplici.

La metrica in questione è la metrica **LCOM**(**Lack of Cohesion of Methods**), che grazie alla seguente formula calcola quanto una classe è coesa:

$$\mathbf{LCOM} = \mathbf{1} - \frac{\sum_{\mathbf{i}}^{\mathbf{a}} \mathbf{m_{Ai}}}{\mathbf{m} * \mathbf{a}}$$

dove:

- **a** è il **numero di attributi**;
- **m** è il **numero di metodi**;
- M_{Ai} è il **numero di metodi che utilizzano l'attributo Ai**.

Infine, se il valore di LCOM è basso allora la classe è molto coesa e sta lavorando bene, altrimenti male. Infatti, se il valore di LCOM è alto significa che:

- metodi condividono pochi attributi;
- fanno cose indipendenti;
- la classe ha troppe responsabilità.

DIFETTI E TEST

Il software con i difetti non è l'eccezione ma la norma, infatti i bugs sono inevitabili perché:

- il software è complesso;
- gli sviluppatori possono commettere errori;
- i requisiti possono essere ambigui o incompleti;

Come facciamo a capire che c'è un problema nel nostro sistema software? (Si intende non errore di compilazione)

Serve un riferimento esterno e questo riferimento esterno è:

- requisito non rispettato;
- comportamento anomalo.

La specifica dei requisiti descrive:

- cosa il sistema deve fare;
- come deve comportarsi;
- quali risultati deve produrre;

Un difetto esiste solo rispetto ad una specifica, altrimenti non possiamo dire che qualcosa è giusto o sbagliato.

Come summenzionato facciamo una distinzione netta tra Verifica e Validazione, anche se l'obiettivo delle due attività è assicurarsi che il software soddisfi i bisogni dell'utente.

Il quesito principale che si pone la verifica è: "**Stiamo costruendo il prodotto nel modo giusto?**". Attraverso la verifica possiamo controllare che il software: sia conforme alle specifiche, sia implementato correttamente e vengano rispettati i requisiti definiti.

L'attenzione è quindi posta nel confronto Codice/Sistema → Specifica. **La verifica controlla la correttezza del codice rispetto alla specifica**.

La domanda che è centrale nella validazione è: "**Stiamo costruendo il prodotto giusto?**". Attraverso la validazione analizziamo se: il software soddisfa i bisogni dell'utente, risolva il problema corretto o se sia realmente utile nel contesto reale.

L'attenzione in questo caso è posta al confronto Codice/Sistema → Bisogni dell'utente. **La validazione analizza la correttezza rispetto ai bisogni reali**.

Le fasi di Verifica e Validazione non sono fasi finali, bensì si applicano continuamente. Queste fasi si applicano a:

- requisiti;
- progettazione;
- implementazione;
- testing.

Si applicano continuamente perché prima viene scoperto un errore e meno costosa è la riparazione. In base a quando nascono i difetti possiamo classificarli come:

- **difetti di specifica** → errore nei requisiti → requisiti ambigui, incompleti, imprecisi o contradittori → errori a cascata nelle fasi successive;
- **difetti di progettazione** → errore durante il design del sistema → struttura inadatta, tipo errato o campo mancante → interfacce incompatibili;
- **difetti di codice** → errore introdotto durante la fase di codifica → le cause sono legate all'incomprensione del progetto o errori nel linguaggio → conversioni di tipo sbagliate, mancata assegnazione, inutilizzo delle variabili o overflow → design corretto ma implementato male.
- **difetti di test** → casi di test incompleti o sbagliati → difetti reali non possono essere rilevati.

I test possono dimostrare la presenza di errori e non la loro assenza. Questo perché non si possono testare tutti i possibili input e coprire tutti i percorsi.

Anche se è un controsenso, un test ha successo quando trova gli errori perché l'obiettivo della fase di testing è trovare gli errori.

Ai test viene affiancata una verifica statica, ossia una code review che potrebbe rilevare altri errori non trovati dai test(verifica dinamica).

La **fase di debugging** è la fase successiva dove: si capisce l'errore, lo si individua e si sistema. Durante la fase di debugging si segue il seguente filo logico:

- formulare le ipotesi → capire dove potrebbe essere l'errore e capire cosa lo causa;
- verificare le ipotesi → eseguire il test e analizzare il comportamento;
- trovare e correggere l'errore → individuare la causa, modificarla e verificare se la correzione è andata a buon fine.

Per effettuare i test serviranno dei dati in input, chiamati **dati di test**. Questi dati verranno immessi in input e attraverso l'output ci accorgeremo se la funzione fa ciò che è specificato nei requisiti.

Se oltre all'input abbiamo anche degli **output attesi**, si può parlare di **caso di test(test case)** ossia un test più completo. In questo caso, il test case serve a verificare se l'output reale è uguale a quello atteso.

Un insieme di test cases è chiamato test suite e ad esempio potrebbe essere utilizzato nel seguente modo:

- test numeri positivi;
- test numeri negativi;
- test zero;
- test overflow.

Solo un **test esaustivo** può garantire che il programma sia privo di difetti, ma questo vorrebbe dire testare tutti i possibili percorsi con tutti i possibili input. Questo è inutile e impraticabile dato che ci vorrebbero anni a testare un programma con tutti i possibili input.

Il **testing** quindi deve essere **selettivo**, per meglio dire si deve decidere prima cosa testare secondo criteri validi.

Di seguito sono riportati dei principi di applicazioni di testing:

- si deve testare non per singolo comportamento ma per capire cosa il sistema fa e analizzare i comportamenti completi;
- le funzioni già esistenti(anche quelle molto vecchie) devono essere sempre testate, perché ogni modifica può rompere qualcosa che prima funzionava → test delle vecchie funzioni > test delle nuove funzioni;
- massimizzare l'efficacia dei test → si scelgono input rappresentativi e si coprono casi frequenti.

Se si trovano molti difetti in un componente è possibile che ce ne siano molti altri ancora, questo perché i difetti tendono a raggrupparsi. Il motivo può essere un design complesso e una logica fragile.

I test che vengono sviluppati devono essere ripetibili e riusabili → **regression testing** → a seguito di modifiche bisogna verificare che tutto funzioni correttamente anche su test precedentemente eseguiti.

I test possono essere fatti seguendo due approcci:

- **black-box testing(scatola nera)** → si testa il codice senza sapere il sistema interno → si danno dati in input e si osservano gli output → si basa sulle specifiche di sistema: requisiti, comportamenti attesi e documentazione → si può progettare prima dell'implementazione e trova errori rispetto ai requisiti ma la copertura totale non è assicurata.
- **white-box testing(glass-box)** → si testa il codice conoscendo il sistema → si basa sulla conoscenza interna del programma → vantaggi legati alla copertura del codice ma bisogna conoscere l'implementazione e non verifica automaticamente se il requisito è soddisfatto.

I due approcci non sono uno l'alternativa dell'altro bensì sono complementari, poiché: black-box testing testa se il codice soddisfa i requisiti mentre white-box testing verifica l'implementazione.

Quando si fanno i test, soprattutto in black-box, si dividono tutti i possibili input in classi, i cui valori all'interno della stessa classe si comportano allo stesso modo. Questa tecnica è chiamata partizionamento in **classi equivalenti**, che permette di testare un solo rappresentante per ogni classe → si riduce drasticamente il numero di test senza perdere efficacia.

Si testano anche e soprattutto i limiti perché spesso gli errori si trovano ai bordi → **valori boundary(di confine)**. Inoltre, una **buona pratica è testare il programma per classi non valide(input errati e casi anomali)**, così da verificare la corretta gestione da parte del sistema.

La finalità è coprire ogni percorso indipendente all'interno del sistema, in modo da verificare la correttezza dell'intero programma. Si può rappresentare il programma tramite un diagramma un diagramma di flusso, dove:

- **nodi rappresentano le condizioni di un programma**;
- **archi rappresentano il flusso di controllo** → passaggio da un'istruzione all'altra;

La complessità ciclomatica indica il numero di percorsi indipendenti da testare. La formula per calcolare la complessità è la seguente: $\boldsymbol{cc = numero\ di\ archi - numero\ di\ nodi + 2}$

Gli **stress test** consistono nell'eseguire il sistema oltre i limiti operativi previsti per osservare:

- come si comporta sotto pressione il sistema;
- se emergono difetti nascosti.

La copertura è la percentuale di codice eseguito durante l'esecuzione dei test e la formula per calcolarla è la seguente:

$$\boldsymbol{Coverage} = \frac{\boldsymbol{unit\grave{a}\ eseguite}}{\boldsymbol{unit\grave{a}\ totali}} \cdot \mathbf{100}$$

L'unità sono parti del programma e a seconda del tipo di copertura richiesta, possono essere:

- funzioni → percentuale di funzioni eseguite → superficiale, si verifica se si entra nelle funzioni;
- istruzioni → percentuale di istruzioni eseguite → ancora limitata;
- rami → percentuali di rami delle decisioni eseguiti → ogni decisione è testata →molto più forte della copertura delle istruzioni;
- archi → legato al grafo di flusso → coprire tutti i passaggi tra nodi;
- condizioni → verifica che ogni sotto espressione booleana sia stata verificata sia come true e sia come false → ad esempio: if(x && y) si deve veficare x = true y = true, x = false, y = false;

Una **copertura di funzioni e di rami** è chiamata **copertura di decisioni** perché:

- si testa ogni punto di ingresso e uscita del programma;
- ogni decisione è valutata;
- ogni possibile esito è testato;

Quindi se scegliessimo la copertura per righe di codice e i test ne eseguono 70, la copertura è del 70%.

Si definisce una soglia di percentuale di coverage da raggiungere e si testa fino a raggiungerla. La percentuale da raggiungere si sceglie sulla base di:

- rischio;
- contesto;
- criticità del sistema.

Ad esempio, in sistemi dove c'è in gioco la salute delle persone o pagamenti si desidera una sicurezza alta, ergo la scelta della soglia deve essere alta.

In sistemi dove c'è in mezzo l'UI o il logging la coverage è più bassa dato che alcuni input sono difficili da simulare.

Una condizione di stop per il testing è l'utilizzo della misura **bug trend**, ossia una misura che ci dice quanti difetti vengono trovati nel tempo durante il testing.

Se il valore della misura diventa stabile allora potremmo scegliere di fermarci, ovviamente il numero di errori deve essere minimale.